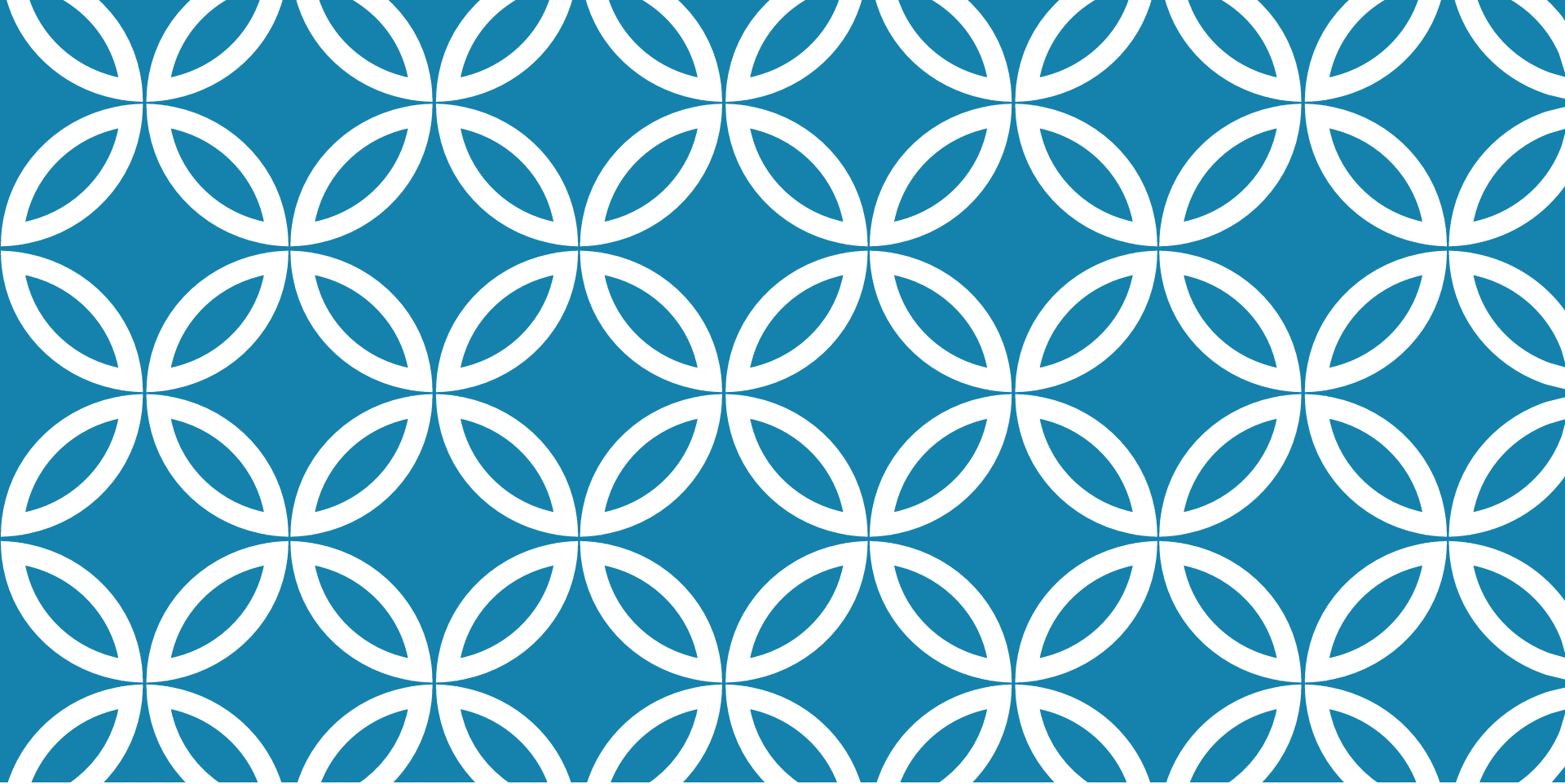




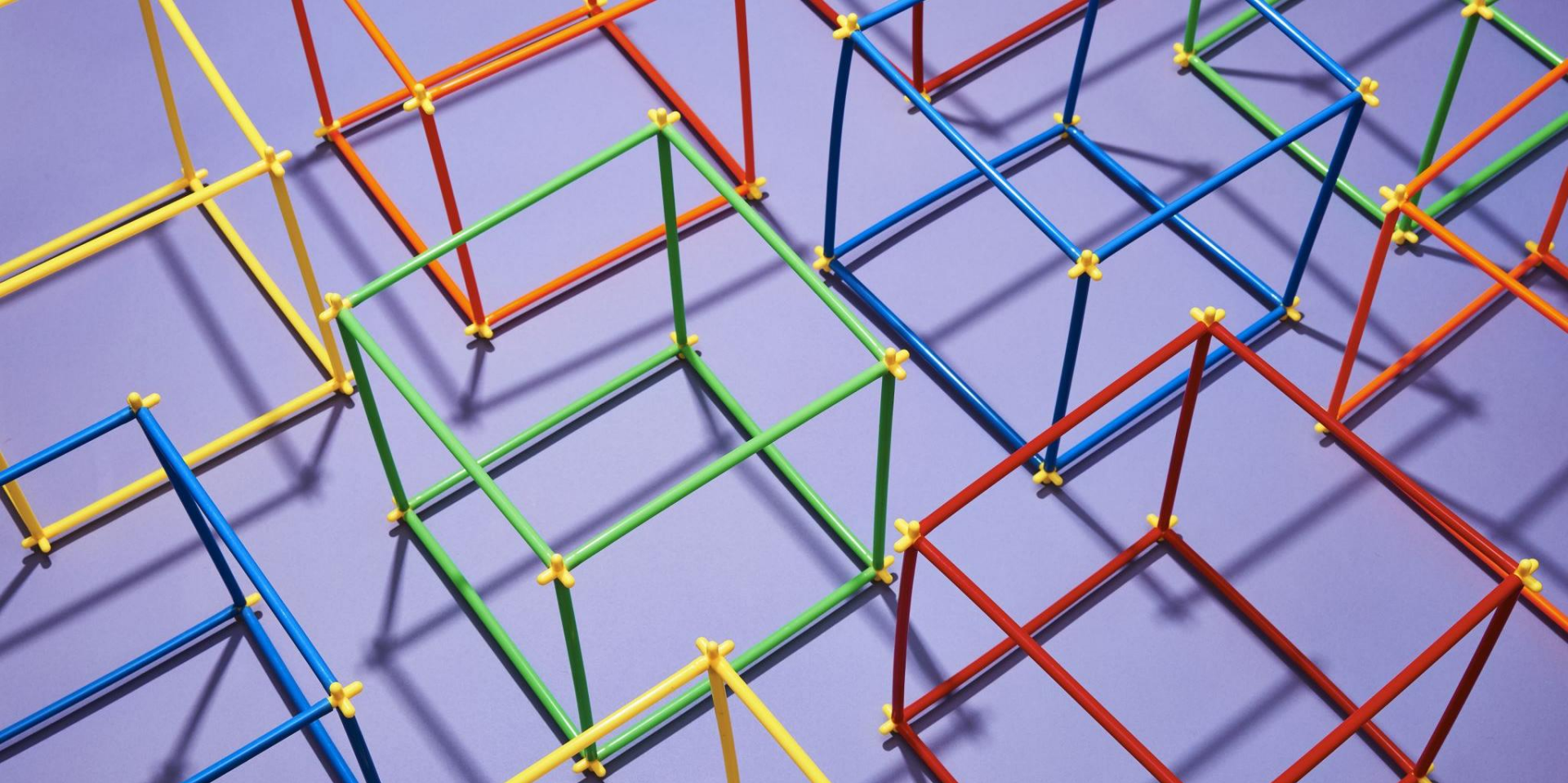
ANÁLISE PREDITIVA

Ciência de Dados para Negócios
FATEC Votorantim

Prof. **Luis Claudio**

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)



| — O QUE É ANÁLISE DE DADOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)

CONCEITO DE ANÁLISE DE DADOS

Análise de Dados (Data Analysis)

- “Etapa de **exame sistemático de valores observados** com o objetivo de **extrair informação relevante.**” (*ISO 3534-1 — Vocabulary and symbols*)
- Dados
 - "Representação reinterpretabil de informação de forma formalizada, adequada para comunicação, interpretação ou processamento." (*ISO 8000-2; ISO/IEC 2382*)
 - “Coleção de fatos, observações ou registros sobre eventos, entidades ou comportamentos.”**

TIPOS DE ANÁLISE DE DADOS

A análise de dados pode assumir diferentes formas, dependendo do **tipo de pergunta que se deseja responder** e do nível de **maturidade analítica da organização**.

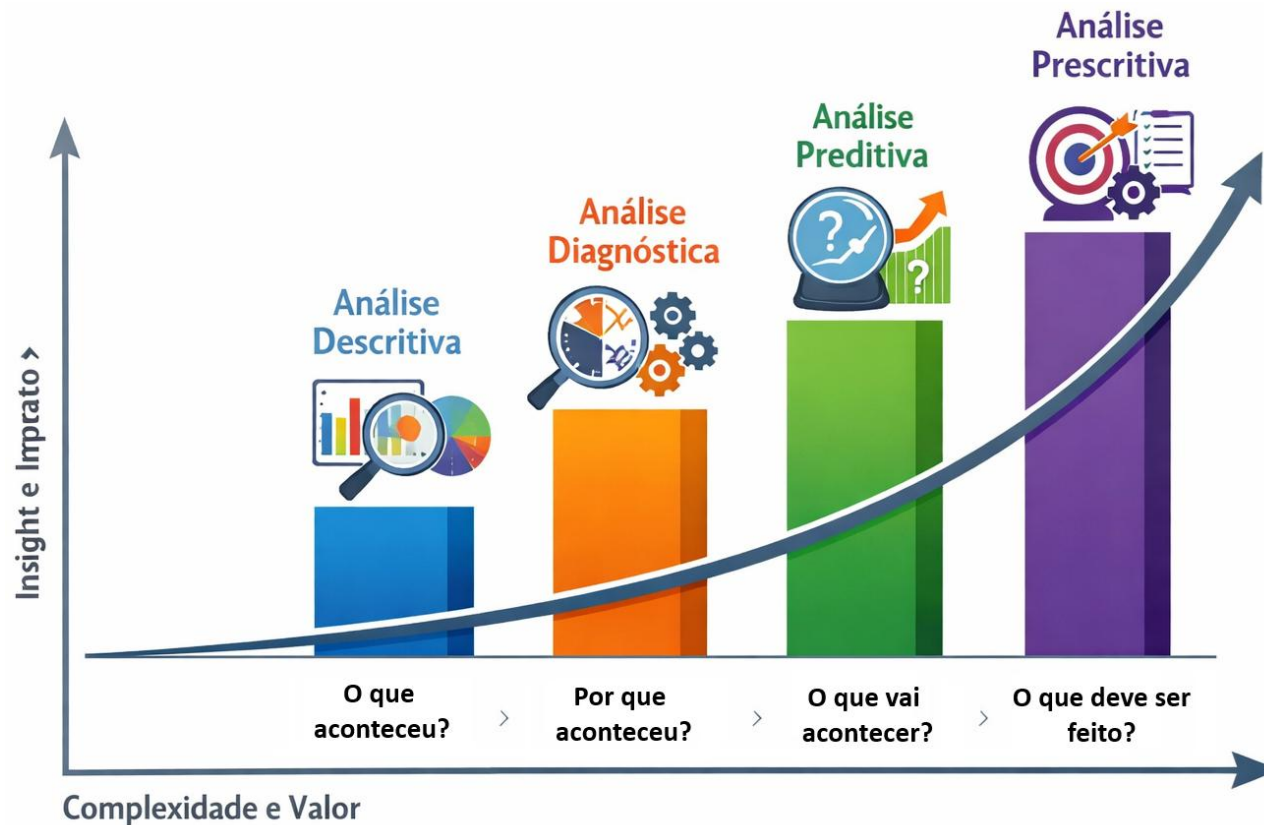
Descritiva

Diagnóstica

Preditiva

Prescritiva

TIPOS DE ANÁLISE DE DADOS



ANÁLISE DESCRITIVA

Definição

- Tem como objetivo resumir e organizar **dados históricos**, fornecendo uma visão clara do que aconteceu no passado por meio da consolidação de métricas, indicadores e estatísticas básicas.

Pergunta típica

- **O que aconteceu?**

Entrega prática

- Gera **dashboards operacionais**, relatórios periódicos e **painéis de indicadores** que estruturam métricas como médias, totais, distribuições e tendências, permitindo monitoramento sistemático do desempenho.

ANÁLISE DIAGNÓSTICA

Definição

- Busca compreender por que determinados resultados ocorreram, **investigando causas**, fatores associados e **relações entre variáveis** por meio de segmentações, comparações e análises aprofundadas.

Pergunta típica

- **Por que isso aconteceu?**

Entrega prática

- Produz relatórios explicativos e análises estruturadas que evidenciam **relações relevantes entre variáveis, decompondo métricas** e apresentando **evidências que sustentam interpretações** sobre os mecanismos que levaram ao comportamento observado.

ANÁLISE PREDITIVA

Definição

- Utiliza **padrões observados em dados históricos** para estimar eventos ou comportamentos futuros por meio de **modelos estatísticos** e algoritmos de **aprendizado de máquina**.

Pergunta típica

- **O que vai acontecer?**

Entrega prática

- Gera **modelos capazes de produzir previsões**, probabilidades associadas a eventos e escores de risco aplicáveis a novos dados, além de métricas que avaliam sua capacidade de **generalização**.

ANÁLISE PRESCRITIVA

Definição

- A análise prescritiva vai **além da previsão e busca recomendar ações com base em previsões**, restrições e objetivos definidos.

Pergunta típica

- **O que deve ser feito?**

Entrega prática

- Fornece **recomendações estruturadas**, políticas de decisão ou soluções de otimização que **orientam a escolha** da alternativa mais adequada diante dos objetivos estabelecidos.

ANÁLISE EXPLORATÓRIA DE DADOS

Vale ressaltar que a EDA (*Exploratory Data Analysis*) **não constitui um outro tipo de análise** dentro desse contexto.

EDA é uma abordagem voltada à **investigação inicial do conjunto de dados**, com o objetivo de identificar padrões, distribuições, inconsistências, valores atípicos e possíveis relações entre variáveis.

A exploração de dados (ou “análise exploratória”) é uma **prática transversal** que ocorre em qualquer projeto analítico, independentemente do contexto (desde a análise descritiva até a análise prescritiva).

"Exploratory data analysis can never be the whole story, but nothing else can serve as the foundation stone."

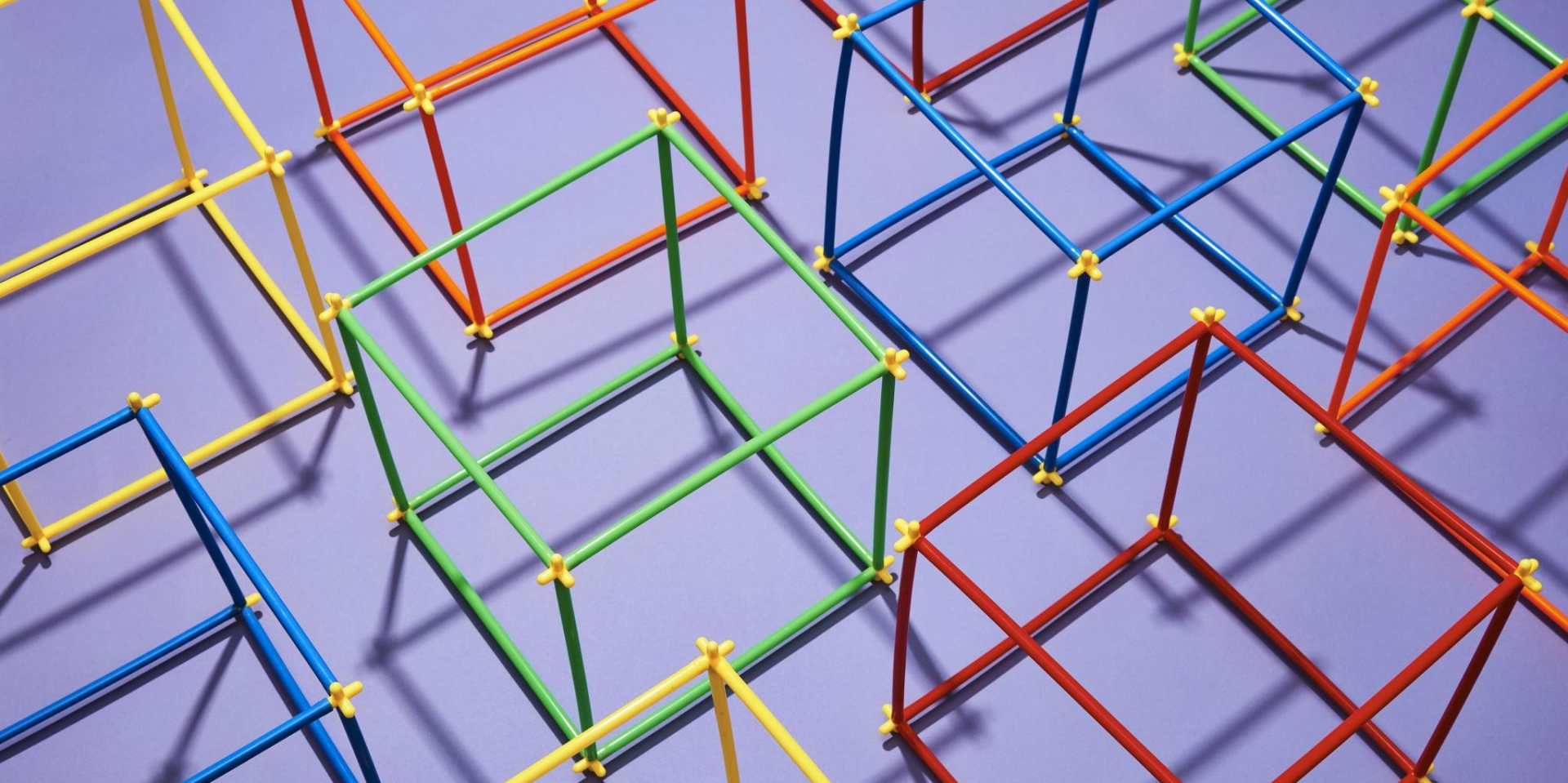
Tukey, J. W. (1977). Exploratory Data Analysis. Addison-Wesley.

POR QUE ANÁLISE PREDITIVA?

A análise de dados tradicional sempre buscou explicar o que aconteceu. Hoje, organizações **querem prever o futuro para agir antes**. Isso desloca o foco para a análise preditiva.

BI e mineração de dados se consolidaram por muito tempo explorando o passado. O cenário atual, porém, utiliza **modelos estatísticos** e de **aprendizado de máquina** para produzir estimativas sobre novos dados, não só relatórios históricos.

A capacidade preditiva já é um **diferencial competitivo**. Empresas que conseguem prever melhor também planejam melhor, mesmo sem chegar ao nível da análise prescritiva.



II – PROCESSO DE ANÁLISE DE DADOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)

PROCESSO DE ANÁLISE DE DADOS

Independentemente da técnica, projetos de análise de dados costumam seguir um conjunto recorrente de etapas que vão do **entendimento do problema à geração de *insights* acionáveis**.

Esse processo foi formalizado sob o termo **KDD** (*Knowledge Discovery in Databases*) em 1996 para descrever a **descoberta de conhecimento** por meio de **etapas** como seleção, pré-processamento, transformação, **mineração** e interpretação.

Essa mesma lógica foi posteriormente estruturada em metodologias operacionais como o **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), amplamente adotado para organizar e conduzir **projetos de mineração de dados** em ambientes organizacionais.

As etapas do CRISP-DM eram: *Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, Deployment*.

PROCESSO DE ANÁLISE DE DADOS

Vamos adotar um processo analítico genérico, organizado em **seis macroetapas** proposto em processos de treinamento de certificação oficial da Google*:

- *Entendimento*
- *Preparação*
- *Pré-processamento*
- *Modelagem*
- *Compartilhamento*
- *Ação*

ENTENDIMENTO

Corresponde ao enquadramento inicial do trabalho analítico, quando uma **percepção difusa** do negócio é estruturada como **objeto de investigação**.

Define o **problema a ser compreendido**, os **objetivos da análise** e o tipo de **pergunta que deverá ser respondida**.

Garante alinhamento com *stakeholders* e evita iniciar exploração de dados sem direção conceitual clara.

PREPARAÇÃO

Consiste em selecionar e compreender as **fontes de dados** que poderão responder à pergunta analítica.

Delimita escopo, período de análise, granularidade, disponibilidade, acesso e **restrições legais** ou operacionais.

Avalia a **viabilidade da análise** antes de qualquer tratamento técnico, assegurando que os dados certos existem e que poderão ser utilizados.

PRÉ-PROCESSAMENTO

Tem como finalidade tornar os dados **tecnicamente confiáveis e consistentes** para uso posterior.

Envolve limpeza, padronização, tratamento de ausências, correção de inconsistências, validação estrutural e organização do *dataset*.

Atua sobre a qualidade e integridade dos dados, **sem ainda buscar interpretações ou padrões**.

MODELAGEM

Utiliza os dados já processados para produzir evidência capaz de responder às perguntas formuladas.

Aplica técnicas de exploração, agregação, comparação, criação de variáveis, **modelagem** ou testes analíticos conforme o objetivo.

Converte **dados confiáveis em informação interpretável**, revelando relações, tendências ou comportamentos relevantes.

COMPARTILHAMENTO

Traduz os resultados analíticos em comunicação estruturada para apoiar decisão organizacional.

Adapta linguagem, visualizações e nível de detalhamento ao **público-alvo**, preservando rigor técnico com clareza interpretativa.

Explicita implicações práticas, limitações e recomendações derivadas da modelagem.

AÇÃO

Materializa os *insights* produzidos em decisões, intervenções ou experimentos no contexto real do negócio.

Implementa mudanças orientadas por evidências e acompanha seus efeitos para avaliar resultados.

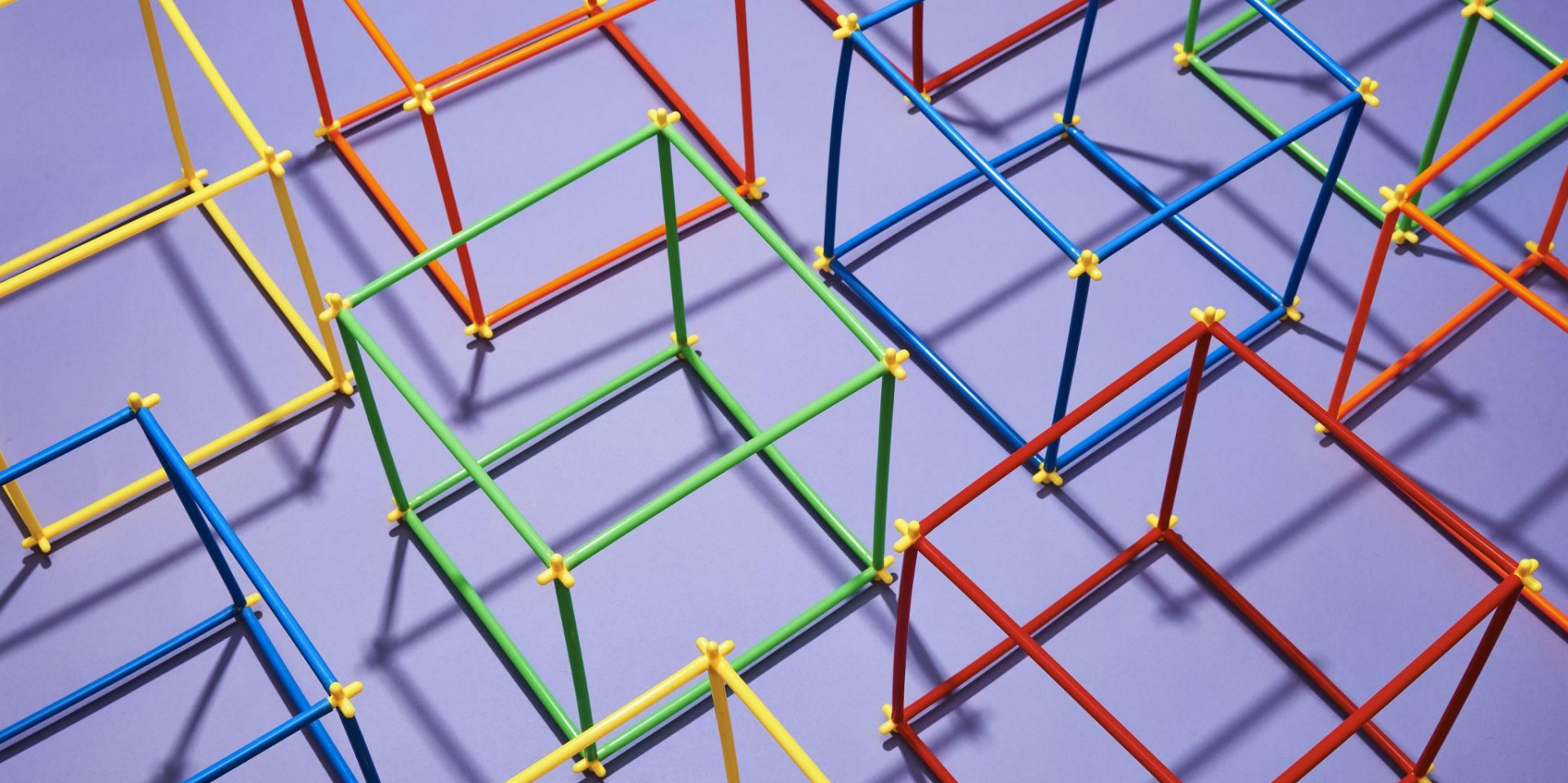
Fecha o ciclo analítico ao gerar aprendizado organizacional e novas perguntas para **iterações futuras**.

PROCESSO VERSUS TIPOS DE ANÁLISE

Os tipos de análise de dados (descritiva, diagnóstica, preditiva e prescritiva) e as etapas do processo **não são concorrentes**.

Enquanto os tipos de análise dizem respeito à **natureza da pergunta** que se deseja responder e ao **valor gerado** a partir dos dados, o processo descreve **como** o trabalho é conduzido, desde o entendimento do problema até a tomada de decisão.

As seis etapas do processo **estão presentes em qualquer análise**.



III – FORMULAÇÃO DO PROBLEMA PREDITIVO

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)

PROBLEMA PREDITIVO - ML SUPERVISIONADO

Quando falamos em análise preditiva, quase sempre estamos lidando com problemas formulados a partir de **uma variável-alvo explícita a ser estimada com base em dados históricos**.

Se você já teve algum contato com aprendizado de máquina, vai reconhecer que esse é o enquadramento típico do **aprendizado supervisionado**.

- **Regressão (valor contínuo):** Linear, Árvore (tree), Random Forest (ensemble), SVR (SVM para regressão), Rede Neural.
- **Classificação (classe/categoria):** Logística, KNN (vizinhos), Árvore (tree), Random Forest (ensemble), SVM (margem máxima), Naive Bayes, Rede Neural.

PROBLEMA PREDITIVO - PERGUNTA ANALÍTICA

Perguntas Analíticas

Traduzem o problema em questões claras, investigáveis (**operacionalizáveis com dados**) e mensuráveis, que orientam o que será analisado e qual tipo de resposta se busca nos dados. A forma como a pergunta é formulada ajuda a definir elementos centrais da análise, como a unidade de análise, o resultado de interesse e o horizonte temporal, facilitando a posterior formulação do problema preditivo.

Como a taxa de cancelamento evoluiu nos últimos meses? (*descritiva*)

Quais fatores estão mais associados ao aumento do cancelamento? (*diagnóstica*)

Quais clientes apresentam maior probabilidade de cancelar em 30 dias? (*preditiva*)

Quais ações de retenção devem ser priorizadas para reduzir esse cancelamento? (*prescritiva*)

PROBLEMA PREDITIVO - FORMULAÇÃO

Quando a pergunta analítica envolve **previsão**, a etapa seguinte é **formular o problema preditivo** (unidade de análise, variáveis etc.).

- **Objetivo** – Decisão ou ação que o modelo pretende apoiar. Conecta análise ao contexto de negócio (conceder ou não empréstimo, oferecer retenção, bloquear transação).
- **Unidade de Análise** – Entidade do mundo real sobre a qual a previsão será feita. Define a granularidade da base (uma transação, um cliente etc.).
- **Variável-Alvo** – Resultado futuro associado à unidade de análise, que será estimado pelo modelo (inadimplência em 90 dias).
- **Variáveis Explicativas** – Informações observáveis antes do evento, usadas para estimar o alvo (renda, idade, score, valor da compra, histórico).
- **Horizonte de Previsão** – Intervalo temporal entre o momento em que a previsão é feita e o momento futuro que se deseja estimar (próximos 30, 60, 90 dias etc.).
- **Janela de Observação** – Período do passado utilizado para coletar e construir as variáveis que servirão de base para a previsão.

PROBLEMA PREDITIVO - FORMULAÇÃO

A unidade de análise define a entidade sobre a qual a previsão será feita e determina a **granularidade do problema**. A mesma base pode gerar modelos diferentes dependendo dessa escolha.

Problema 1

Objetivo: prever reprovação na disciplina

Unidade de análise: aluno-disciplina

Alvo: `reprovado_final`

Explicativas (após av2): av1, av2

Problema 2

Objetivo: prever evasão no próximo semestre

Unidade de análise: aluno

Alvo possível: `evasao_sem2`

Explicativas: médias parciais, histórico de reprovações

Notas por disciplina

Cada linha = aluno em uma disciplina

aluno	disciplina	av1	av2	av3	av4	trabalho	reprovado
Ana	Matemática	4.0	5.0	6.0	7.0	6.5	0
Ana	Português	6.0	7.0	8.0	7.5	7.0	0
Bruno	Matemática	3.5	4.0	4.5	5.0	4.0	1
Bruno	Português	5.5	6.0	6.5	6.0	6.0	0
Carla	Matemática	7.0	6.5	8.0	7.5	8.0	0
Carla	Português	4.0	4.5	5.0	5.5	5.0	1

PROBLEMA PREDITIVO - FORMULAÇÃO

Considere uma base contendo notas de alunos por disciplina ao longo do semestre, com quatro avaliações e um trabalho.

Se o objetivo for **prever a reprovação de um aluno em uma disciplina** específica ao final do semestre, a unidade de análise deve ser **aluno-disciplina**, pois a previsão é feita para **cada matéria individualmente**, utilizando notas parciais como av1 e av2 (ou também av3...).

Por outro lado, se o objetivo for prever algo como **evasão** ou **desempenho acadêmico global no próximo semestre**, a unidade de análise passa a ser o **aluno**, **agregando informações de todas as disciplinas**.

Assim, o objetivo define a **unidade de análise**, que por sua vez determina a estrutura da base de dados e das variáveis utilizadas pelo modelo.

QUALIDADE DAS VARIÁVEIS EXPLICATIVAS

As variáveis explicativas (também chamadas de *features* na literatura de ML) possuem valor preditivo na medida em que **contribuem efetivamente para reduzir a incerteza associada à variável-alvo.**

Em análise preditiva, o simples fato de uma variável estar presente no *dataset* **não garante que ela seja útil para previsão.**

QUALIDADE DAS VARIÁVEIS

VAZAMENTO DE INFORMAÇÃO

O **vazamento de informação** (*data leakage*) ocorre quando são utilizadas variáveis que só se tornam **conhecidas após o evento a ser previsto** ou que incorporam diretamente o próprio resultado.

Nesses casos, o modelo passa a utilizar “informação do futuro”, produzindo resultados artificialmente **precisos nos dados históricos, mas inviáveis em uso real**.

- *Exemplo:* tentar prever se um aluno será aprovado utilizando no *dataset* de treinamento uma coluna com a nota final da prova final, informação que só existe após a avaliação.

QUALIDADE DAS VARIÁVEIS

VARIÁVEIS IRRELEVANTES

Variáveis irrelevantes são variáveis que **não apresentam relação estatística significativa** com a variável-alvo e, portanto, não contribuem para a capacidade preditiva.

Seu uso tende a introduzir ruído, aumentar a complexidade do modelo e reduzir a interpretabilidade, sem ganhos de desempenho preditivo.

- *Exemplo:* utilizar número identificador do cliente (IDs), códigos de produto ou outros campos meramente administrativos que não possuem relação com o fenômeno a ser previsto, apenas adicionando ruído ao modelo.

QUALIDADE DAS VARIÁVEIS

VARIÁVEIS REDUNDANTES

Variáveis redundantes são variáveis que carregam **informação muito semelhante ou praticamente repetida** em relação a outras variáveis já presentes no conjunto de dados (frequentemente operacionalizada como alta correlação ou sobreposição informacional).

Em vez de acrescentarem conhecimento novo ao modelo, elas tendem a repetir padrões já capturados por outros atributos, aumentando a complexidade (dimensionalidade etc.) e podendo dificultar a interpretação sem trazer ganho preditivo relevante.

- *Exemplo:* utilizar simultaneamente **idade** e **ano de nascimento**, ou **renda mensal** e **renda anual**, pois essas variáveis representam essencialmente a mesma informação.

QUALIDADE DAS VARIÁVEIS

VARIÁVEIS INVIÁVEIS (INDISPONÍVEIS)

Variáveis inviáveis ou indisponíveis são variáveis que podem existir apenas em contextos experimentais, estudos pontuais ou coletas **não incorporadas à rotina operacional**, não estando disponíveis quando o modelo precisa ser aplicado.

Embora não caracterizem vazamento de informação, essas variáveis têm baixo valor prático para a previsão, pois impedem o uso consistente do modelo em produção.

- *Exemplo:* utilizar dados obtidos em uma pesquisa pontual que não fazem parte da coleta regular de dados da organização.

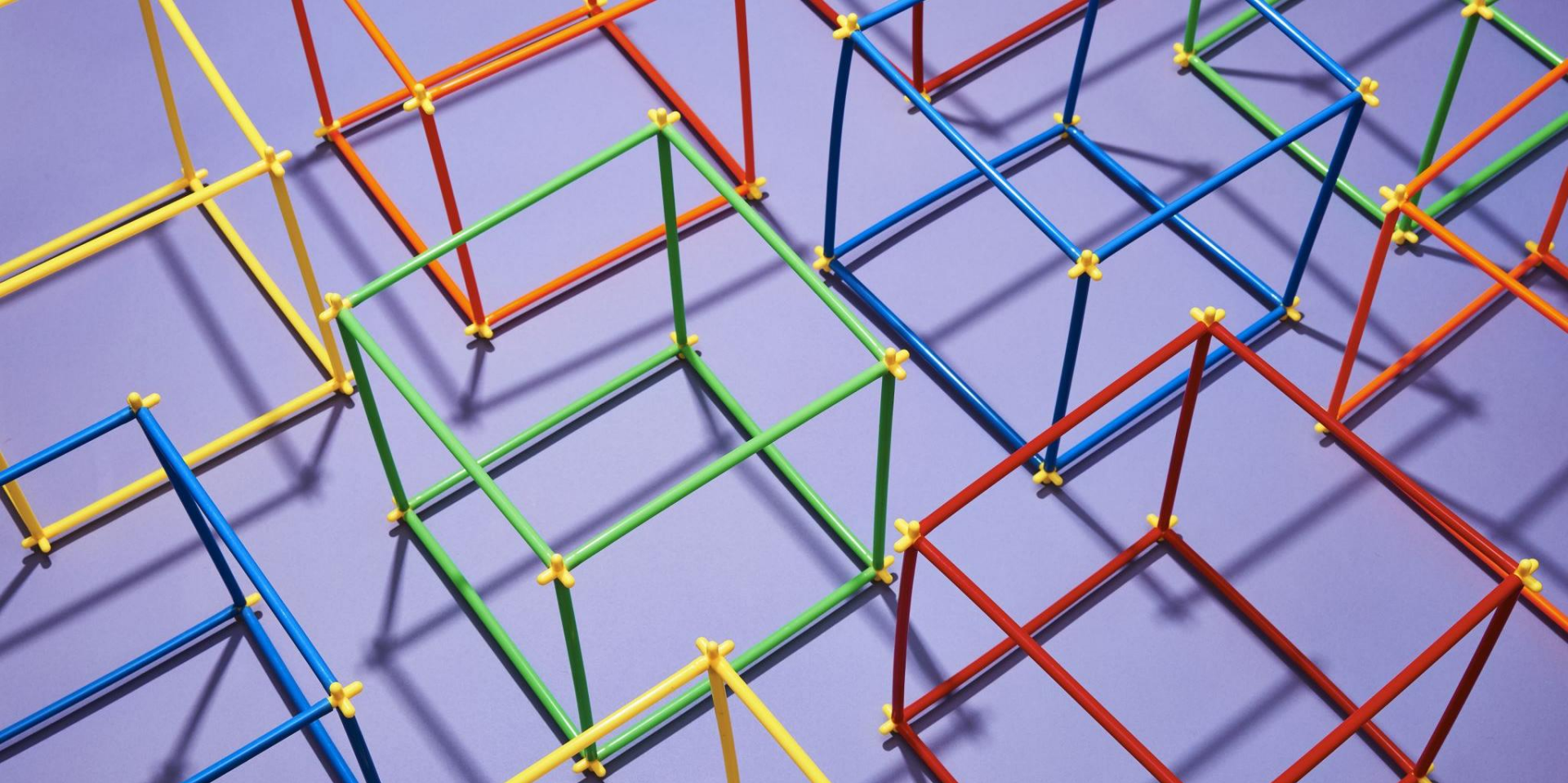
QUALIDADE DAS VARIÁVEIS

VARIÁVEIS INSTÁVEIS

Variáveis instáveis são variáveis cujo comportamento ou distribuição pode mudar ao longo do tempo (*drift*). Quando isso ocorre, os padrões aprendidos no passado deixam de representar adequadamente o fenômeno, reduzindo a capacidade de generalização do modelo.

Esse tipo de instabilidade é comum em contextos organizacionais sujeitos a mudanças de mercado, estratégia ou perfil de clientes.

- *Exemplo:* durante períodos promocionais, como a Black Friday, o comportamento de compra se altera, fazendo com que relações observadas no passado deixem de se repetir da mesma forma no futuro.



IV — PIPELINE PREDITIVO ENGENHARIA DE ATRIBUTOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)

PIPELINE PREDITIVO

O processo em seis etapas (*Entender, Preparar, Processar, Analisar, Compartilhar e Implementar*) continua sendo o ‘guarda-chuva’ da gestão do projeto analítico.

O que muda, conforme o tipo de análise, é o **fluxo técnico dentro da etapa de análise**. Em análises descritiva e diagnóstica, o fluxo técnico dentro da etapa *Analisar* costuma ser exploração, agregações, segmentações e testes para resumir e explicar o que aconteceu.

Na análise preditiva, esse fluxo ganha camadas adicionais: **treinar e validar o modelo e testar em dados fora da amostra**.

PIPELINE PREDITIVO

Chamamos de *pipeline* o encadeamento de etapas técnicas executadas dentro da macroetapa **Analisar**, nas quais os textos são transformados em representações computacionais e utilizados para especificar, **treinar, validar e avaliar modelos de Análise Preditiva**.

É esse fluxo que detalharemos a seguir, a saber:

- Engenharia de atributos *
- Especificação do modelo
- Treinamento
- Validação
- Teste

* Utilizamos a expressão “Engenharia de atributos” como um **conceito guarda-chuva** para designar qualquer transformação orientada ao modelo, isto é, decisões que moldam os dados para torná-los adequados a alguma tarefa preditiva específica.

PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Engenharia de atributos (*feature engineering*) é o processo de criar, transformar, selecionar ou agregar variáveis a partir dos dados brutos com o **objetivo de melhorar a capacidade do modelo**.

As etapas específicas variam conforme o tipo de dado, mas o princípio é o mesmo: transformar informação bruta em representação útil ao modelo. Em **Análise Preditiva**, isso envolve:

- **Construção de Atributos** (gerar informação derivada)
- **Codificação de Variáveis Categóricas** (transformar valores categóricos)
- **Transformações Matemáticas** (ajustar distribuição e reduzir assimetrias)
- **Escalonamento** (padronizar escala)
- **Seleção e Redução de Atributos** (controlar dimensionalidade e redundância)

Aqui, os dados são tabulares. As variáveis geralmente já nascem **estruturadas em colunas**. Então *feature engineering* foca em construir e transformar colunas (derivadas, escalas, codificação e seleção).



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Construção de Atributos

Criação de novas variáveis a partir das variáveis existentes com o objetivo de capturar relações, padrões ou estruturas que **não estão explícitos nos dados originais**. Envolve **gerar atributos derivados** por meio de combinações, agregações ou transformações simples das colunas.

Em dados tabulares, a construção de atributos parte de variáveis já estruturadas (colunas) e busca **produzir novas colunas com maior poder explicativo** para o modelo.

Em PLN, a situação é diferente. O texto bruto **não possui variáveis explícitas**. Antes da construção de atributos, é necessário estruturar o texto por meio de **segmentação, normalização, extração de informação e representação vetorial**. Assim, muitos atributos em PLN **emergem** dessas etapas estruturais (POS Tagging, NER), enquanto em dados tabulares a construção de atributos é uma operação direta sobre colunas já existentes.





PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Construção de Atributos

Combinações criam novos atributos por operações entre **colunas da mesma linha da tabela**. O objetivo é capturar **relações entre variáveis do mesmo registro**.

- Exemplos:

Razões / proporções → renda total / pessoas na família

Diferenças → valor_pago — valor_devido

Interações → idade × renda

Ideia central: operar entre **atributos do mesmo indivíduo/registo**.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Construção de Atributos

Agregações criam novos atributos por **operações sobre múltiplas linhas da tabela**, normalmente agrupadas por uma entidade. A ideia é calcular, métricas para uma unidade de análise a partir de **vários registros**.

- Exemplos:
 - Soma (total gasto)
 - Média (gasto médio)
 - Contagem (número de compras)
 - Máximo / mínimo

Ideia central: resumir **informação de vários registros** em um atributo.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Construção de Atributos

Atributos temporais são variáveis criadas a partir de **campos de data ou hora** presentes no dataset. A ideia é decompor a informação temporal para tornar **padrões recorrentes mais visíveis para o modelo**.

- Exemplos:

data_compra → ano, mês, dia

data_compra → dia_da_semana (segunda, terça, ...)

data_compra → trimestre

data_compra → dias_desde_última_compra

hora_acesso → período_do_dia (manhã, tarde, noite).



Uma data isolada normalmente vira apenas um **número ordinal** (*timestamp*) dentro do modelo. Ao decompor a data em novos atributos, tornamos **sazonalidade, ciclos semanais e padrões de comportamento mais explícitos** para o treinamento do modelo.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Construção de Atributos

A **discretização** (*binning*) consiste em transformar uma variável numérica contínua em **faixas ou categorias**. A ideia é representar valores por **intervalos que capturam comportamentos semelhantes**.

- Exemplos:

idade → faixa_etaria (18–25, 26–40, 41+)

renda → faixa_renda (baixa, média, alta)

valor_compra → categoria_valor (pequena, média, grande)

A discretização pode ajudar quando o efeito da variável não é linear, permitindo ao modelo aprender padrões associados a **intervalos de valores em vez de valores exatos**. Ao reduzir a granularidade dos dados, facilita a identificação de comportamentos que variam **por faixas de valores**, e não de forma contínua. Também pode ajudar a resolver problemas de outliers (faixas de valores “muito alto” e “muito baixo”).



PIPELINE PREDITIVO

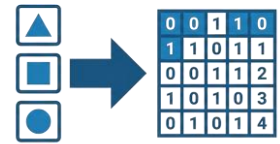
ENGENHARIA DE ATRIBUTOS

Construção de Atributos

Indicadores Binários (*flags*) assumem apenas **dois valores (0 ou 1)** para indicar se uma determinada condição ocorre ou não.

- Exemplos:
 - valor_compra → compra_acima_1000 (0/1)
 - dias_atraso → cliente_inadimplente (0/1)
 - data_cadastro → cliente_novo (0/1)
 - idade → maior_de_idade (0/1)

Flags permitem transformar **condições em atributos claros para o modelo**, facilitando identificar comportamentos ou eventos relevantes.



PIPELINE PREDITIVO

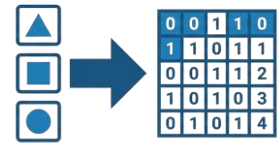
ENGENHARIA DE ATRIBUTOS

Codificação de Variáveis Categóricas

É o processo de transformar **valores categóricos** (rótulos ou texto curto) em **valores numéricos** para que possam ser utilizados por algoritmos de aprendizado de máquina.



Enquanto em dados tabulares normalmente codificamos **categorias discretas** (One-Hot, ordinal etc.), em PLN a representação vetorial lida com estruturas linguísticas mais complexas e textos potencialmente longos. No processamento de linguagem natural essa etapa chamada de representação vetorial (palavras ou textos são transformados em vetores numéricos para serem processados pelos modelos). Exemplos lá são: BoW, TF-IDF, PPMI e outros baseados em embedding.



PIPELINE PREDITIVO

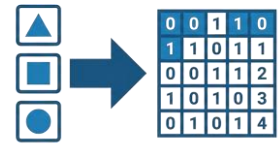
ENGENHARIA DE ATRIBUTOS

Codificação de Variáveis Categóricas

A **codificação One-Hot** transforma uma variável categórica com **k categorias** em **k variáveis binárias indicadoras**. Como a maior parte dos modelos opera com entradas numéricas, variáveis categóricas precisam ser convertidas para representação numérica antes do treinamento.

- Exemplo: varejo e atacado; cartão, boleto e pix; SP, RJ e MG.
- Após One-Hot:
varejo, atacado → 10, 01 (ou simplesmente 0 e 1)
cartão, boleto, pix → 100, 010, 001
SP, RJ, MG → 100, 010, 001

Cada linha passa a indicar explicitamente a qual categoria pertence por meio de **variáveis binárias mutuamente exclusivas (ortogonais)**.



PIPELINE PREDITIVO

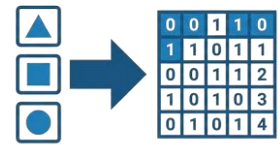
ENGENHARIA DE ATRIBUTOS

Codificação de Variáveis Categóricas

A **codificação ordinal** consiste em substituir categorias por valores numéricos inteiros que representem uma **hierarquia previamente existente**. Há imposição explícita de ordem entre as categorias.

- Exemplo: baixo, médio e alto; júnior, pleno e sênior; bronze, prata e ouro; etc.
- Após Ordinal:
 - baixo, médio, alto → 1, 2, 3
 - júnior, pleno, sênior → 1, 2, 3
 - bronze, prata, ouro → 1, 2, 3

Cada linha passa a indicar explicitamente a posição relativa da categoria dentro de uma **escala ordenada**.



0	0	1	1	0
1	1	0	1	1
0	0	1	1	2
1	0	1	0	3
0	1	0	1	4

PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

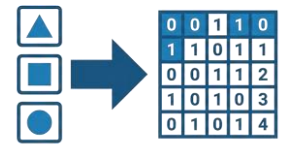
Codificação de Variáveis Categóricas

O **frequency encoding** substitui cada categoria pela frequência com que ela aparece no dataset. Assim, cada valor categórico passa a ser representado por um número que indica **quão comum aquela categoria é**.

Esse método também é útil quando a variável possui muitas categorias.

Note que o **frequency encoding não utiliza a variável-alvo**. A codificação depende apenas da **distribuição da própria variável categórica no dataset**, e não de seu comportamento em relação ao alvo.





PIPELINE PREDITIVO

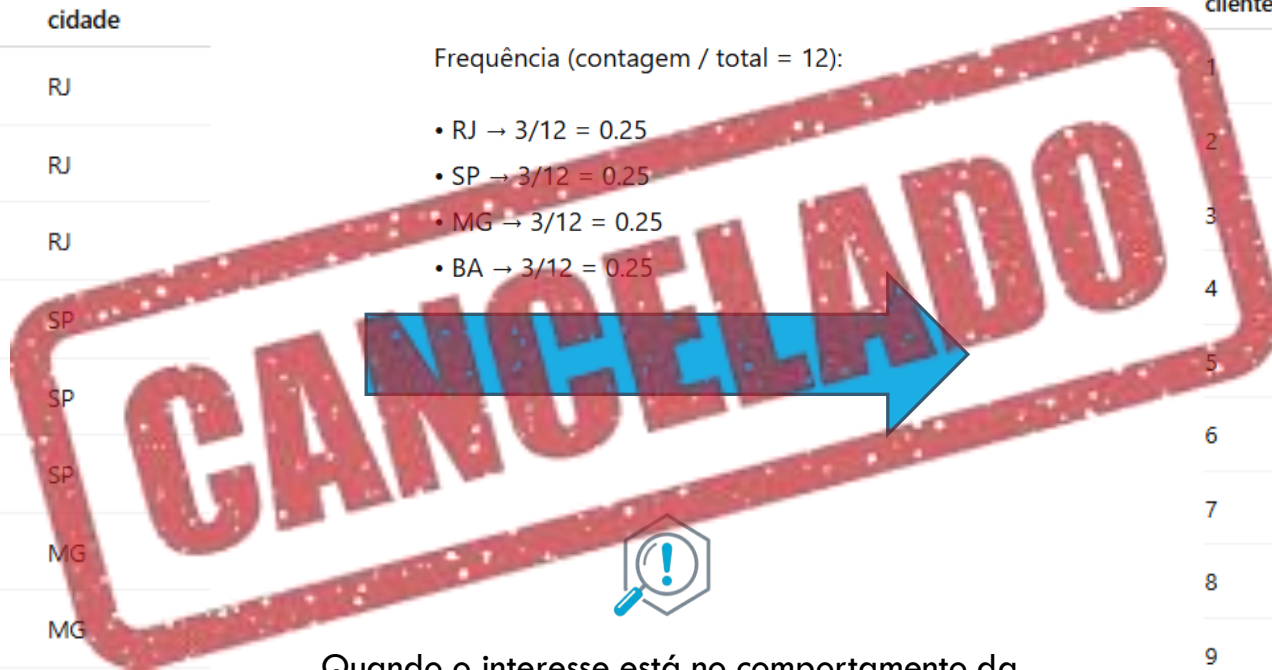
ENGENHARIA DE ATRIBUTOS

Dataset original

cliente	cidade
1	RJ
2	RJ
3	RJ
4	SP
5	SP
6	SP
7	MG
8	MG
9	MG
10	BA
11	BA
12	BA

Frequência (contagem / total = 12):

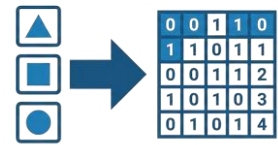
- RJ $\rightarrow 3/12 = 0.25$
- SP $\rightarrow 3/12 = 0.25$
- MG $\rightarrow 3/12 = 0.25$
- BA $\rightarrow 3/12 = 0.25$



Quando o interesse está no comportamento da variável em relação ao alvo, *Target* pode capturar essa informação. *Frequency*, por outro lado, apenas representa quão comum cada categoria é no dataset. É útil quando categorias **muito raras** ou **muito frequentes** podem carregar informação relevante para o modelo.

Após Frequency Encoding

cliente	cidade_encoded
1	0.25
2	0.25
3	0.25
4	0.25
5	0.25
6	0.25
7	0.25
8	0.25
9	0.25
10	0.25
11	0.25
12	0.25



0	0	1	1	0
1	1	0	1	1
0	0	1	1	2
1	0	1	0	3
0	1	0	1	4

PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

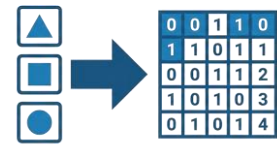
Codificação de Variáveis Categóricas

O **target encoding** substitui cada categoria da variável categórica por uma **estatística calculada a partir da variável-alvo**, geralmente a média do alvo para os registros daquela categoria. Assim, o valor categórico é transformado em um número que representa o **comportamento médio do alvo associado àquela categoria**.

Esse método é útil quando a variável possui muitas categorias, situação em que técnicas como One-Hot gerariam muitas colunas.

Nesse processo, a **identidade original da categoria é perdida**, e o atributo **passa a representar uma informação estatística derivada do alvo**. A escolha deve considerar se essa informação estatística é **mais relevante para o modelo do que a identidade original da categoria**.





PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Dataset original

cliente	cidade	inadimplente
1	RJ	1
2	RJ	0
3	RJ	1
4	SP	0
5	SP	0
6	SP	1
7	MG	0
8	MG	0
9	MG	0
10	BA	1
11	BA	1
12	BA	0

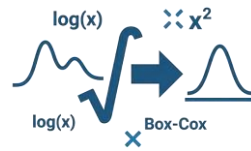
Médias do alvo por cidade:

- RJ $\rightarrow (1+0+1)/3 = 0.67$
- SP $\rightarrow (0+0+1)/3 = 0.33$
- MG $\rightarrow (0+0+0)/3 = 0.00$
- BA $\rightarrow (1+1+0)/3 = 0.67$

Após Target Encoding

cliente	cidade_encoded
1	0.67
2	0.67
3	0.67
4	0.33
5	0.33
6	0.33
7	0.00
8	0.00
9	0.00
10	0.67
11	0.67
12	0.67

A escolha depende do problema. Quando a **identidade da categoria é importante**, **One-Hot** preservam essa informação. **Target** é adequado quando o interesse está **no comportamento estatístico associado à categoria**, especialmente em situações com muitas categorias diferentes.



PIPELINE PREDITIVO

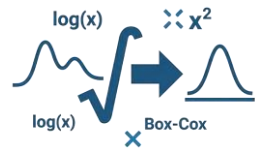
ENGENHARIA DE ATRIBUTOS

Transformações Matemáticas

Consistem em aplicar funções diretamente sobre variáveis existentes com o objetivo de **modificar propriedades de sua distribuição**, como assimetria, amplitude ou presença de valores extremos. Essas transformações geram novas versões da variável original para produzir padrões mais estáveis ou mais facilmente capturados por modelos.



Em PLN, transformações não são aplicadas diretamente sobre o texto. Antes disso, o texto é convertido em representações numéricas, como vetores ou matrizes. Após essa etapa (BoW, TF-IDF, PMI ou embeddings), os valores gerados normalmente já são utilizados diretamente pelos modelos. Essas representações vetoriais já incorporam mecanismos próprios de escalonamento etc., de modo que transformações adicionais são menos comuns.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Transformações Matemáticas

A **transformação logarítmica** é utilizada principalmente quando a distribuição apresenta **forte assimetria à direita** (“cauda longa” à direita) ou quando existem **ordens de grandeza muito diferentes** entre os valores.

Antes (assimetria forte)

renda

1200

3000

15000

120000

$\log_e(x)$ ou $\ln(x)$



Após log (log natural)

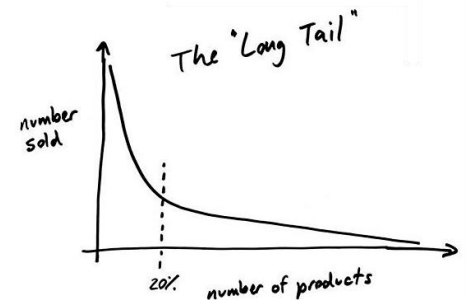
renda_log

7.09

8.01

9.62

11.70



O **logaritmo natural** é o logaritmo na base e . $\log_e(x)$ ou $\ln(x)$

O número e é uma constante matemática: $e \approx 2,718281828$

PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Transformações Matemáticas

O **clipping** serve para **reduzir a influência de valores extremos (outliers)** que podem distorcer a distribuição dos dados ou afetar o treinamento do modelo e distorcer a análise. A técnica consiste em **impor limites fixos definidos pelo analista**. Valores acima ou abaixo desses limites são **substituídos pelo próprio limite**.

Antes valor	Após clipping valor_clipped
20	20
35	35
50	50
65	65
80	80
400	100

Limite superior definido: 100

PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Transformações Matemáticas

A **winsorização** também é utilizada para **reduzir a influência de valores extremos** (igual ao clipping).

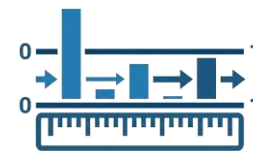
Porém, os limites **não são definidos arbitrariamente**, mas sim **derivados da própria distribuição dos dados** (geralmente por percentis).

Antes valor	Após winsorização valor_winsor
10	10
12	12
14	14
15	15
16	16
18	18
20	20
200	20

Percentis calculados:

- **P5 = 10**
- **P95 = 20**





PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

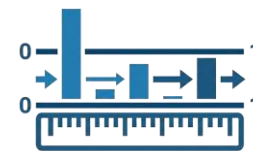
Escalonamento

É um tipo de transformação matemática cujo objetivo é **padronizar a escala das variáveis numéricas**, para que passem a operar em **intervalos comparáveis**. Essa etapa é importante porque muitos algoritmos são sensíveis à magnitude das variáveis, podendo atribuir maior influência a atributos com valores numericamente maiores.

Em dados tabulares, o escalonamento é aplicado diretamente sobre colunas numéricas já existentes, produzindo versões transformadas dessas variáveis em escalas padronizadas (por exemplo, valores normalizados entre 0 e 1 ou variáveis centradas e padronizadas).



Em PLN, a situação é diferente. Após a representação vetorial do texto (como TF-IDF ou embeddings), os vetores geralmente já incorporam mecanismos de ponderação ou normalização. Por isso, o escalonamento adicional costuma ser menos necessário do que em dados tabulares, embora em alguns casos ainda se aplique normalização.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Escalonamento - Normalização

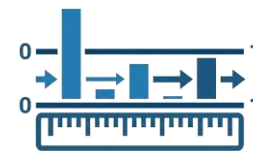
A **normalização** (Min–Max) reescala os valores de uma variável numérica para um intervalo fixo, geralmente $[0,1]$. O menor valor da variável passa a ser **0** e o maior valor passa a ser **1**, enquanto os demais valores são posicionados proporcionalmente entre esses limites.

Esse procedimento é útil quando diferentes variáveis possuem **escalas muito distintas**, pois coloca todas na mesma faixa numérica sem alterar a ordem relativa dos valores.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$



A normalização **não altera a forma da distribuição dos dados** (as distâncias relativas entre os valores permanecem as mesmas). Ela coloca todos os valores dentro de uma faixa, normalmente entre 0 e 1.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Antes (escala original) — idade

18
25
40
60

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$



Cálculos:

idade	cálculo	resultado
18	$(18-18)/42$	0.00
25	$(25-18)/42 = 7/42$	0.17
40	$(40-18)/42 = 22/42$	0.52
60	$(60-18)/42$	1.00

Antes (escala original) — idade

18
25
40
60

$$x' = a + \frac{(x - x_{\min})}{(x_{\max} - x_{\min})}(b - a)$$

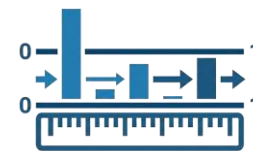


Intervalo [-1,1]

$$x' = -1 + 2 \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Cálculos:

idade	cálculo	resultado
18	$2 * (0/42) - 1$	-1.00
25	$2 * (7/42) - 1$	-0.67
40	$2 * (22/42) - 1$	0.05
60	$2 * (42/42) - 1$	1.00



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Escalonamento - Padronização

A **padronização** (também chamada de Z-score) transforma os valores de uma variável para que passem a ter **média 0** e **desvio padrão 1**. Cada valor passa a representar **quantos desvios-padrão ele está acima ou abaixo da média** da variável.

Esse procedimento é útil quando os algoritmos dependem da **escala das variáveis**, pois coloca todas em uma escala comparável baseada na distribuição dos dados.

$$x' = \frac{x - \mu}{\sigma}$$



A padronização também **não altera a forma da distribuição dos dados** (as distâncias relativas entre os valores permanecem as mesmas). Ela **reescala** os valores da variável **em relação à média e ao desvio padrão**.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Desvio Padrão

O desvio padrão é uma medida de dispersão que indica o **quanto os valores de uma variável se afastam**, em média, da média.

Desvio Padrão Populacional

Utilizado quando se considera **toda a população** de interesse. Mede a dispersão dos valores em relação à média populacional.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

Desvio Padrão Amostral

Utilizado quando os dados representam apenas uma amostra da população. Nesse caso, aplica-se uma correção (**n-1**) no denominador para **compensar o fato de a média ter sido estimada a partir da própria amostra**.

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

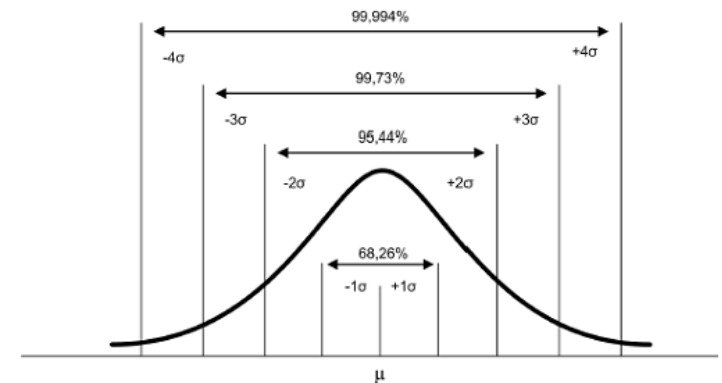
Desvio Padrão

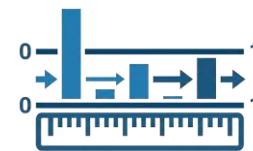
$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

Em distribuições aproximadamente normais, o desvio padrão permite interpretar a concentração dos dados em torno do centro da distribuição.

Quanto menor o desvio padrão, mais concentrados estão os valores próximos da média. Quanto maior o desvio padrão, mais espalhados estão os valores ao longo da distribuição.

- Cerca de **68%** dos valores tendem a $\mu \pm 1\sigma$
- Cerca de **95%** dos valores tendem a $\mu \pm 2\sigma$
- Cerca de **99,7%** dos valores tendem a $\mu \pm 3\sigma$





Distribuições antes de Z-Score:

6, 7, 7, 8, 8, 8, 9, 9, 10

1. Média

$$\bar{x} = \frac{\sum x_i}{n}$$

Cálculo:

$$\bar{x} = \frac{6 + 7 + 7 + 8 + 8 + 8 + 9 + 9 + 10}{9}$$

$$\bar{x} = \frac{72}{9} = 8$$

2. Mediana

Série ordenada:

6, 7, 7, 8, 8, 8, 9, 9, 10

Como $n = 9$:

$$\text{posição} = \frac{n + 1}{2} = \frac{10}{2} = 5$$

Mediana = 5º valor = 8

3. Moda

Valor mais frequente.

Frequências:

valor	frequência
6	1
7	2
8	3
9	2
10	1

Moda = 8

4. Variância (populacional)

Fórmula:

$$\sigma^2 = \frac{\sum (x_i - \bar{x})^2}{n}$$

Tabela de cálculo:

x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$
6	-2	4
7	-1	1
7	-1	1
8	0	0
8	0	0
8	0	0
9	1	1
9	1	1
10	2	4

$$\sum (x_i - \bar{x})^2 = 12$$

$$\sigma^2 = \frac{12}{9} = 1,33$$

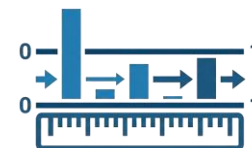
5. Desvio padrão

Fórmula:

$$\sigma = \sqrt{\sigma^2}$$

Cálculo:

$$\sigma = \sqrt{1,33} \approx 1,15$$



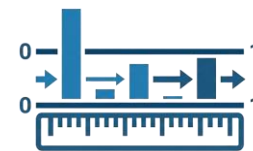
Resultado final

Medida	Valor
Média	8
Mediana	8
Moda	8
Variância	1,33
Desvio padrão	1,15

Observação:

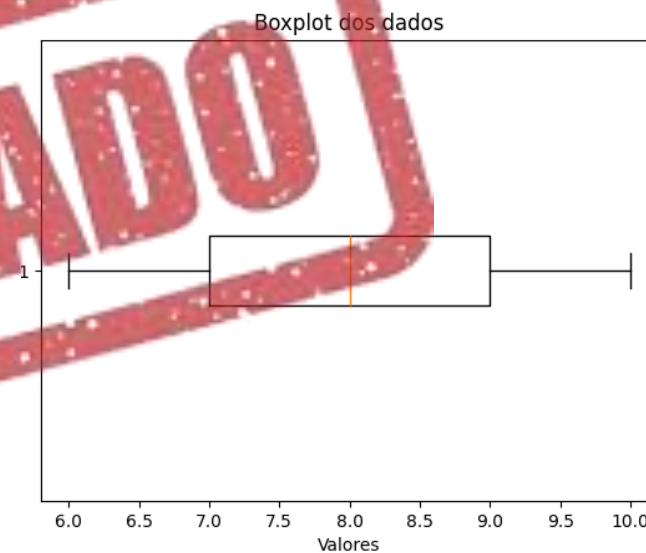
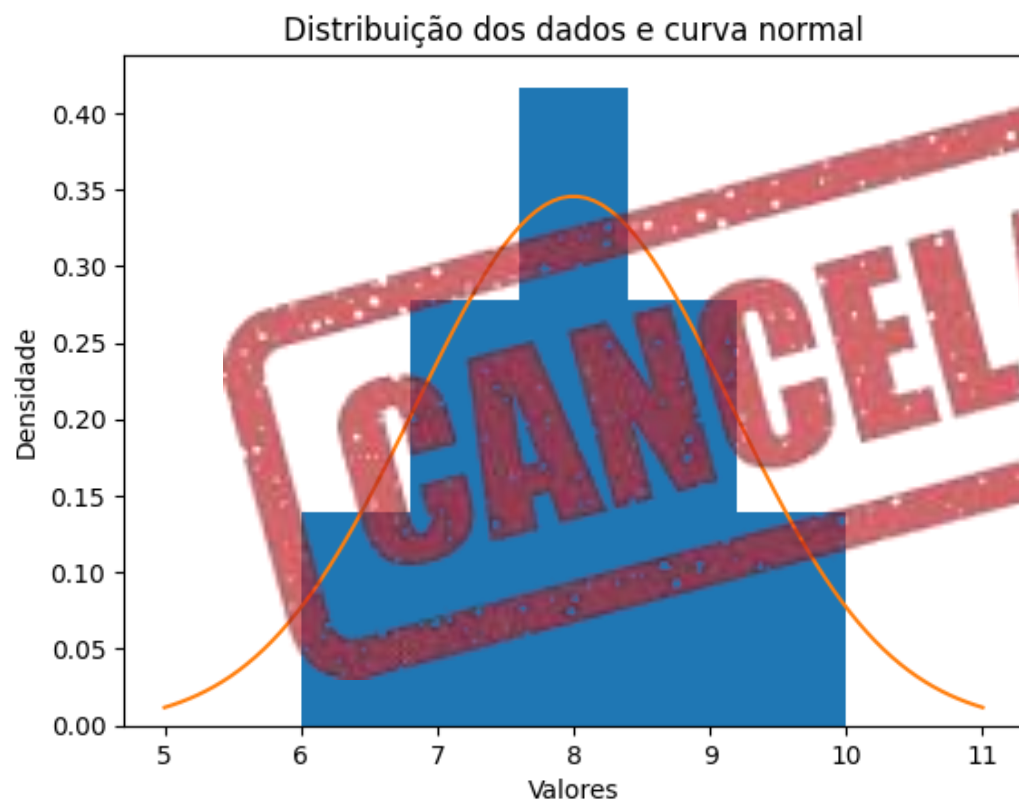
Se os dados fossem tratados como **amostra**, a variância seria calculada por

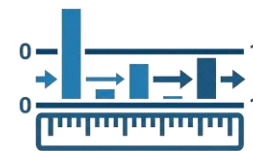
$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$$



Distribuições antes de Z-Score:

6, 7, 7, 8, 8, 8, 9, 9, 10



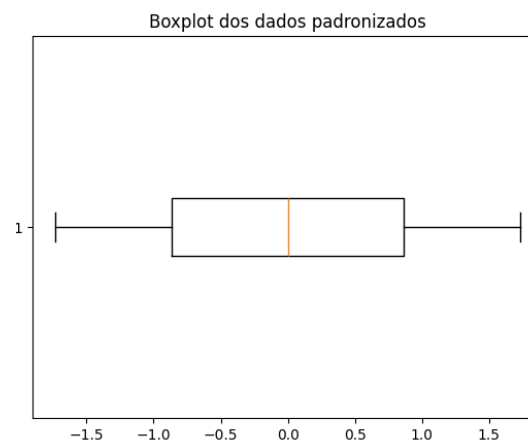


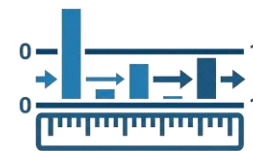
Distribuições após Z-Score:

6, 7, 7, 8, 8, 8, 9, 9, 10

$$x' = \frac{x - \mu}{\sigma}$$

x	cálculo	z
6	$(6 - 8) / 1,15$	-1,74
7	$(7 - 8) / 1,15$	-0,87
7	$(7 - 8) / 1,15$	-0,87
8	$(8 - 8) / 1,15$	0,00
8	$(8 - 8) / 1,15$	0,00
8	$(8 - 8) / 1,15$	0,00
9	$(9 - 8) / 1,15$	0,87
9	$(9 - 8) / 1,15$	0,87
10	$(10 - 8) / 1,15$	1,74





PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Antes (pressão sistólica – mmHg)

110

120

130

150

Média = 127.5

Desvio padrão ≈ 14.79

$$x' = \frac{x - \mu}{\sigma}$$



Cálculos:

x	cálculo	z
110	$(110 - 127.5) / 14.79$	-1.18
120	$(120 - 127.5) / 14.79$	-0.51
130	$(130 - 127.5) / 14.79$	0.17
150	$(150 - 127.5) / 14.79$	1.52

Diferentemente da normalização, a **padronização não produz um intervalo fixo de valores**. Ainda assim, o efeito é **tornar as variáveis comparáveis entre si**. Como cada variável é dividida pelo seu próprio **desvio padrão**, aquelas com valores maiores acabam sendo reduzidas proporcionalmente. Dessa forma, nenhuma variável tende a “dominar o modelo” apenas por possuir números maiores, o que é importante para algoritmos sensíveis à escala.

Em geral, a **padronização é a técnica mais utilizada em ML**. A normalização costuma ser usada quando se deseja limitar os valores de entrada para evitar saturação ou instabilidade numérica durante o treinamento (**sigmoid** satura rapidamente para valores grandes ou muito negativos).



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Seleção e Redução de Atributos

Consiste em **identificar e manter apenas as variáveis mais relevantes**, removendo ou reduzindo aquelas que são redundantes, irrelevantes ou que contribuem pouco para a capacidade preditiva.

Em dados tabulares, essa etapa atua **diretamente sobre as colunas do dataset**, selecionando um subconjunto de variáveis ou criando representações reduzidas que preservem a maior parte da informação relevante para o modelo.

Em PLN, a situação pode variar conforme o tipo de representação. Em representações esparsas, como BoW ou TF-IDF, é comum aplicar técnicas de seleção ou redução de atributos para controlar a alta dimensionalidade do vocabulário. Em representações densas, como embeddings, é menos frequente, pois esses vetores já são construídos de forma compacta.





PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Seleção de Atributos

A **correlação** mede o grau de **relação linear entre duas variáveis**. Ela pode ser usada para identificar variáveis altamente correlacionadas entre si ou pouco relacionadas com a variável-alvo.

Quando duas variáveis apresentam **correlação muito alta**, elas tendem a carregar **informação redundante**. Nesse caso, pode-se manter apenas uma delas, reduzindo a complexidade do modelo sem perda significativa de informação.

É simples e rápido de aplicar, sendo frequentemente utilizado como uma etapa inicial de análise para reduzir redundâncias no conjunto de dados.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Dataset

id	area_m2	quartos	preco_k
1	30	1	210
2	40	1	250
3	50	2	310
4	60	2	360
5	70	3	420
6	80	3	470
7	90	4	530
8	100	4	580

A **area_m2** e **quartos** tendem a ser altamente correlacionadas (maior área implica mais quartos).

Em seleção por **correlação**, pode-se remover uma das duas para reduzir redundância (manter **area_m2** e remover quartos, ou o inverso).

O **preco_k** é a variável alvo e tende a crescer com a área (ou com o número de quartos).



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Conceito Estatístico de Correlação

A correlação mede o grau de **relação linear entre duas variáveis**.

Ela é calculada a partir da covariância entre as variáveis, normalizada pelo produto de seus desvios padrão. Essa normalização faz com que o coeficiente de **correlação varie entre -1 e 1**.

Valores próximos de 1 indicam forte relação linear positiva, valores próximos de -1 indicam forte relação linear negativa, e valores próximos de 0 indicam ausência de relação linear significativa.

Correlação de Pearson

$$\rho_{X,Y} = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Forma expandida:

$$\rho = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Onde:

- x_i, y_i = observações
- $\bar{x}, \bar{y}$ = médias
- n = número de observações



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Seleção de Atributos

A **informação mútua** mede o **quanto o conhecimento de uma variável reduz a incerteza sobre outra**. Na seleção de atributos, é utilizada para avaliar o quanto cada variável **contribui para explicar a variável-alvo**.

Diferentemente da correlação, a informação mútua não se limita a relações lineares, podendo capturar dependências mais complexas.

Variáveis com maior informação mútua em relação ao alvo tendem a ser mais relevantes para o modelo, podendo ser priorizadas ou mantidas durante o processo de seleção de atributos.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Dataset

id	temperatura_c	produtividade
1	10	55
2	14	70
3	18	85
4	22	100
5	25	95
6	28	80
7	31	65
8	34	50

A variável **temperatura** influencia a produtividade (em temperaturas muito baixas ou muito altas o desempenho tende a ser menor, enquanto temperaturas moderadas favorecem maior produtividade).

Nesse caso, a relação **não é linear** (a produtividade aumenta até certo ponto e depois volta a diminuir conforme a temperatura continua subindo).

Diferentemente da correlação, que captura apenas relações lineares, a **informação mútua** consegue detectar dependências mais gerais, incluindo relações não lineares e não monotônicas.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Conceito Estatístico de Informação Mútua

A informação mútua mede o **quanto o conhecimento de uma variável reduz a incerteza sobre outra.**

Ela é definida a partir das probabilidades conjuntas e marginais das variáveis.

Quando a informação mútua é zero, as **variáveis são independentes**; quanto maior o valor, maior é a dependência entre elas.

Informação Mútua

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right)$$

Onde:

- $p(x, y)$ = probabilidade conjunta
- $p(x), p(y)$ = probabilidades marginais

Interpretação:

- $I(X; Y) = 0 \rightarrow$ variáveis independentes
- $I(X; Y) > 0 \rightarrow$ existe dependência entre elas.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Variável alvo: `inadimplente_prox_mes`)

<code>dias_atraso_pagamento</code>	0.570484
<code>valor_gasto_mes</code>	0.176635
<code>limite_credito</code>	0.113032
<code>renda_normalizada</code>	0.101661
<code>log_renda</code>	0.097873
<code>renda_mensal</code>	0.077040
<code>idade_cliente</code>	0.045964
<code>gasto_medio_cliente</code>	0.023007
<code>numero_transacoes</code>	0.000000
<code>gasto_medio_transacao</code>	0.000000

`dtype: float64`

A rigor, a **informação mútua** é uma técnica de seleção de atributos.

Ela é usada tipicamente para **rankear** e **remover atributos irrelevantes/redundantes** antes de treinar o modelo. Isso aparece de forma explícita na literatura clássica de seleção de variáveis (taxonomia filter/wrapper/embedded) e em trabalhos fundacionais de seleção baseada em ML.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Redução de Atributos

O **PCA** (*Principal Component Analysis*) é uma técnica de redução de dimensionalidade que transforma o conjunto original de **atributos** em um número menor de **componentes principais**. Cada componente é uma **combinação linear** das variáveis originais e é construído de forma a capturar a maior parte da variabilidade presente nos dados.

Diferentemente da **seleção de atributos**, o PCA não escolhe variáveis existentes, mas **cria novas representações** que resumem a informação do conjunto original. Assim, é possível reduzir o número de dimensões do **conjunto de dados** mantendo grande parte da informação relevante para a modelagem.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Redução de Atributos

Ao aplicar o PCA, cada observação deixa de ser representada pelas variáveis originais e passa a ser representada pelos valores nos componentes principais. O resultado é um novo espaço onde os dados são descritos por eixos que **capturam a maior variabilidade possível**.

Esses componentes são ordenados de acordo com a **quantidade de variância que explicam** ($PC1 > PC2 > PC3 \dots$).



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Redução de Atributos

Uma consequência direta dessa transformação é a possibilidade de **reduzir a dimensionalidade dos dados**. Como os primeiros componentes concentram a maior parte da variabilidade, é possível manter apenas alguns deles e descartar os demais, reduzindo o número de variáveis **sem perder grande parte da informação**.

Ao mesmo tempo, o PCA não apenas transforma os dados, mas aprende **uma regra de transformação a partir deles**. Essa regra define como as variáveis originais devem ser combinadas para gerar os componentes principais. Uma vez aprendida, essa transformação pode ser aplicada a novos dados.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Considere um dataset com variáveis altamente correlacionadas:

Cliente	Renda Mensal	Renda Anual	Gasto Mensal	Gasto Anual
A	5.000	60.000	2.000	24.000
B	6.000	72.000	2.400	28.800
C	4.000	48.000	1.600	19.200

Após aplicar **PCA**, o dataset pode ser representado assim:

Cliente	Componente 1	Componente 2
A	0.85	-0.10
B	1.02	-0.05
C	0.68	-0.12

Essas variáveis carregam **informação semelhante**.

Os novos componentes são **combinações das variáveis originais** e concentram a maior parte da variabilidade dos dados, permitindo trabalhar com **menos dimensões** sem perder muita informação.



PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Antes do PCA, tenho apenas as variáveis explicativas originais:

	iameter narrowin	age	gender	chest pain	rest SBP	cholesterol	ng blood sugar >	exerc ind ang	ajor vessels color	thal	slope peak exc ST	ST by exercise	max HR	rest ECG
1	0	0.949	male	typical ang	0.758	-0.2649	1	0	-0.723	fixed defect	downsloping	1.087	0.017	left vent hypert...
2	1	1.392	male	asymptomatic	1.611	0.7604	0	1	2.504	normal	flat	0.397	-1.822	left vent hypert...
3	1	1.392	male	asymptomatic	-0.665	-0.3423	0	1	1.428	reversible defect	flat	1.346	-0.902	left vent hypert...
4	0	-1.933	male	non-anginal	-0.096	0.0640	0	0	-0.723	normal	downsloping	2.123	1.637	normal
5	0	-1.489	female	atypical ang	-0.096	-0.8259	0	0	-0.723	normal	upsloping	0.311	0.981	left vent hypert...
6	0	0.173	male	atypical ang	-0.665	-0.2069	0	0	-0.723	normal	upsloping	-0.207	1.243	normal
7	1	0.838	female	asymptomatic	0.473	0.4122	0	0	1.428	normal	downsloping	2.209	0.455	left vent hypert...
8	0	0.284	female	asymptomatic	-0.665	2.0759	0	1	-0.723	normal	upsloping	-0.379	0.586	normal
9	1	0.949	male	asymptomatic	-0.096	0.1414	0	0	0.353	reversible defect	flat	0.311	-0.114	left vent hypert...
10	1	-0.159	male	asymptomatic	0.473	-0.8453	1	1	-0.723	reversible defect	downsloping	1.777	0.236	left vent hypert...

Após o PCA, posso trabalhar somente com os PCs (PC1, PC2 etc.):

variance	iameter narrowin	PC1 0.256458	PC2 0.132157	age	gender	chest pain	rest SBP	cholesterol	ng blood sugar >	exerc ind ang	ajor vessels color	thal	slope peak exc ST	ST by exercise	max HR	rest ECG
1	0	-0.83553	0.315858	0.949	male	typical ang	0.758	-0.2649	1	0	-0.723	fixed defect	downsloping	1.087	0.017	left vent hypert...
2	1	-3.52298	0.802846	1.392	male	asymptomatic	1.611	0.7604	0	1	2.504	normal	flat	0.397	-1.822	left vent hypert...
3	1	-2.65239	-1.11573	1.392	male	asymptomatic	-0.665	-0.3423	0	1	1.428	reversible defect	flat	1.346	-0.902	left vent hypert...
4	0	1.2955	-0.658518	-1.933	male	non-anginal	-0.096	0.0640	0	0	-0.723	normal	downsloping	2.123	1.637	normal
5	0	1.83815	-0.176243	-1.489	female	atypical ang	-0.096	-0.8259	0	0	-0.723	normal	upsloping	0.311	0.981	left vent hypert...
6	0	1.63133	0.176836	0.173	male	atypical ang	-0.665	-0.2069	0	0	-0.723	normal	upsloping	-0.207	1.243	normal
7	1	-1.84299	0.745071	0.838	female	asymptomatic	0.473	0.4122	0	0	1.428	normal	downsloping	2.209	0.455	left vent hypert...
8	0	0.546397	1.39261	0.284	female	asymptomatic	-0.665	2.0759	0	1	-0.723	normal	upsloping	-0.379	0.586	normal
9	1	-1.17028	-0.0226817	0.949	male	asymptomatic	-0.096	0.1414	0	0	0.353	reversible defect	flat	0.311	-0.114	left vent hypert...
10	1	-0.932747	-1.04341	-0.159	male	asymptomatic	0.473	-0.8453	1	1	-0.723	reversible defect	downsloping	1.777	0.236	left vent hypert...
11	0	0.25666	-0.621066	0.284	female	asymptomatic	0.473	-1.0581	0	0	-0.723	fixed defect	flat	-0.552	-0.070	normal
12	0	0.0551549	1.21279	0.173	male	atypical ang	0.473	0.9152	0	0	-0.723	normal	flat	0.225	0.149	left vent hypert...
13	1	-0.645763	-0.0695848	0.838	female	asymptomatic	-0.096	0.1800	1	1	-0.723	fixed defect	flat	-0.379	-0.333	left vent hypert...
14	0	2.02881	-0.0548133	-1.157	male	atypical ang	-0.665	0.3155	0	0	-0.723	reversible defect	upsloping	-0.897	1.024	normal

Substituídos pelos componentes principais

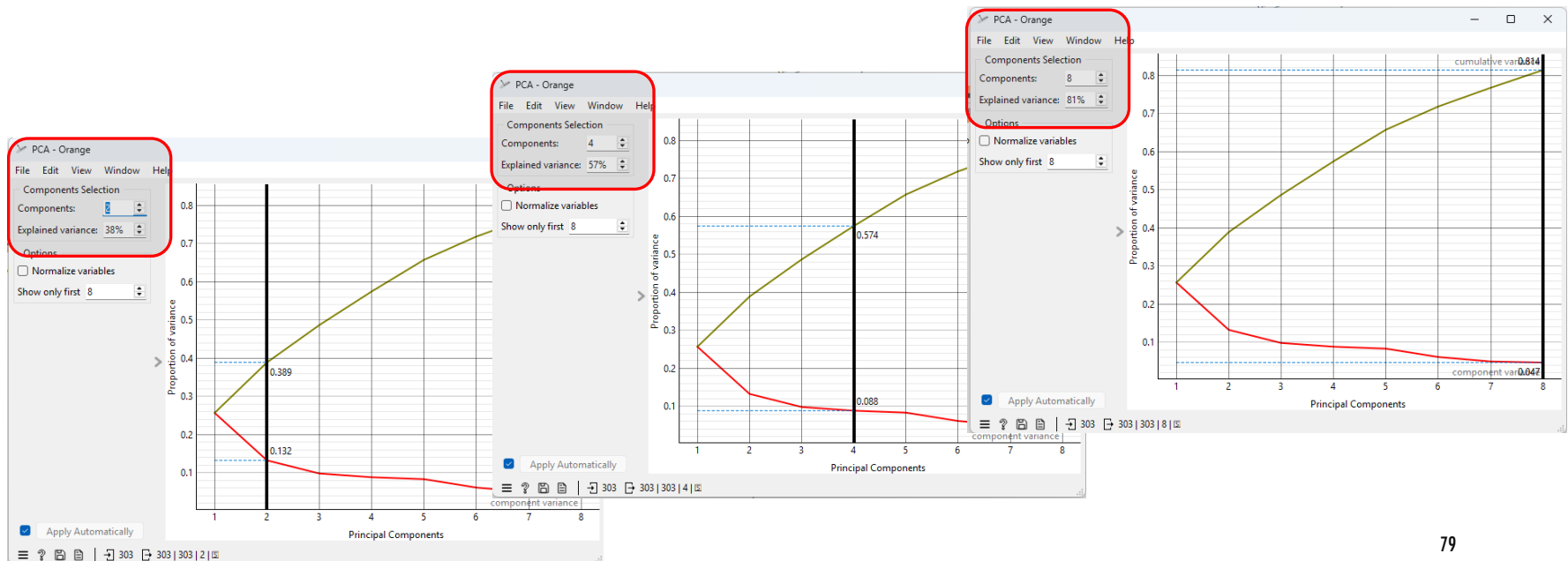


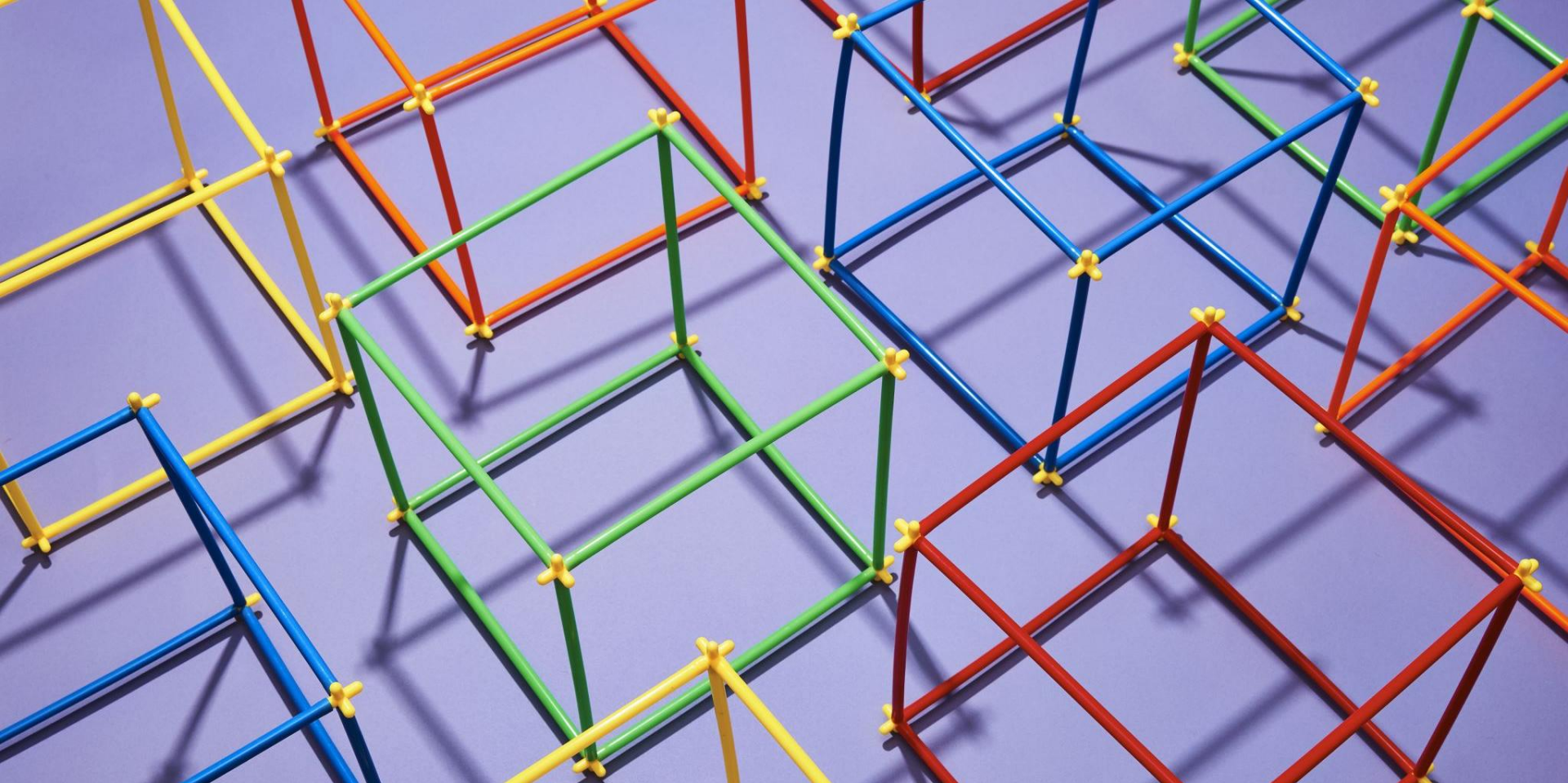
PIPELINE PREDITIVO

ENGENHARIA DE ATRIBUTOS

Limitação do PCA: Baixa Correlação entre Variáveis

A escolha de quantos PCs leva em consideração a **quantidade de variância** explicada. O principal benefício do PCA é reduzir a dimensionalidade. Logo, o número de componentes para a capturar uma quantidade grande de variância não pode ser tão grande quanto o número de variáveis explicativas original (como no caso abaixo).





V — PIPELINE PREDITIVO ESPECIFICAÇÃO DE MODELOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)

ESPECIFICAÇÃO DE MODELOS CLÁSSICOS MAIS USADOS EM ANP

Supervisionados Clássicos

- Regressão Linear
- Regressão Logística
- KNN
- Árvores / Floresta Aleatória (Random Forest)

Não Supervisionado Clássicos

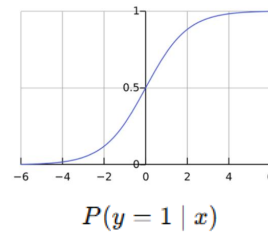
- PCA (Principal Component Analysis) *

* O PCA às vezes é listado como “modelo” em sentido amplo porque ajusta parâmetros a partir dos dados (ou seja, ele “aprende uma representação”). No entanto, não é um modelo preditivo, pois não estima uma variável alvo (y). Rigorosamente falando (sentido restrito), PCA é técnica de redução de dimensionalidade (engenharia de atributos).

ESPECIFICAÇÃO DE MODELOS CLÁSSICOS POUCO USADOS EM PLN

Outros Modelos menos Usados em PLN

Naive Bayes - O problema central é a **hipótese de independência entre variáveis**, que raramente se sustenta em dados tabulares reais. Em análise preditiva, atributos costumam ser correlacionados (ex: renda, gasto, idade), e o modelo ignora essas relações, levando a estimativas probabilísticas distorcidas. Além disso, sua simplicidade limita a capacidade de capturar interações mais complexas entre variáveis, resultando em desempenho inferior a modelos como regressão logística, árvores e ensembles.

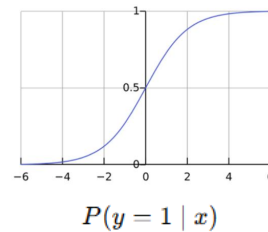


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

No aprendizado supervisionado, modelos são treinados a partir de dados rotulados ($X \rightarrow y$), com o objetivo de **aprender a relação entre variáveis explicativas e a variável alvo** para realizar previsões.

Essa abordagem se divide em dois tipos de problema: **regressão**, quando o objetivo é prever **valores contínuos** (y contínuo), e **classificação**, quando o objetivo é prever **classes/categorias** (y discreto).

No contexto de classificação em PLN, a regressão logística é um dos modelos mais utilizados, devido à sua simplicidade (:o), eficiência e boa **interpretabilidade**.

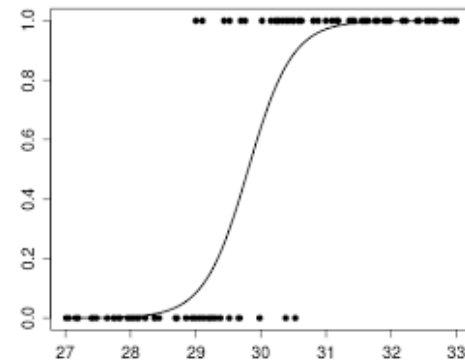
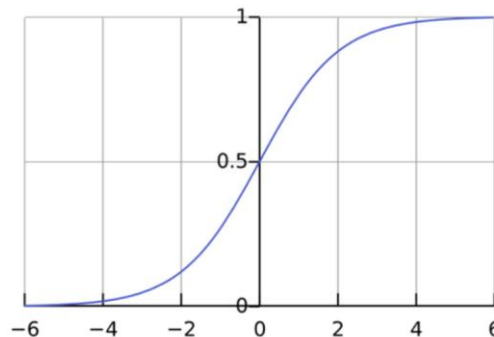


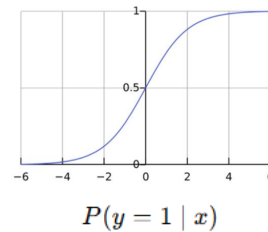
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Em **análise preditiva**, a classificação é uma das tarefas mais comuns (inadimplência, churn, fraude etc.). Nestes casos, a regressão logística é utilizada para estimar a probabilidade de uma observação pertencer à classe 1, dado um conjunto de atributos.

Para isso, o modelo transforma uma saída linear em uma **probabilidade entre 0 e 1** por meio de uma **função específica** (*sigmoide*, para binária, *softmax* para classes múltiplas).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

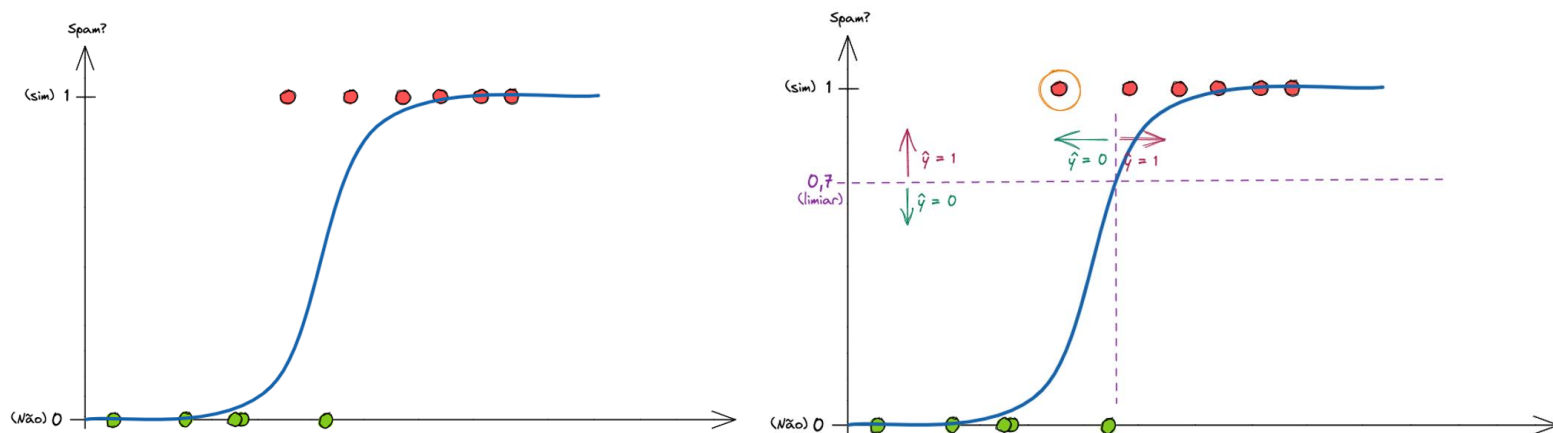




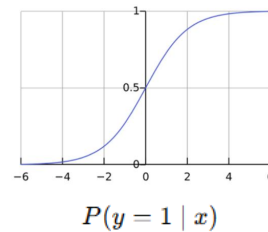
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Regra de decisão

O modelo retorna uma probabilidade $p = \sigma(z)$ para a classe positiva. Para obter a classificação, define-se um limiar, geralmente 0.5.



Esse limiar não é obrigatório (pode ser ajustado conforme o custo do erro). Deixar passar fraude ou bloquear compra legítima? Posso definir limiar em 0.3, e ser conservador; ou 0.7, e ser agressivo.



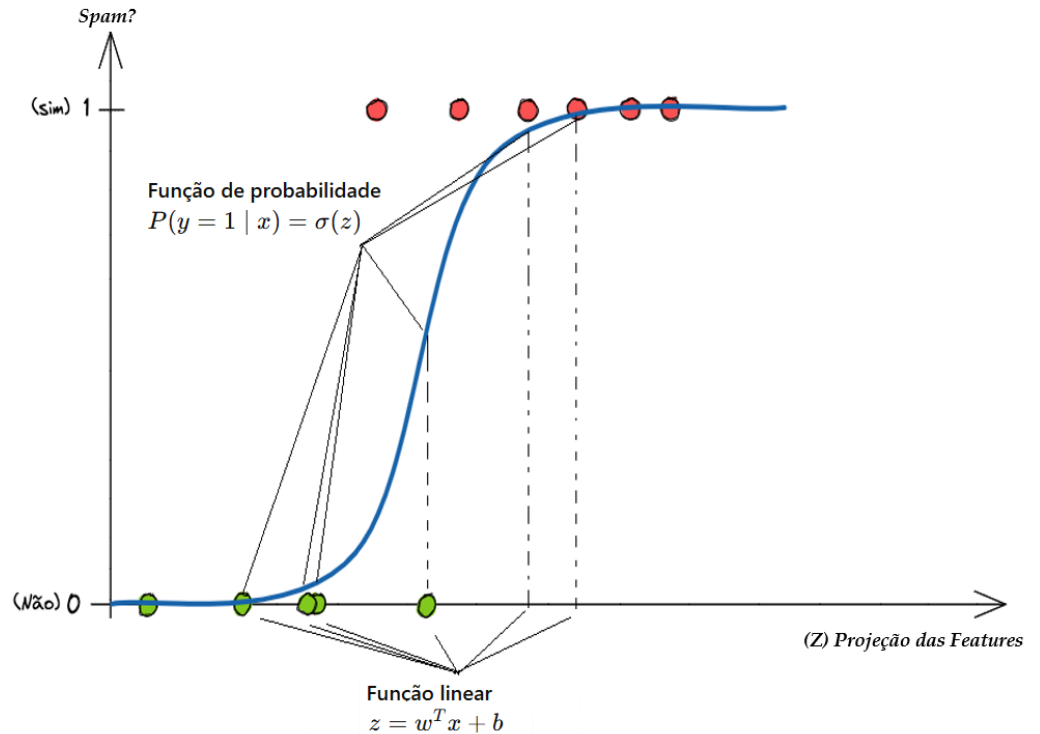
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

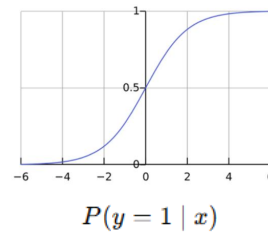
Regra de decisão

Os pontos no gráfico representam os **dados reais** do conjunto de treino, onde cada observação está **rotulada como 0 ou 1** (por exemplo, não spam ou spam). Ou seja, representam os **rótulos**.

Durante o treinamento, o modelo ajusta seus parâmetros (w, b) para gerar a “melhor” curva sigmoide representa a probabilidade de um registro **pertencer à classe 1**.

A curva **não separa** **perfeitamente os pontos**.



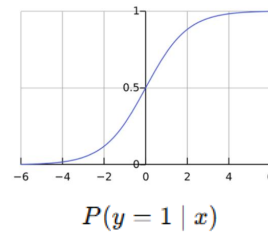


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

A regressão logística clássica utiliza a função **sigmoide** (função logística) para problemas binários. Para problemas multiclasse, utiliza-se uma generalização desse modelo baseada na função **softmax**

O modelo calcula inicialmente uma **combinação linear** das variáveis de entrada, cujo resultado pode variar de $-\infty$ a $+\infty$, e em seguida aplica uma função de transformação para convertê-lo em probabilidades. No caso binário, utiliza-se a função sigmoide; no caso multiclasse, a função softmax.

$$z = w \cdot X + b \quad \longrightarrow \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

X representa o vetor de entrada (*features*) do modelo.

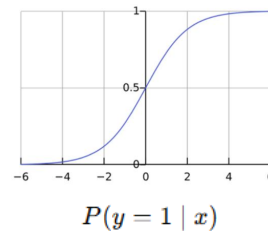
Passo (1) - Durante o treinamento, o modelo utiliza dados rotulados (X, y) para ajustar os parâmetros w (pesos) e b (viés). Com esses parâmetros, o modelo calcula a combinação linear:

$$z = w^T x + b$$

É esse valor z que pode variar de $-\infty$ a $+\infty$.

Passo (2) - Calculado o valor de z , o próximo passo é calcular, via função sigmoide $\sigma(z)$, a probabilidade entre 0 e 1 utilizada na decisão de classe.

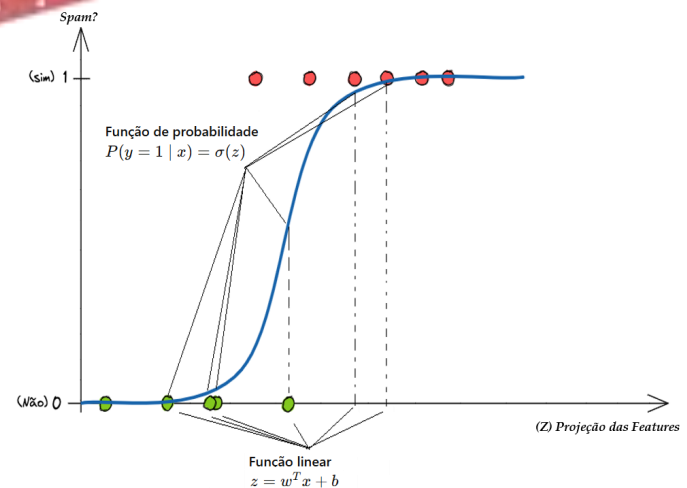
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

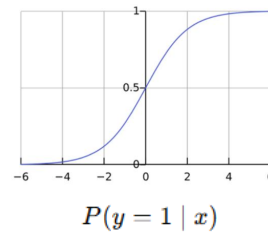


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Os parâmetros w (pesos) e b (viés) são **ajustados durante o treinamento** por meio de um processo de **otimização** que **minimiza** o erro entre as previsões do **modelo** e os **valores reais**. O detalhamento desse **processo** não será **abordado neste momento**.

$$(w, b) = \arg \min \sum_i [-y_i \log(\sigma(w^T x_i + b)) - (1 - y_i) \log(1 - \sigma(w^T x_i + b))]$$





ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

$|V|$: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra “**excelente**” = +2
- Peso (w) da palavra “**ruim**” = -2
- Viés (b) = 0

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto “o produto é excelente”:

$$X = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

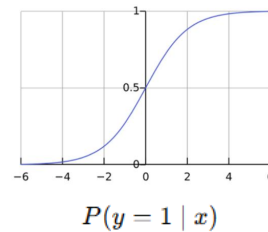
Cálculo para “o produto é excelente”:

$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0 \quad z = w^T x + b$$

$$z = (2 \cdot 1) + (-2 \cdot 0) = 2$$

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx 0.88$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

$|V|$: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra “**excelente**” = +2
- Peso (w) da palavra “**ruim**” = -2
- Viés (b) = 0

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto “o produto é ruim”:

$$X = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Cálculo para “o produto é ruim”:

$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0$$

$$z = w \cdot X + b$$

$$z = (2 \cdot 0) + (-2 \cdot 1) = -2$$

$$\sigma(-2) = \frac{1}{1 + e^2} \approx 0.12$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Quando usar?

Problemas de **classificação** (probabilidade de ocorrência de um evento)

Relação **aproximadamente linear** entre atributos e a probabilidade do evento

Quando é necessário um modelo **baseline simples e interpretável**

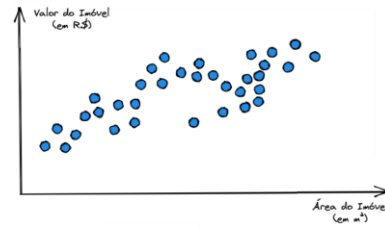
Vantagens

Treinamento rápido e estável em cenários preditivos

Alta interpretabilidade dos coeficientes

Gera probabilidades diretamente utilizáveis para decisão

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



A regressão linear é um modelo utilizado quando o objetivo é estimar um **valor numérico contínuo a partir de variáveis explicativas**. Nesse contexto, assume-se que a variável-alvo pode ser representada como uma **combinação linear dessas variáveis**, permitindo construir uma função matemática que aproxima o comportamento observado nos dados históricos.

Podemos tentar prever a **receita de uma empresa**, as **vendas de um produto ou serviço**, o **valor de uma ação no mercado financeiro** etc.

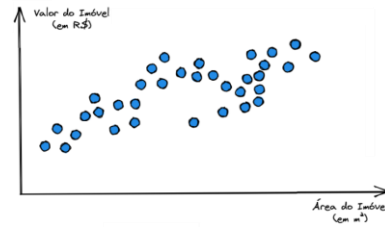
Vamos lembrar que uma fórmula linear tem a seguinte estrutura.

$$y = wx + b$$

Quando temos mais atributos (mais x's), a **Regressão Linear é Múltipla**:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



Imagine que você quer prever o **preço de uma casa** (y).

Considere um dataset em que a variável explicativa é apenas o **tamanho da casa** (m^2). É possível escrever o preço da casa como uma função dessa variável, ou seja, expressar y em função de x ?

Modelo (combinação linear):

$$y = w_1 x_1 + b$$

Parâmetros do modelo (exemplo):

- $w_1 = 2000$ (cada m^2 aumenta o preço em 2000)
- $b = 50000$ (valor base / intercepto)



Instância (uma casa específica):

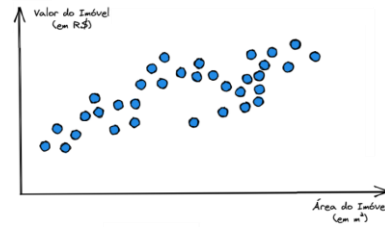
- tamanho = 100

Aplicando:

$$y = 2000 \cdot 100 + 50000$$

$$y = 200000 + 50000 = 250000$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



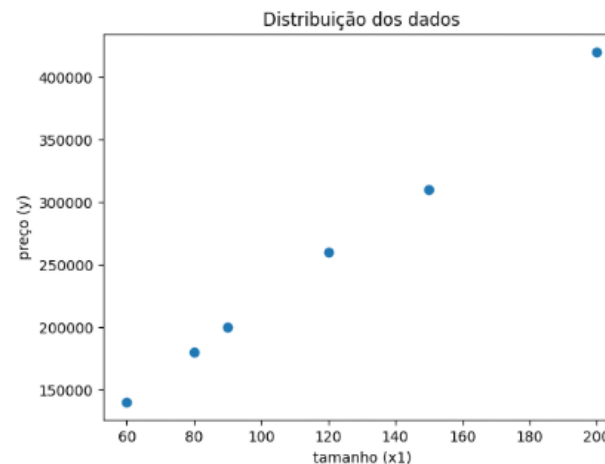
Considerando todas as linhas simultaneamente, existe um único conjunto de parâmetros w_1 , w_2 , w_3 e b , que resolve perfeitamente todos os casos?

Na prática, não existe ajuste perfeito para todos os pontos. O modelo aprende parâmetros (w_1 , w_2 , w_3 e b) que produzem o **melhor ajuste global possível**, representado por um hiperplano, e não por um ajuste exato ponto a ponto.

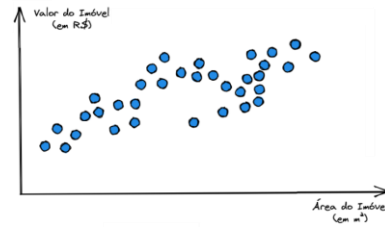


Dataset:

tamanho (x_1)	preço (y)
80	180000
120	260000
60	140000
200	420000
150	310000
90	200000



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



Imagine que você quer prever o **preço de uma casa** (y).

Considere um dataset em que as variáveis explicativas são o tamanho da casa (m^2), o número de quartos e a idade do imóvel. É possível escrever o preço da casa como uma função dessas variáveis, ou seja, expressar y em função de x_1 , x_2 e x_3 ?

Modelo (combinação linear):

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Instância (uma casa específica):

- tamanho = 100
- quartos = 3
- idade = 10

Parâmetros do modelo (exemplo):

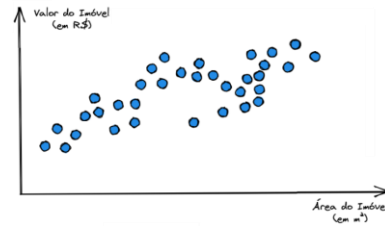
- $w_1 = 2000$ (cada m^2 aumenta o preço em 2000)
- $w_2 = 10000$ (cada quarto aumenta 10k)
- $w_3 = -1500$ (cada ano reduz 1500)
- $b = 50000$ (valor base / intercepto)

Aplicando:

$$\begin{aligned} y &= 2000 \cdot 100 + 10000 \cdot 3 - 1500 \cdot 10 + 50000 \\ y &= 200000 + 30000 - 15000 + 50000 = 265000 \end{aligned}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



Considerando todas as linhas simultaneamente, existe um único conjunto de parâmetros w_1 , w_2 , w_3 e b , que resolve perfeitamente todos os casos?

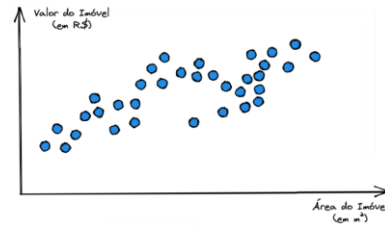
Na prática, não existe ajuste perfeito para todos os pontos. O modelo aprende parâmetros (w_1 , w_2 , w_3 e b) que produzem o **melhor ajuste global possível**, representado por um hiperplano, e não por um ajuste exato ponto a ponto.



Dataset:

tamanho (x_1)	quartos (x_2)	idade (x_3)	preço (y)
80	2	20	180000
120	3	15	260000
60	2	30	140000
200	4	5	420000
150	3	10	310000
90	2	25	200000

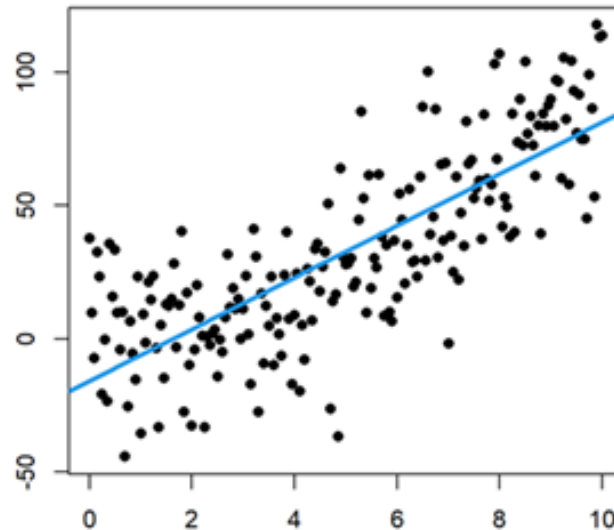
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



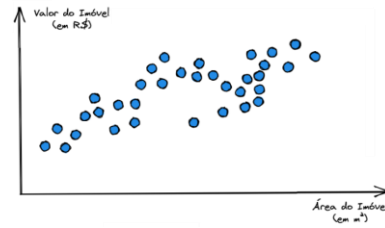
Regressão Linear Simples

A regressão linear segue a forma $y = ax + b$, onde há apenas **uma variável explicativa**. O parâmetro a representa o peso dessa variável na previsão, enquanto b é o intercepto. A entrada x corresponde à única variável utilizada para estimar y .

$$Y = 9.72x - 15.69$$



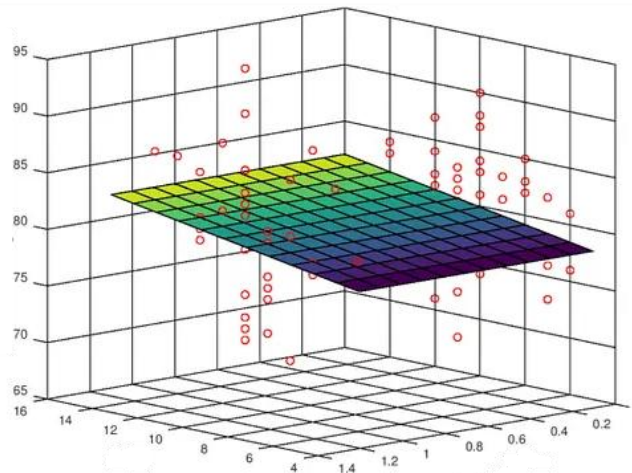
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



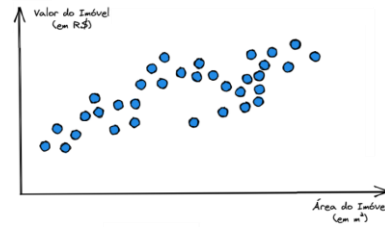
Regressão Linear Múltipla

Quando há várias variáveis explicativas, a regressão linear assume a forma $y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$. Cada coeficiente w_i indica o impacto da respectiva variável x_i na previsão, enquanto b permanece como intercepto. A entrada x representa o vetor de características utilizadas para estimar a variável-alvo y .

$$y = w_1x_1 + w_2x_2 + b$$

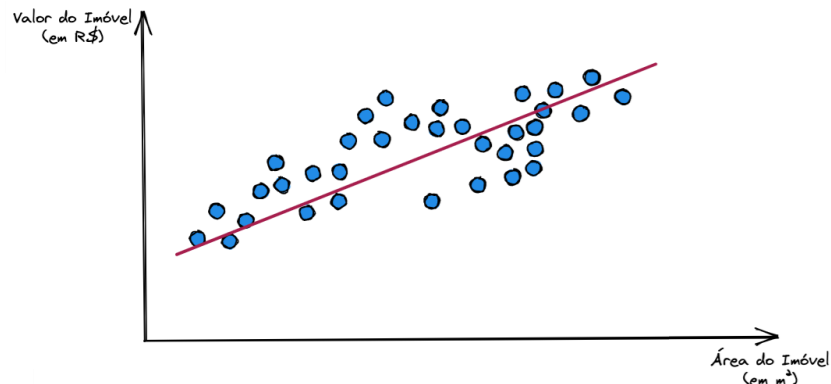
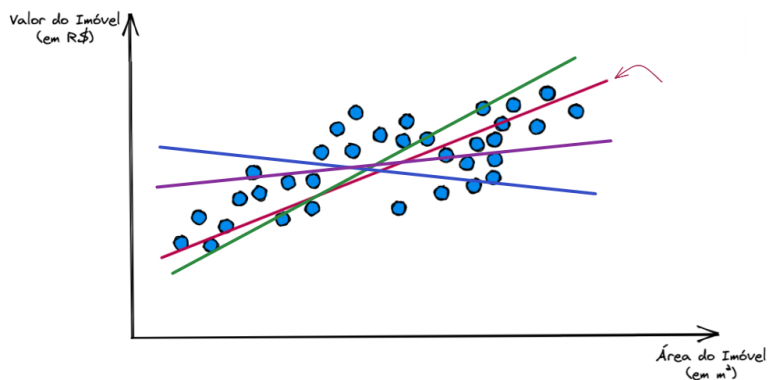


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR



O Melhor Ajuste

A reta de regressão linear é aquela que melhor se ajusta aos dados, ou seja, a que minimiza o erro entre os valores reais e os valores previstos. O modelo ajusta os parâmetros w e b por meio de um processo de otimização, buscando reduzir essa diferença. O erro de um ponto é medido pela **diferença entre o valor real e o valor previsto pela reta**, sendo chamado de residual.



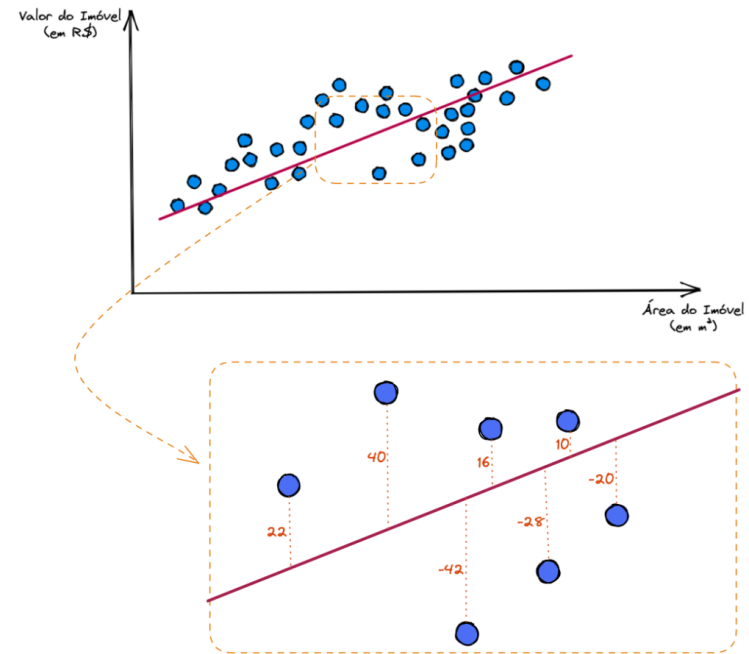
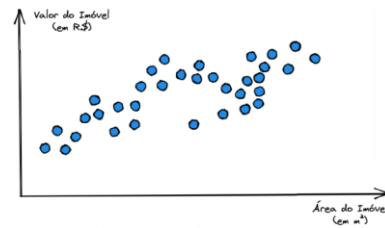
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR

O Melhor Ajuste

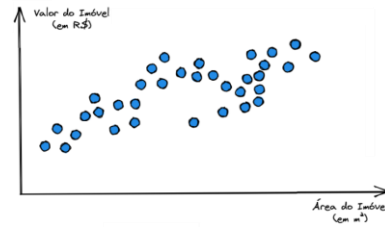
A **reta** de regressão linear define um valor previsto ($\hat{y}$) para cada ponto do conjunto de dados. A diferença entre o valor real (y) e o valor previsto é chamada de **erro (residual)**, representando a distância vertical entre o ponto e a reta.

$$\sum e_i^2 = 22^2 + 40^2 + 16^2 + 10^2 + (-42)^2 + (-28)^2 + (-20)^2$$

$$\sum e_i^2 = 484 + 1600 + 256 + 100 + 1764 + 784 + 400 = 5388$$

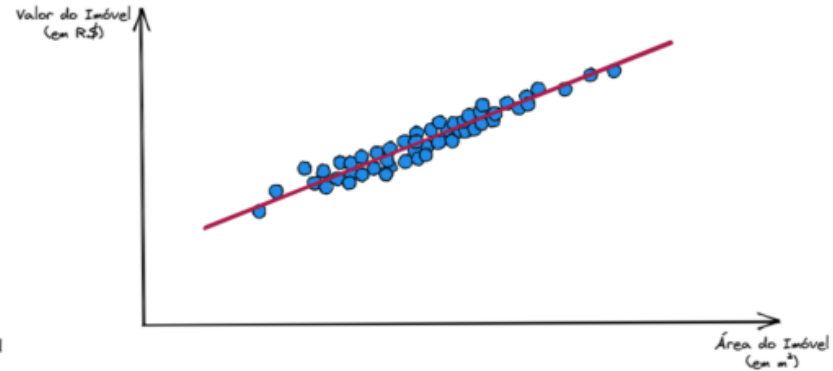
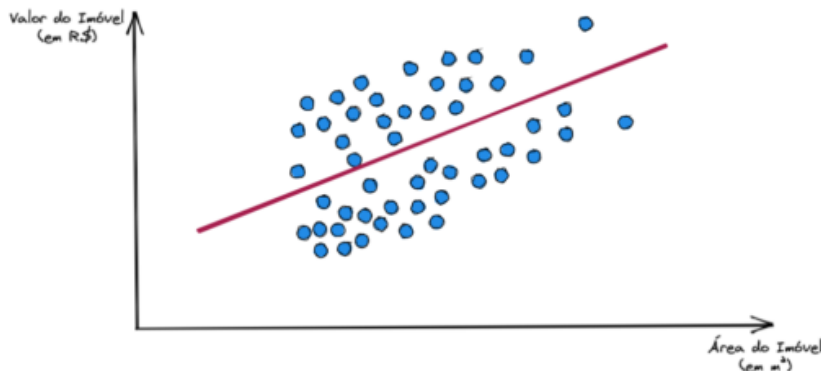


ESPECIFICAÇÃO DE MODELOS REGRESSÃO LINEAR - MÉTRICAS



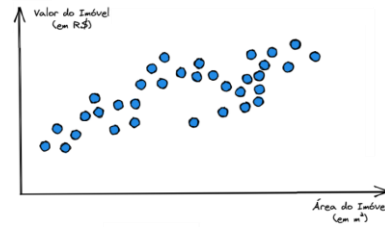
Métricas

As métricas são utilizadas para avaliar o desempenho de um modelo de regressão, indicando **o quanto ele consegue explicar os dados observados**. Após ajustar a melhor reta aos dados, é necessário verificar se ela é suficientemente boa para realizar previsões. A **melhor reta possível** para um conjunto de dados **nem sempre apresenta desempenho adequado**.



ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR - MÉTRICAS



R^2 (coeficiente de determinação)

- Mede quanto da variabilidade da variável-alvo (y) é explicada pelo modelo. Ele é definido como a comparação entre o erro do modelo gerado e o erro de um modelo de referência simples (ingênuo) que prevê sempre a média histórica da variável-alvo, ignorando as variáveis explicativas (x).

Interpretação:

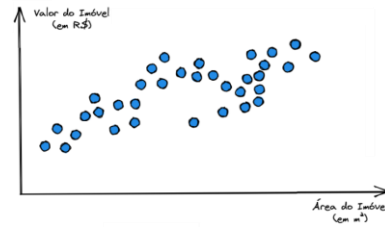
- 0 $\rightarrow$ modelo não explica nada
- 1 $\rightarrow$ explica tudo

$$R^2 = 1 - \frac{\text{erro do modelo}}{\text{erro de prever a média}}$$

Um R^2 igual a 0,8 indica que o modelo explica 80% da variabilidade da variável-alvo. Valores baixos de R^2 indicam que o modelo explica pouco dessa variabilidade, o que pode estar associado a baixo desempenho preditivo, embora isso **não seja uma regra absoluta**.

ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR - MÉTRICAS



MSE (Mean Squared Error)

- Mede o erro médio ao quadrado entre valores reais e previstos.
- Intervalo: $[0, +\infty]$

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

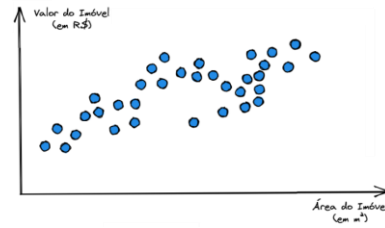
Interpretação:

- $0 \rightarrow$ modelo perfeito
- Quanto maior $\rightarrow$ pior o modelo

MSE penaliza erros grandes de forma mais intensa. É sensível a *outliers*, pois utiliza erro ao quadrado. Valores altos indicam grande dispersão dos erros.

ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR - MÉTRICAS



RMSE (Root MSE)

- Raiz quadrada do MSE (mesma unidade de y).
- Intervalo: $[0, +\infty]$

$$RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

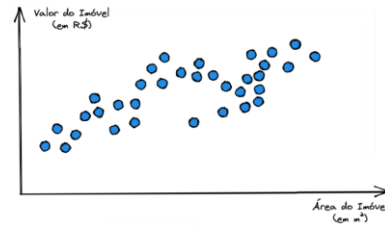
Interpretação:

- $0 \rightarrow$ modelo perfeito
- Quanto maior $\rightarrow$ pior o modelo

RMSE facilita a interpretação por estar na mesma escala da variável alvo. Assim como o MSE, também é sensível a outliers.

ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR - MÉTRICAS



MAE (Mean Absolute Error)

- Média do valor absoluto dos erros.
- Intervalo: $[0, +\infty]$

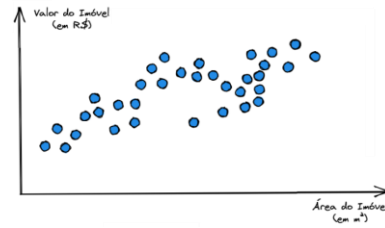
$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Interpretação:

- $0 \rightarrow$ modelo perfeito
- Quanto maior $\rightarrow$ pior o modelo

MAE representa o erro médio real. É mais robusto a outliers que o RMSE.

ESPECIFICAÇÃO DE MODELOS REGRESSÃO LINEAR - MÉTRICAS



MAE versus RMSE

$$MAE = \frac{1}{n} \sum |y - \hat{y}| \quad RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

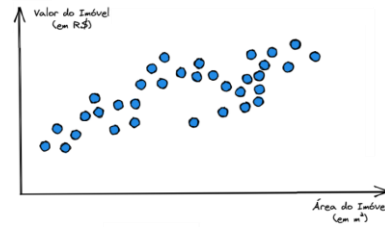
- **MAE $\approx$ RMSE** (ambos altos) $\rightarrow$ modelo ruim geral
- **MAE $\approx$ RMSE** (ambos baixos) $\rightarrow$ erros pequenos e distribuídos
- **RMSE $\gg$ MAE** $\rightarrow$ presença de erros grandes (possíveis **outliers**)

Obs.: matematicamente o valor de RMSE sempre será maior ou igual ao valor de MAE, isto é, **MSE $\geq$ MAE sempre**.

Obs.: na prática, RMSE é preferido ao MSE para interpretação, pois possui a mesma unidade da variável alvo (y).

ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR - MÉTRICAS



No dia a dia, o conjunto mais comum e interpretável de métricas são, de fato, **MAE + MSE/RMSE + R^2** ; porém, há diversas outras.

Você pode ver detalhes sobre essas e outras métricas em:

<https://scikit-learn.org/stable/api/sklearn.metrics.html>



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LINEAR

Quando usar?

Problemas de **regressão** (prever valores contínuos)

Relação **aproximadamente linear** entre atributos e o valor alvo

Quando é necessário um modelo **baseline simples e interpretável**

Vantagens

Treinamento rápido e estável em cenários preditivos

Alta interpretabilidade dos coeficientes

Baixo custo computacional

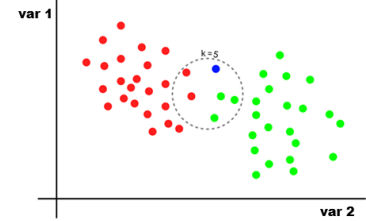
ESPECIFICAÇÃO DE MODELOS

REGRESSÃO LINEAR VS. REGRESSÃO LOGÍSTICA

Aspecto	Regressão Linear	Regressão Logística
Tipo de problema	Regressão (prever valor contínuo)	Classificação (prever probabilidade de evento)
Saída	Valor real (ex: preço, receita)	Probabilidade (0 a 1)
Objetivo preditivo	Estimar magnitude de uma variável	Estimar chance de ocorrência de um evento
Função base	$y = wx + b$	Sigmoide
Interpretação	Impacto direto no valor previsto	Impacto na probabilidade (log-odds)
Vantagens	Simples, rápido, interpretável	Simples, rápido, interpretável
Limitações	Não captura não linearidade	Não captura padrões não lineares complexos
Uso típico	Preço, demanda, tempo	Churn, fraude, conversão
Papel no projeto	Baseline para regressão	Baseline para classificação

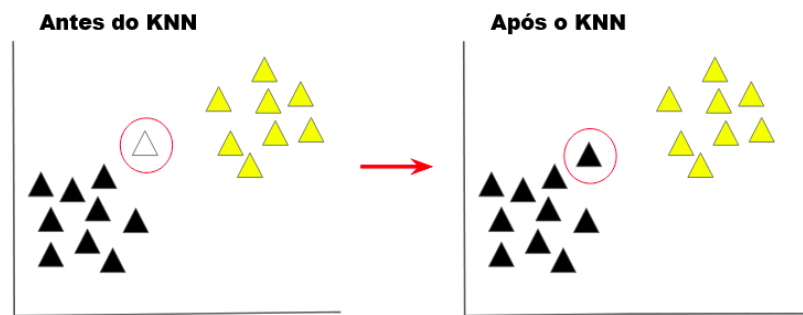
ESPECIFICAÇÃO DE MODELOS

KNN



O **KNN** (*K-Nearest Neighbors*) é um **modelo supervisionado** que realiza previsões com base na **similaridade entre dados**. Para cada nova instância, o modelo identifica os **K vizinhos mais “próximos”** no conjunto de treino para determinar o resultado.

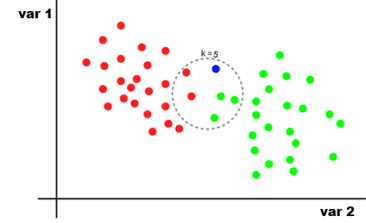
Ele pode ser utilizado em tarefas de classificação e de regressão.



“Diga-me quem são seus vizinhos, que eu te digo quem você é.”

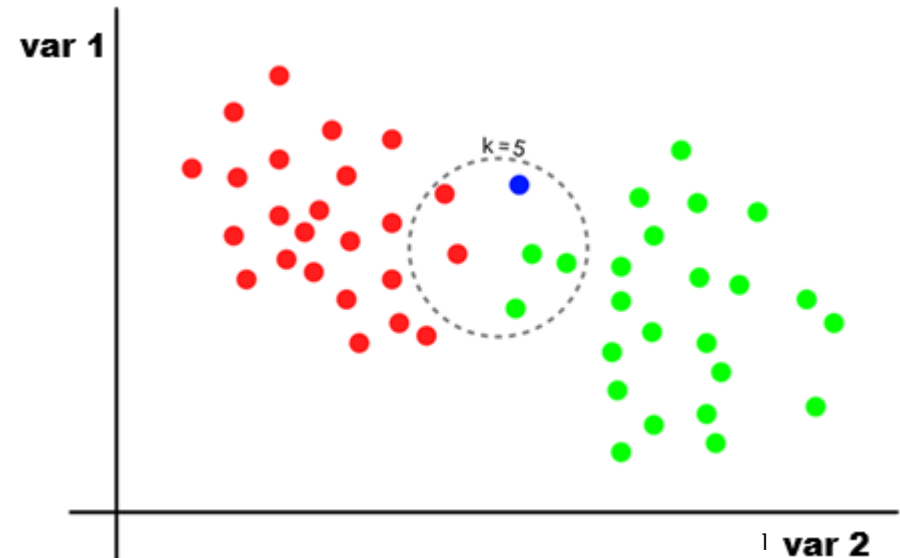
ESPECIFICAÇÃO DE MODELOS

KNN



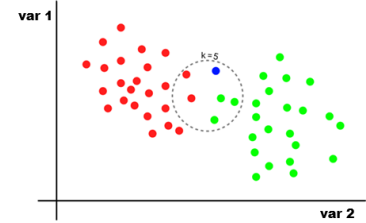
Como funciona o algoritmo

- Define o valor de K
- Recebe um novo dado/ponto/instância para prever
- Calcula a sua distância para **todos os dados de treino**
- Seleciona os K vizinhos mais próximos
- Realiza a predição:
 - Classificação → maioria
 - Regressão → média



ESPECIFICAÇÃO DE MODELOS

KNN



Estratégias de aprendizado

Em machine learning, os algoritmos podem ser diferenciados pela forma como constroem o conhecimento a partir dos dados. Alguns **aprendem um modelo durante o treino** e depois o utilizam para fazer previsões, enquanto **outros** não constroem um modelo explícito e **utilizam diretamente os dados no momento da predição**.

Isso divide as estratégias em dois tipos:

Lazy learning (ex: KNN)

Treinamento inexistente (ou mínimo)

Não constrói modelo explícito

Usa os **dados diretamente na predição**

Predição lenta/tardia

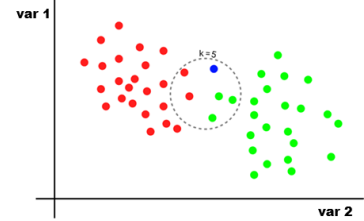
Eager learning (ex: Regressão Linear, Árvores etc.)

Treinamento explícito

Constrói modelo explícito

Usa apenas o modelo na predição

Predição rápida/antecipada



ESPECIFICAÇÃO DE MODELOS

KNN

Métrica de distância

Define como medimos a proximidade entre dois pontos. Isso **impacta diretamente** quais são os vizinhos.

A distância mais utilizada é a **Euclidiana**, que mede a distância em linha reta. Por padrão (**default**), é a mais usada em muitas implementações (ex: scikit-learn).

Também é possível utilizar a distância **Manhattan**, que é mais robusta a **outliers**. Ela é calculada pela soma das diferenças absolutas entre as variáveis.

Distância Euclidiana

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Distância em linha reta

Mais usada

Distância Manhattan

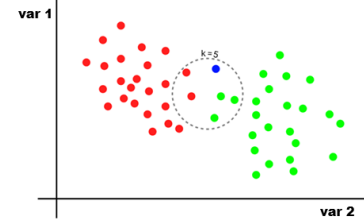
$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Soma das diferenças absolutas

Mais robusta a outliers

ESPECIFICAÇÃO DE MODELOS

KNN



Métrica de distância

A similaridade de cosseno mede o grau de alinhamento entre dois vetores, avaliando o ângulo entre eles em vez da distância em linha reta. Diferentemente das métricas tradicionais, ela não considera a magnitude dos vetores, mas sim a proporção entre suas componentes.

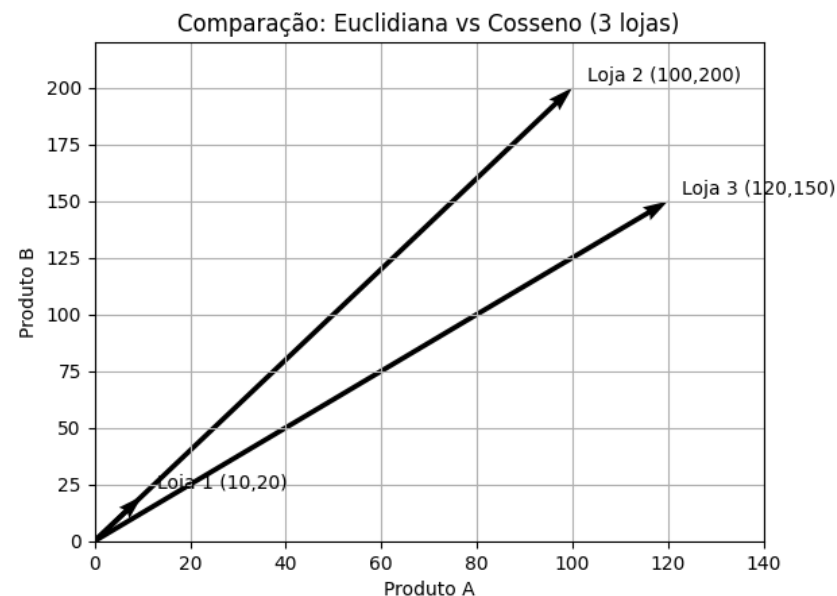
Por exemplo: duas lojas podem apresentar volumes de vendas próximos, mas **padrões de distribuição distintos entre produtos**; nesse caso, a distância euclidiana indica proximidade, enquanto a similaridade de cosseno revela similaridade no mix de vendas.

Similaridade de cosseno:

$$\cos(\theta) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \cdot \sqrt{\sum_{i=1}^n y_i^2}}$$

Distância do cosseno:

$$d(x, y) = 1 - \cos(\theta)$$



SIMILARIDADE DE COSSENO: FOCO NO MIX DE VENDAS (PROPORÇÃO), NÃO NO VOLUME

Exemplo: vendas de 2 produtos (A e B) em 3 lojas

DADOS DE VENDAS

Loja	Produto A (unidades)	Produto B (unidades)	Vetor (A, B)
Loja 1 (menor volume)	10	20	(10, 20)
Loja 2 (mesmo mix da Loja 1)	100	200	(100, 200)
Loja 3 (mix diferente)	120	150	(120, 150)

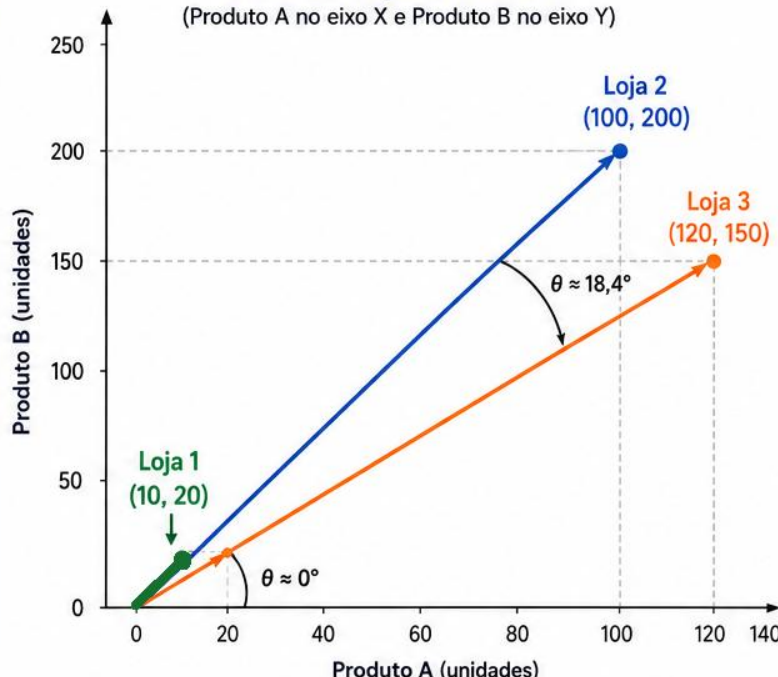


Loja 1 e Loja 2 têm o **MESMO MIX** ($B = 2 \times A$). Apenas volumes diferentes.

Loja 3 tem um **MIX DIFERENTE** (proporção de A maior e B menor).

REPRESENTAÇÃO VETORIAL

(Produto A no eixo X e Produto B no eixo Y)



DISTÂNCIA EUCLIDIANA (considera magnitude)



Foca no volume absoluto (tamanho do vetor).

- Lojas com mais vendas ficam "mais longe", mesmo que o mix seja igual.

DISTÂNCIAS EUCLIDIANAS

$d(\text{Loja 1, Loja 2}) = 201,25$ (muito distante!)
 $d(\text{Loja 2, Loja 3}) = 53,85$ (mais próxima)
 $d(\text{Loja 1, Loja 3}) = 154,32$ (distante)



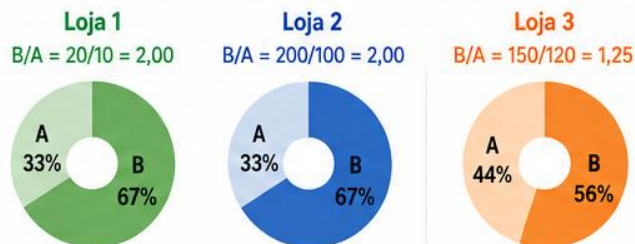
SIMILARIDADE DE COSSENO (foca na direção / mix)

- Foca no mix de vendas (proporção entre A e B).
- Vetores com mesma direção têm alta similaridade, independentemente do volume.

SIMILARIDADES DE COSSENO ($\cos \theta$)

$\cos(\text{Loja 1, Loja 2}) = 1,000$ (mesmo mix!)
 $\cos(\text{Loja 2, Loja 3}) = 0,948$
 $\cos(\text{Loja 1, Loja 3}) = 0,949$

MIX DE VENDAS (proporção B/A)



INTERPRETAÇÃO



Loja 1 e Loja 2:

Mesmo mix (mesma direção) → alta similaridade de cosseno (1,000), mas distância euclidiana muito grande porque o volume é diferente.



Loja 2 e Loja 3:

Volumes parecidos, mas mix diferente → euclidiana pode considerar próxima, enquanto cosseno percebe diferença no padrão de vendas.

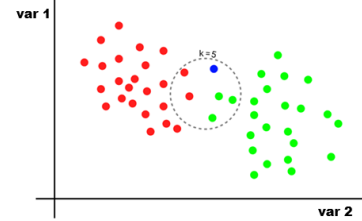


CONCLUSÃO

Quando queremos comparar **PERFIS** ou **MIXES** (comportamento de vendas), a **similaridade de cosseno** faz mais sentido do que as distâncias tradicionais, pois ignora o tamanho e olha apenas para a direção (proporções).



Use cosseno quando o que importa é **COMO** vende (mistura de produtos). Use euclidiana quando o que importa é **QUANTO** vende (volume).



ESPECIFICAÇÃO DE MODELOS

KNN

Escolha do K

O valor de K define quantos vizinhos serão considerados na predição. Esse parâmetro influencia diretamente o comportamento do modelo.

Efeito do K (viés vs variância)

Quando **K é pequeno**, o modelo considera poucos vizinhos e tende a se **ajustar muito aos dados de treino**, capturando inclusive ruídos. Pequenas mudanças nos dados podem alterar significativamente a predição.

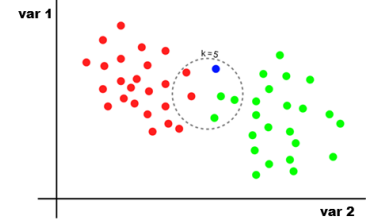
- Modelo aprende **ruído** 🖐️ **alta variância** e risco de **overfitting**
- k pequeno → baixa simplificação → alta variância → overfitting

Quando **K é grande**, o modelo considera muitos vizinhos e passa a suavizar as decisões, ignorando variações locais importantes. Nesse caso, o modelo se torna mais estável, porém **menos sensível aos padrões reais dos dados**

- Modelo **não aprende o padrão** 🖐️ **alto viés** e risco de **underfitting**.
- k grande → alta simplificação → baixa variação → underfitting

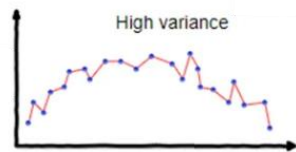
ESPECIFICAÇÃO DE MODELOS

KNN

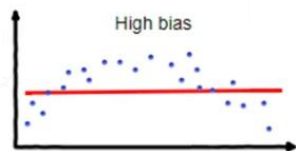


Escolha do K

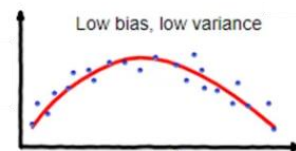
Na plotagem, verdes indica compra ("1") e vermelho indica não compra ("0")



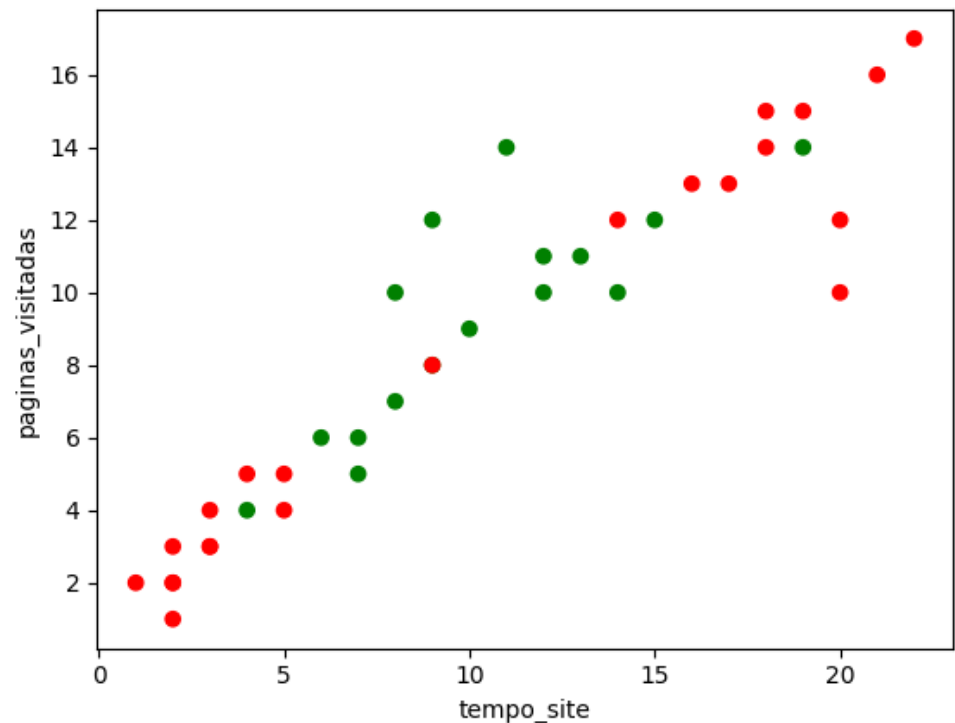
Overfitting

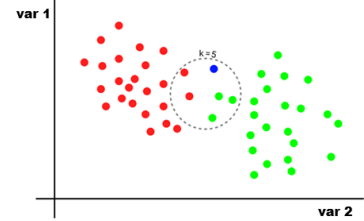


Underfitting



Ideal





ESPECIFICAÇÃO DE MODELOS

KNN

Escalonamento dos dados

Como no KNN a **distância entre os pontos é o principal critério** de decisão, variáveis em **escalas diferentes** fazem com que aquelas com valores maiores dominem o cálculo da distância, distorcendo o resultado e influenciando indevidamente a escolha dos vizinhos.

Por isso, é **essencial escalonar os dados**, colocando todas as variáveis na mesma escala. Dessa forma, cada atributo contribui de maneira equilibrada para o cálculo da distância.

Já vimos essas duas formas de escalonamento (suficientes):

Min-Max (normalização)

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- Escala para [0,1]

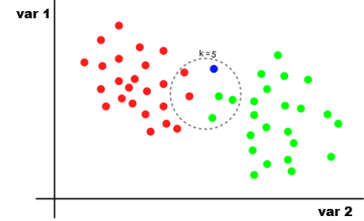
Padronização (Z-score)

$$x' = \frac{x - \mu}{\sigma}$$

- Média = 0, desvio = 1

ESPECIFICAÇÃO DE MODELOS

KNN



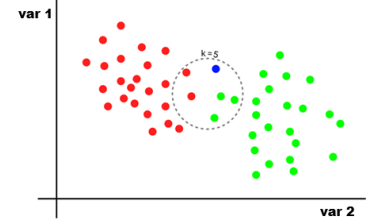
Limitações do KNN – Custo Computacional

No KNN, a predição é **computacionalmente cara**, pois exige o cálculo da distância entre o novo dado e todos os pontos do conjunto de treino a cada predição. Assim, o modelo escala mal com grandes volumes de dados.

Diferente de outros modelos, o KNN **não resume os dados em parâmetros aprendidos**. Assim, a cada nova predição, todo o conjunto de treino precisa ser consultado, resultando em custo proporcional ao número de dados.

ESPECIFICAÇÃO DE MODELOS

KNN



Limitações do KNN – Custo Computacional

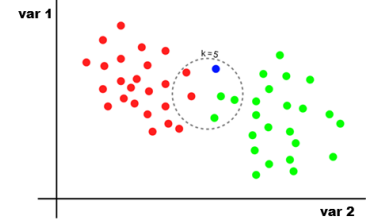
Considere um dataset com **100.000 registros** e **10 variáveis**.

Na regressão linear, durante o treino, o modelo aprende apenas **11 parâmetros** (10 coeficientes + 1 intercepto). Após o treinamento, esses parâmetros resumem todo o padrão dos dados. Para fazer uma predição, basta calcular um produto escalar simples ($\mathbf{y} = \mathbf{w} \cdot \mathbf{x} + \mathbf{b}$), com custo baixo.

Já no KNN, nenhum parâmetro é aprendido. O modelo precisa manter os **100.000 registros** e, a cada nova predição, calcular a distância entre o novo ponto e todos os dados de treino. O processo é significativamente mais custoso.

ESPECIFICAÇÃO DE MODELOS

KNN

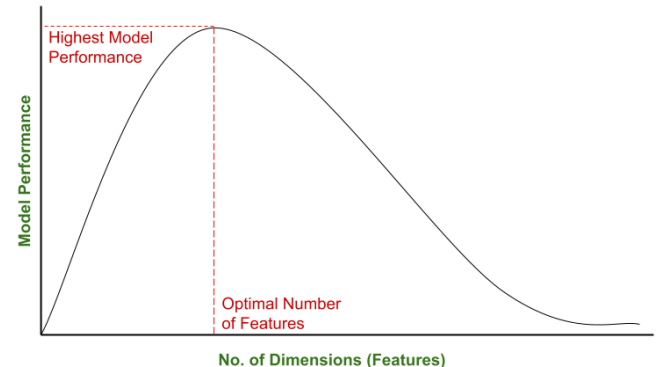


Limitações do KNN – Maldição da Dimensionalidade

Em espaços de **alta dimensionalidade**, a noção de distância perde poder discriminativo, pois as distâncias entre os pontos tendem a se tornar semelhantes, dificultando distinguir quais são realmente próximos (fenômeno conhecido como maldição da dimensionalidade).

A **maldição da dimensionalidade** afeta diversos modelos, mas é especialmente crítica em métodos baseados em distância, como KNN.

Esse é um dos motivos pelos quais o KNN é menos utilizado em problemas de PLN, pois o número de dimensões costuma ser muito elevado.





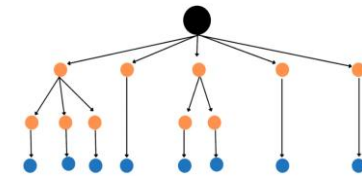
ESPECIFICAÇÃO DE MODELOS KNN

Quando usar?

- há poucos atributos;
- há pouca dimensionalidade;
- existe proximidade local relevante;
- os dados apresentam separação razoável no espaço;
- o volume de dados não é gigantesco;
- E quando o comportamento previsto é não linear.

Limitações

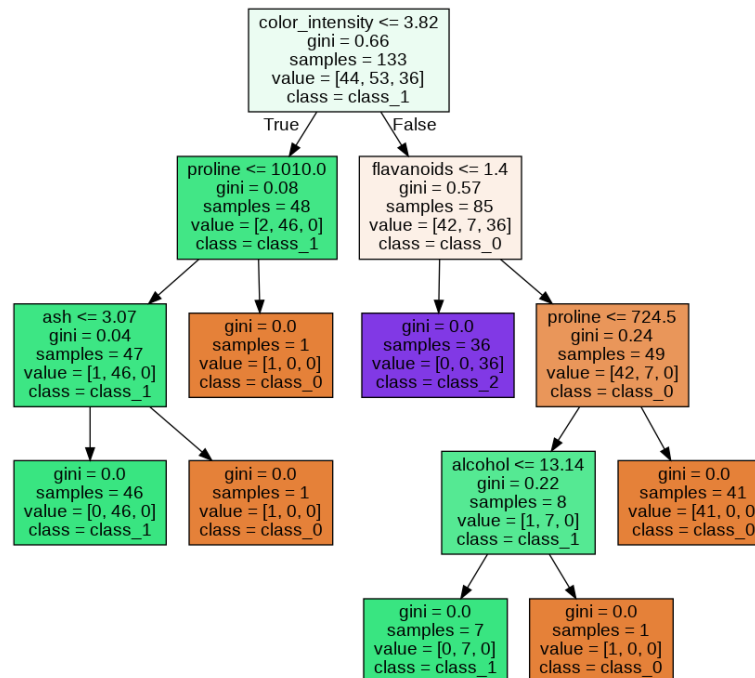
- Alto custo computacional na predição
- Sensível a escala das variáveis
- Sensível a ruído e outliers
- Sofre com alta dimensionalidade (maldição da dimensionalidade)

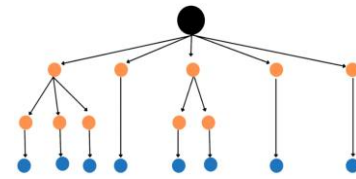


ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Árvores de decisão são modelos supervisionados que realizam previsões por meio de uma estrutura hierárquica de decisões sequenciais. O modelo **divide recursivamente o espaço de dados em regiões mais homogêneas** com base nas variáveis explicativas, utilizando regras do tipo “**se-então**” (if/else).





ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Critério de divisão (*split*)

Em cada divisão (*split*), escolhe-se a variável e o ponto de corte que melhor separam os dados segundo um critério específico do tipo de problema.

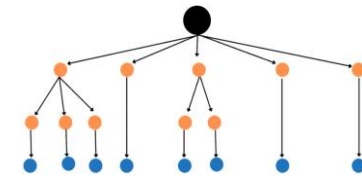
Em **classificação**, busca-se maximizar a pureza dos grupos (usando métricas como **Gini** ou **ganho de informação**). Em **regressão**, busca-se reduzir o erro de previsão, normalmente medido por métricas como o erro quadrático médio (**MSE**). O resultado é uma estrutura em forma de árvore, onde cada caminho da raiz até uma folha representa uma regra de decisão.

Classificação:

- Gini (redução da impureza)
- Entropia (ganho de informação)

Regressão:

- MSE (erro quadrático) — mais comum
- MAE (erro absoluto) — alternativa



ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Índice de Gini (Conrado Gini)

Índice foi criado pelo matemático italiano Conrado Gini, é um instrumento para medir o grau de concentração de renda em determinado grupo. Ele aponta a diferença entre os rendimentos dos mais pobres e dos mais ricos. Varia de 0 (máxima igualdade) a 1 (máxima desigualdade).

Distribuição de renda:

Pessoa	Renda
A	1000
B	1000
C	1000
D	1000
E	10000

Rendas: [1000, 1000, 1000, 1000, 10000]

N = 5

Soma total = 14000

Fórmula (discreta):

$$\text{Gini} = (1 / (2N^2\mu)) * \sum |x_i - x_j|$$

$$\mu = 14000 / 5 = 2800$$

Diferenças absolutas:

$$|1000 - 1000| = 0 \text{ (várias vezes)}$$

$$|1000 - 10000| = 9000 \text{ (4 vezes)}$$

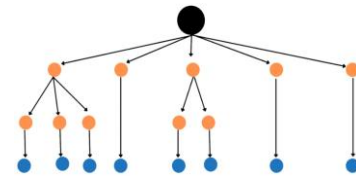
$$|10000 - 1000| = 9000 \text{ (4 vezes)}$$

Soma total das diferenças = 72000

$$\text{Gini} = 72000 / (2 * 25 * 2800)$$

$$\text{Gini} = 72000 / 140000$$

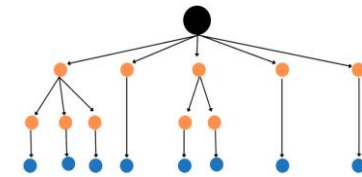
$$\text{Gini} = \mathbf{0.514}$$



ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

Árvores de Decisão – Algoritmo

- Começa com **todos os dados** no nó inicial.
- Testa várias divisões possíveis nas variáveis.
- Calcula o índice de Gini em cada divisão.
- Escolhe a divisão com menor impureza.
- Separa os dados em novos nós.
- Repete o processo em cada nó.
- Para quando o **nó fica puro** ou **atinge o limite definido**.

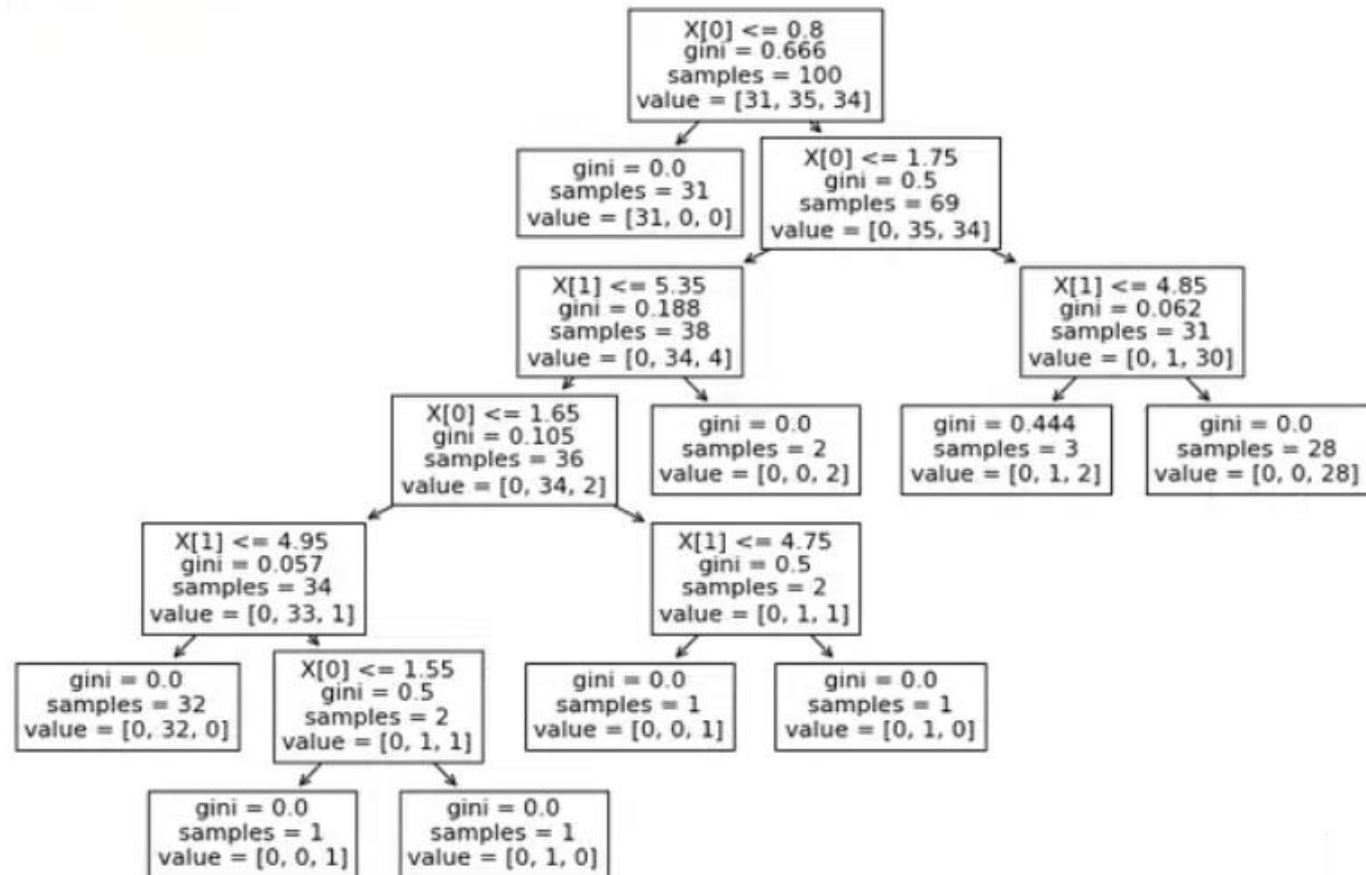


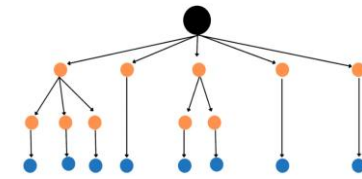
ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

renda_anual	tempo_emprego	risco
18000	1	Alto risco
22000	2	Alto risco
35000	3	Médio risco
42000	5	Médio risco
55000	6	Médio risco
70000	7	Baixo risco
85000	8	Baixo risco
90000	4	Médio risco
65000	3	Médio risco
78000	9	Baixo risco

X0	X1	Classe
0.2	5.0	A
0.3	4.8	A
0.4	5.2	A
1.0	5.1	B
1.2	5.4	B
1.4	5.0	B
1.6	4.9	B
1.8	4.7	C
2.0	4.6	C
2.2	4.5	C





ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

$$\text{Gini} = 1 - \sum (p_i^2)$$

Índice de Gini para Árvores de Decisão

O índice de Gini em árvores de decisão é utilizado para medir a **impureza de um nó**, ou seja, **o quão misturadas estão as classes naquele conjunto de dados**. O algoritmo busca escolher, a cada divisão, a variável e o ponto de corte que minimizem a impureza, resultando em grupos cada vez mais **homogêneos**, onde **uma única classe tende a predominar**.

X[0] <= 0.8
gini = 0.666
samples = 100
value = [31, 35, 34]

value = [31, 35, 34]

N = 100

$$p_0 = 31/100 = 0.31$$

$$p_1 = 35/100 = 0.35$$

$$p_2 = 34/100 = 0.34$$

$$\text{Gini} = 1 - (0.31^2 + 0.35^2 + 0.34^2)$$

$$\text{Gini} = 1 - (0.0961 + 0.1225 + 0.1156)$$

$$\text{Gini} = 1 - 0.3342$$

$$\text{Gini} = 0.6658 \approx 0.666$$

X[1] <= 4.95
gini = 0.057
samples = 34
value = [0, 33, 1]

value = [0, 33, 1]

N = 34

$$p_0 = 0/34 = 0$$

$$p_1 = 33/34 \approx 0.9706$$

$$p_2 = 1/34 \approx 0.0294$$

$$\text{Gini} = 1 - (0^2 + 0.9706^2 + 0.0294^2)$$

$$\text{Gini} = 1 - (0 + 0.9421 + 0.0009)$$

$$\text{Gini} = 1 - 0.9430$$

$$\text{Gini} = 0.057$$

gini = 0.0
samples = 28
value = [0, 0, 28]

value = [0, 0, 28]

N = 28

$$p_0 = 0/28 = 0$$

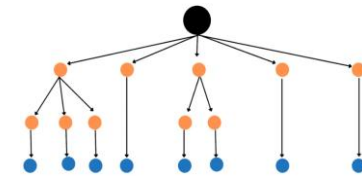
$$p_1 = 0/28 = 0$$

$$p_2 = 28/28 = 1$$

$$\text{Gini} = 1 - (0^2 + 0^2 + 1^2)$$

$$\text{Gini} = 1 - 1$$

$$\text{Gini} = 0.0$$



ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

Critério de divisão (split) – p/ Classificação

Entropia:

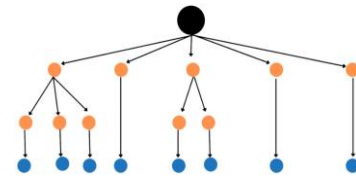
A entropia mede o nível de incerteza em um nó com base na distribuição das classes. Quanto mais equilibradas as classes, maior a entropia; quanto mais concentradas, menor. O algoritmo escolhe divisões que reduzam essa incerteza, maximizando o ganho de informação.

Entropia (Information Gain)

$$H = - \sum p_i \log_2(p_i)$$

$$IG = H_{pai} - \left(\frac{n_{esq}}{n} H_{esq} + \frac{n_{dir}}{n} H_{dir} \right)$$

- H_{pai}
→ entropia antes da divisão (nó atual)
- H_{esq}, H_{dir}
→ entropia dos nós filhos (após o split)
- n_{esq}, n_{dir}
→ número de amostras em cada filho
- n
→ total de amostras no nó pai
- $\frac{n_{esq}}{n}, \frac{n_{dir}}{n}$
→ peso de cada filho (proporção)



ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Critério de divisão (split) – p/ Regressão

MSE

O erro quadrático médio (MSE) mede a diferença entre os valores reais e previstos em problemas de regressão, penalizando mais fortemente erros maiores. É o critério mais comum em árvores de regressão, que buscam divisões que minimizem esse erro dentro dos nós.

MAE (alternativa)

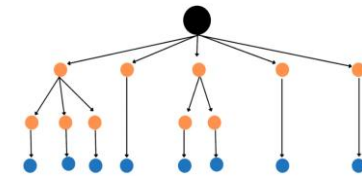
O erro absoluto médio (MAE) mede a diferença média em valor absoluto entre valores reais e previstos. Também pode ser utilizado como critério de divisão, sendo mais robusto a outliers, embora seja menos comum que o MSE.

MSE:

$$MSE = (1/N) \cdot \sum (y_i - \hat{y}_i)^2$$

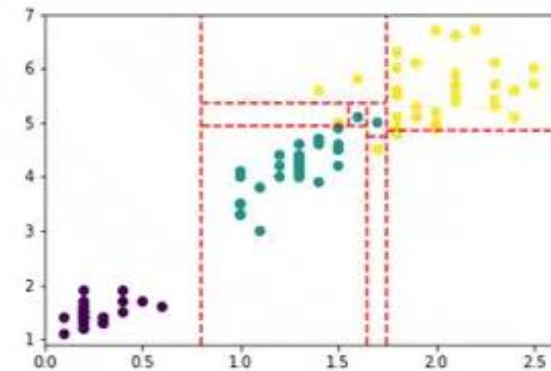
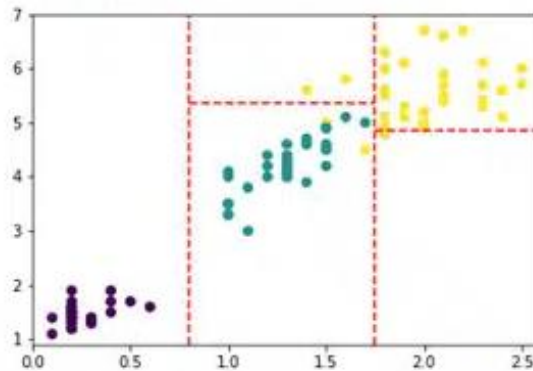
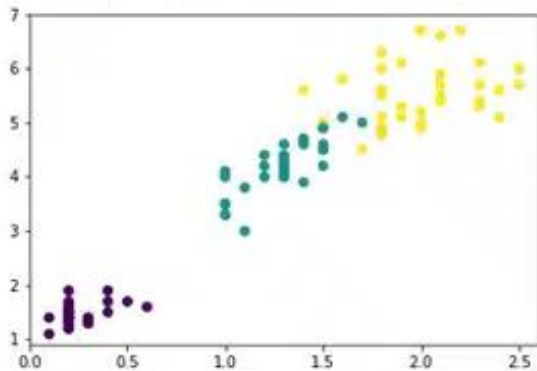
MAE:

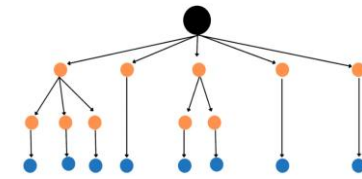
$$MAE = (1/N) \cdot \sum |y_i - \hat{y}_i|$$



ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

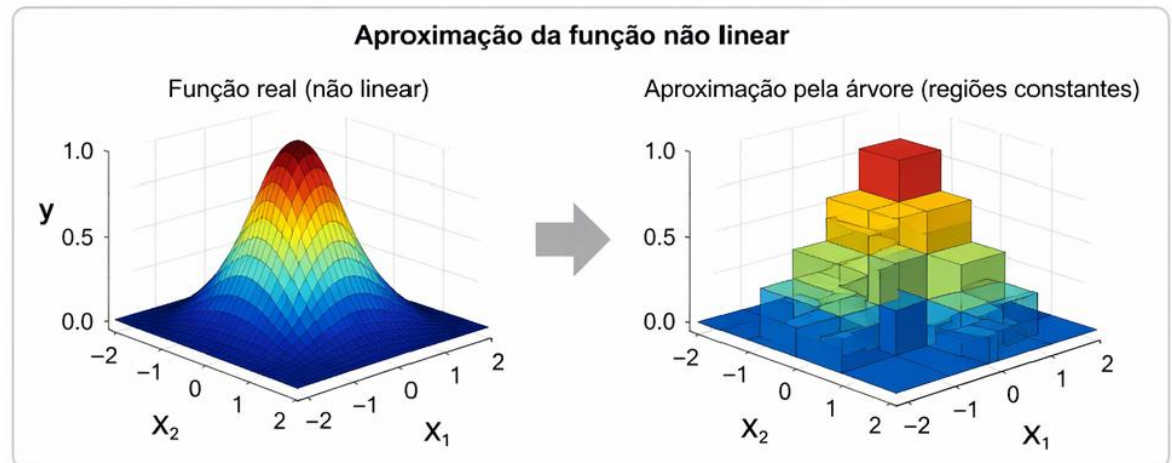
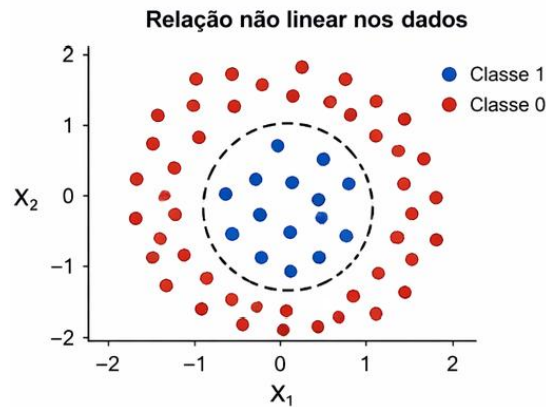
O modelo aprende essa estrutura diretamente dos dados, buscando particionar o conjunto de treino de forma a minimizar o erro nas previsões. Por ser **não linear**, a árvore consegue **capturar interações complexas** entre variáveis sem necessidade de transformações explícitas. Mas essa que é a sua maior força também é a característica que torna as árvores muito suscetíveis a overfitting.

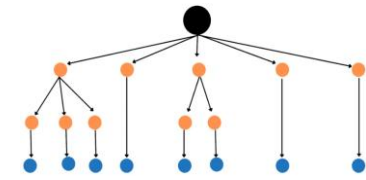




ESPECIFICAÇÃO DE MODELOS

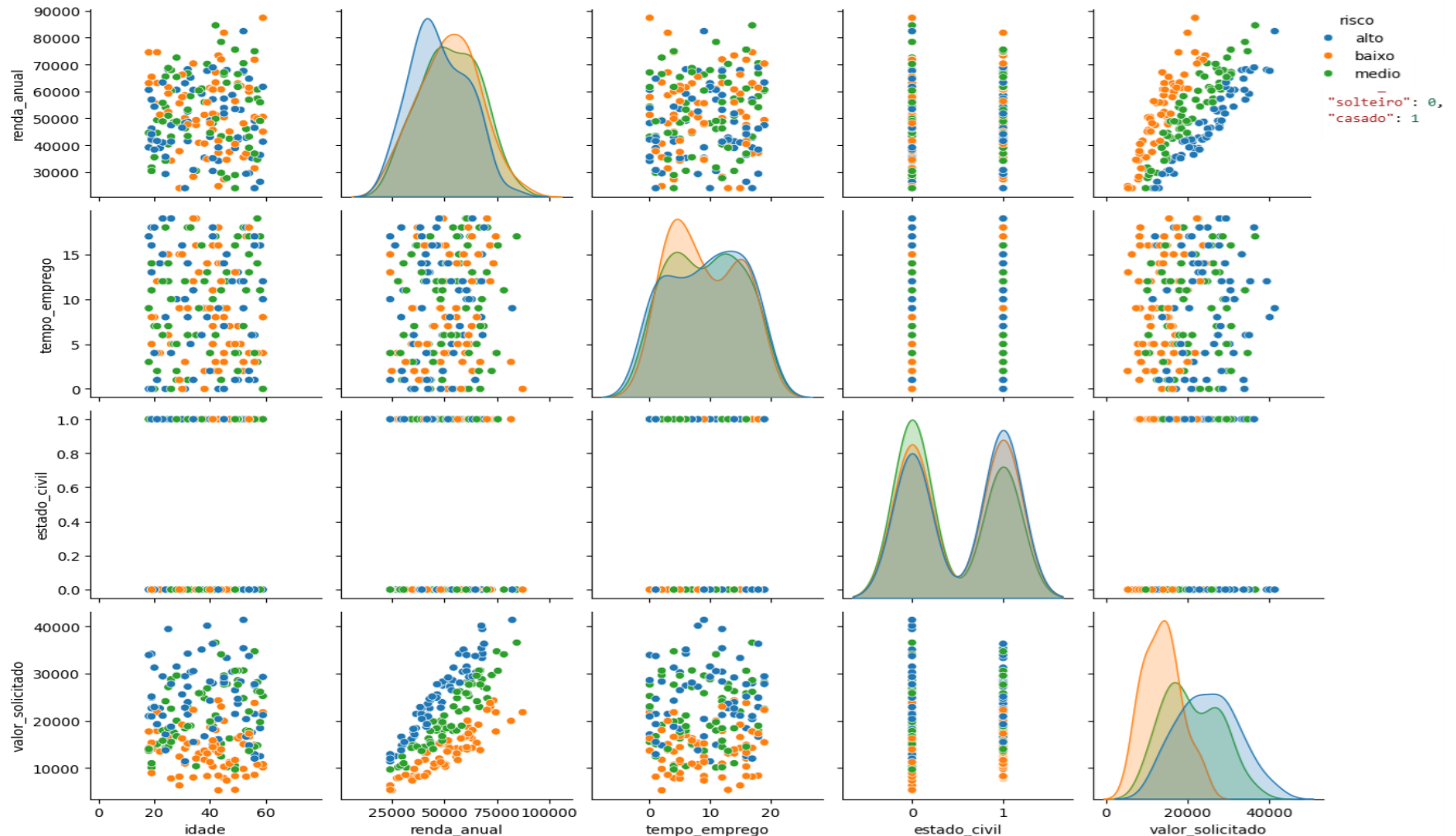
ÁRVORES DE DECISÃO

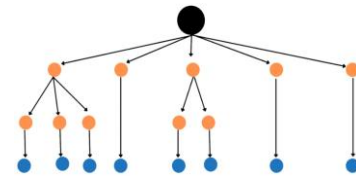




ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO





ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

Quando parar?

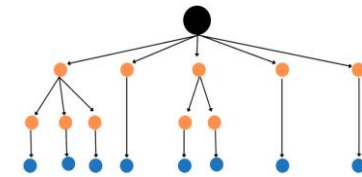
A construção da árvore deve ser interrompida para evitar modelos excessivamente complexos ou simplistas.

Para controlar o crescimento são utilizados critérios como:

- Profundidade máxima ($max_depth = None$)
- Mínimo de amostras em um nó para permitir uma divisão ($min_samples_split = 2$)
- Mínimo de amostras em uma folha após a divisão ($min_samples_leaf = 1$)
- Ausência de ganho relevante no split ($min_impurity_decrease = 0.0$)

Se a árvore cresce demais, tende a memorizar os dados de treino (**overfitting**); se cresce pouco, pode não capturar os padrões do problema (**underfitting**).

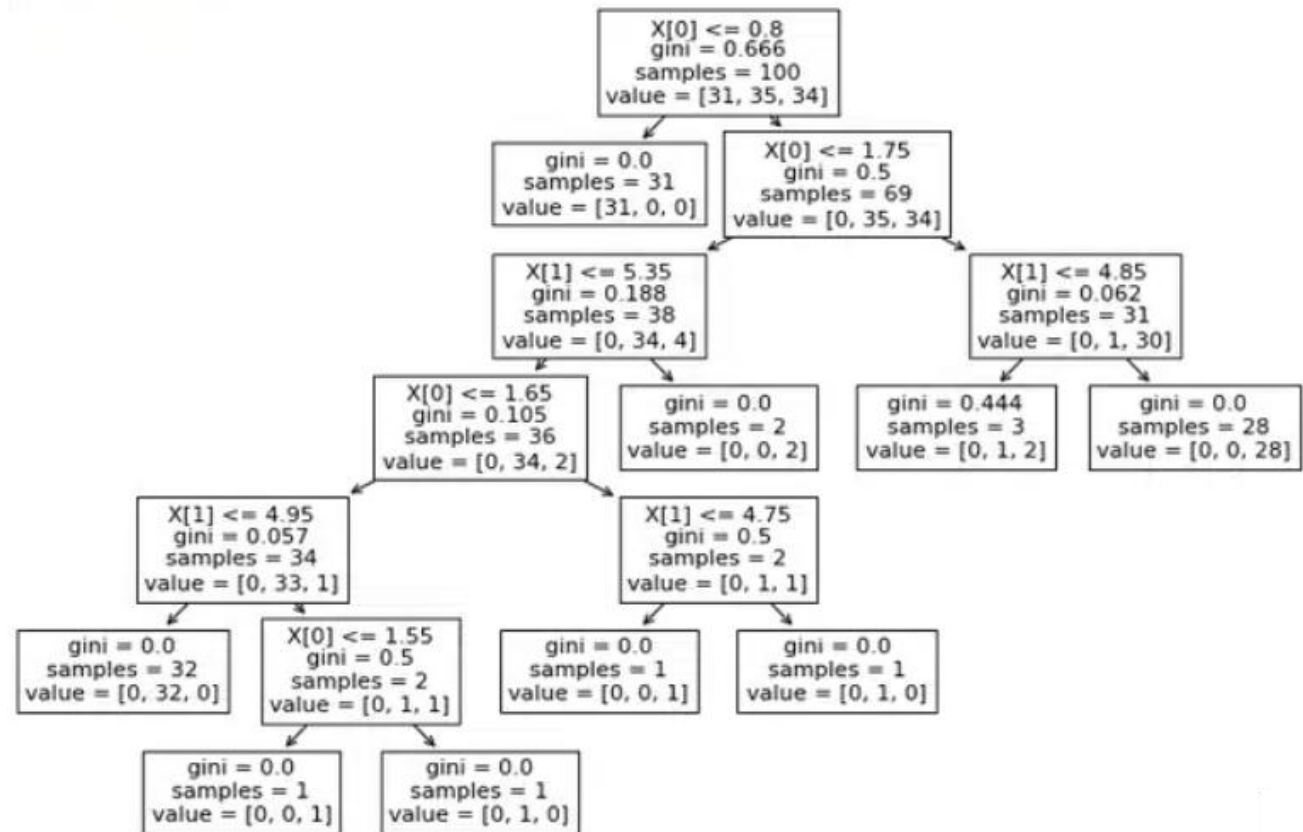
Sem controle de complexidade, árvores tendem a **overfitting** porque ajustam padrões reais e também o ruído dos dados.



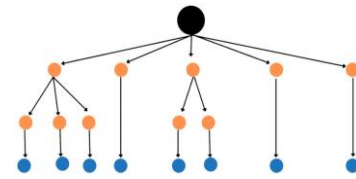
ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Quando parar?



Sem controle de complexidade, árvores tendem a **overfitting** porque ajustam padrões reais e também o ruído dos dados.



ESPECIFICAÇÃO DE MODELOS

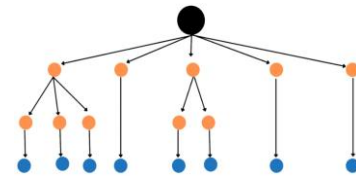
ÁRVORES DE DECISÃO

Principais Variações da Árvores de Decisão

CART (Classification and Regression Trees)

Algoritmo que constrói árvores sempre binárias (cada nó gera dois filhos), podendo ser utilizado tanto para **classificação** quanto para **regressão**. Utiliza índice de Gini (classificação) e MSE (regressão) como critério de divisão.

É a base atual de implementação em bibliotecas com scikit-learn.



ESPECIFICAÇÃO DE MODELOS

ÁRVORES DE DECISÃO

Principais Variações da Árvores de Decisão

ID3

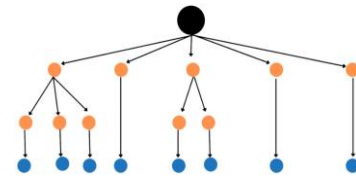
Algoritmo voltado apenas para classificação, que pode gerar **múltiplos filhos em um nó**. Utiliza entropia e ganho de informação (*information gain*) para definir as divisões.

C4.5

Evolução do ID3, também voltada apenas para classificação, com suporte a variáveis contínuas e tratamento de valores ausentes. Utiliza *gain ratio* como critério de divisão.

C5.0

Versão otimizada do C4.5, também voltada apenas para classificação, com maior eficiência computacional e melhor desempenho geral, mantendo o uso do *gain ratio*.



ESPECIFICAÇÃO DE MODELOS

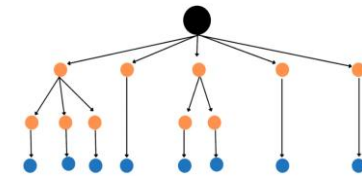
ÁRVORES DE DECISÃO

Limitações das Árvores de Decisão Isoladas

Árvores isoladas apresentam limitações importantes, principalmente relacionadas à **alta variância do modelo**. Pequenas variações no conjunto de treino podem resultar em árvores completamente diferentes, tornando o modelo instável. Além disso, tendem facilmente ao *overfitting* quando não são adequadamente controladas, ajustando-se ao ruído dos dados. Por fim, sua capacidade de generalização pode ser limitada, especialmente em cenários mais complexos, o que motiva o uso de métodos baseados em múltiplas árvores.

Principais problemas:

- Alta variância (instabilidade)
- Tendência ao *overfitting*
- Sensibilidade a pequenas mudanças nos dados de treinamento (causa)
- Baixa capacidade de generalização isoladamente (efeito)



ESPECIFICAÇÃO DE MODELOS ÁRVORES DE DECISÃO

[←](#) **customer_ltv** Machine learning ⓘ [Share](#)

sql-cluster-organiza...

RunsModelsTraces

MLflow 3 is available!
Featuring unified ML and GenAI experiment tracking, improved model logging, prompt versioning, enhanced LLM judges, advanced tracing for end-to-end agent observability, and beyond. [Learn more about ML features](#)
[| Learn more about GenAI features](#)

metrics.rmse < 1 and params.model = "tree"

Time created: Last 24 hours

State: Active

Datasets

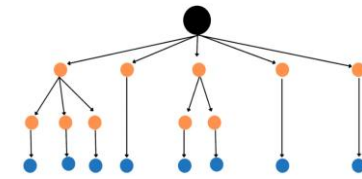
+ New run

Sort: r2_ltv_log_observado_vs_previsto

Columns

Group by

Metrics										Parameters
☐	Run Name	Created	Duration	corr_ltv_log_ob	mae_ltv_log	mean_ltv_log_obs	mean_ltv_log_pre	r2_ltv_log_obs	rmse_ltv_log	model
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	1.4min	0.73412536503...	1.762665498...	3.2137368216187...	3.204774166683161	0.538810151...	2.311936683...	Gradient Bo...
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	39.9s	0.72197271679...	1.864356761...	3.2137368216187...	3.212576785959885	0.517133314...	2.365645625...	RandomFor...
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	22.4s	0.71473442861...	1.807434567...	3.2137368216187...	3.2199801370353...	0.509034792...	2.385401151...	DecisionTree
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	1.2min	0.71473442861...	1.807434567...	3.2137368216187...	3.2199801370353...	0.509034792...	2.385401151...	DecisionTree
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	4.9min	0.71473442861...	1.807434567...	3.2137368216187...	3.2199801370353...	0.509034792...	2.385401151...	DecisionTree
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	16.3s	0.71473442861...	1.807434567...	3.2137368216187...	3.2199801370353...	0.509034792...	2.385401151...	DecisionTree
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	23.7s	0.68316107232...	1.915702145...	3.2137368216187...	3.2086307073430...	0.466675061...	2.486176815...	LinearRegre...
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	23.6s	0.68316107232...	1.915702145...	3.2137368216187...	3.2086307073430...	0.466675061...	2.486176815...	LinearRegre...
☐	lr_jo3_hp36_tr0.8_seed42	✓ 20 hours ago	8.4min	0.68316107232...	1.915702145...	3.2137368216187...	3.2086307073430...	0.466675061...	2.486176815...	LinearRegre...



ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

Ensemble (“Conjunto de Modelos”)

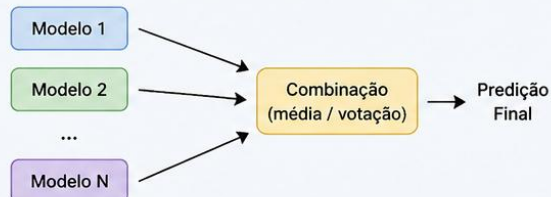
Técnica que combina **múltiplos modelos** para gerar uma única predição mais robusta, explorando a diversidade entre eles para reduzir erros individuais e melhorar a capacidade de generalização.

Bagging (Bootstrap Aggregating)

É um tipo de ensemble que treina vários modelos independentes em diferentes amostras dos dados (geradas por bootstrap) e combina suas previsões por média ou votação, com o objetivo principal de reduzir a variância e aumentar a estabilidade do modelo.

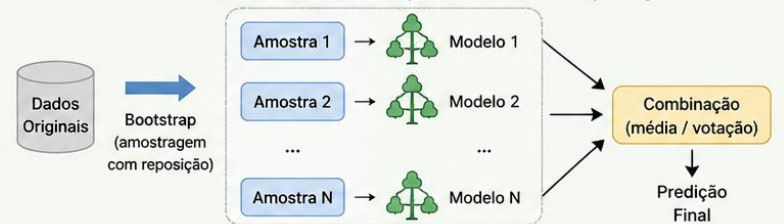
1. ENSEMBLE – Conceito Geral

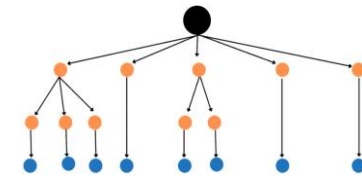
Ensemble é uma abordagem que combina múltiplos modelos para obter um resultado final mais robusto e com melhor capacidade de generalização.



2. BAGGING – Um tipo específico de Ensemble

Bagging (Bootstrap Aggregating) é um método de ensemble que treina vários modelos do mesmo tipo em diferentes amostras dos dados (bootstrap) e combina suas previsões.



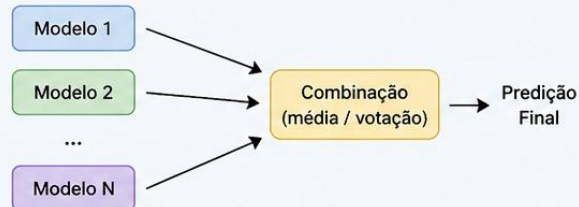


ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

Ensemble vs Bagging

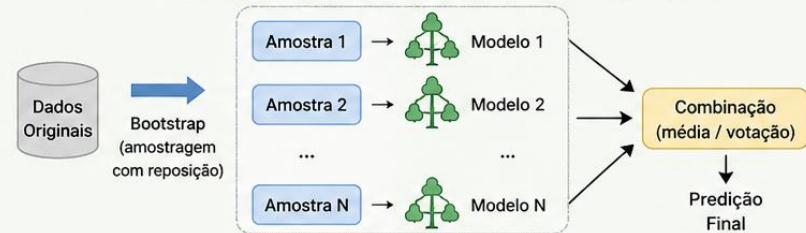
1. ENSEMBLE – Conceito Geral

Ensemble é uma abordagem que combina múltiplos modelos para obter um resultado final mais robusto e com melhor capacidade de generalização.



2. BAGGING – Um tipo específico de Ensemble

Bagging (Bootstrap Aggregating) é um método de ensemble que treina vários modelos do mesmo tipo em diferentes amostras dos dados (bootstrap) e combina suas previsões.



3. RELAÇÃO ENTRE ELES



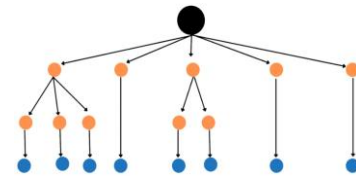
- Todo bagging é um ensemble.
- Nem todo ensemble é bagging.

Exemplos:

- ✓ Random Forest = Bagging (e também é um ensemble)
- ✗ Gradient Boosting = Ensemble, mas NÃO é bagging

4. COMPARAÇÃO RÁPIDA

Característica	Bagging (ex.: Random Forest)	Boosting (ex.: Gradient Boosting)
🎯 Como os modelos são treinados?	Em paralelo (independentes)	Em sequência (cada modelo corrige o erro do anterior)
📊 Como os dados são usados?	Amostras bootstrap + subconjunto de variáveis	Usa os mesmos dados, focando nos erros anteriores
📉 O que reduz?	Reduz principalmente a variância	Reduz viés (e também variância)
🏗️ Exemplos de algoritmos	Random Forest, Extra Trees, Bagging Classifier	Gradient Boosting, XGBoost, LightGBM, CatBoost

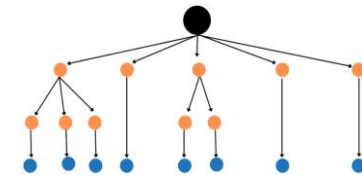


ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

Random Forest

Random Forest é um método que combina **múltiplas árvores** de decisão em **paralelo** para **reduzir a variância** e **melhorar a generalização**. Cada árvore é treinada a partir de uma **amostra aleatória dos dados** (*bootstrap*) e, em cada divisão, considera apenas um **subconjunto aleatório de variáveis**.

Essa aleatoriedade faz com que as árvores sejam diferentes entre si, e a predição final é obtida pela **agregação dos resultados** (voto majoritário em classificação ou média em regressão), tornando o modelo mais estável e robusto.



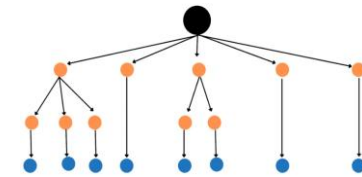
ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

🌳 Árvore de Decisão (passo a passo)

1. Começa com todo o dataset no nó raiz
2. Para cada variável:
 - testa vários pontos de corte
3. Para cada corte:
 - calcula o critério (Gini, entropia ou MSE)
4. Escolhe o **melhor split global**
5. Divide os dados
6. Repete recursivamente
7. Para quando atende critérios de parada
8. Predição: segue o caminho até a folha

🌳🌳🌳🌳 Random Forest (passo a passo)

1. Para cada árvore:
 - cria uma amostra bootstrap dos dados (linhas)
2. No nó raiz:
 - sorteia um subconjunto de variáveis
3. Para esse subconjunto:
 - testa pontos de corte
 - calcula o critério
4. Escolhe o **melhor split dentro do subconjunto**
5. Divide os dados
6. Repete recursivamente (sempre sorteando features)
7. Constrói várias árvores independentes
8. Predição:
 - classificação → voto majoritário
 - regressão → média



ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

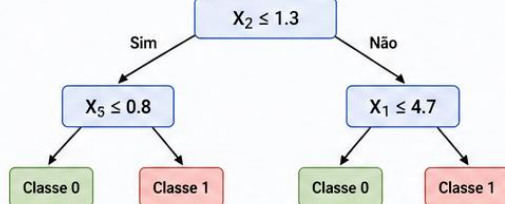
ÁRVORE DE DECISÃO (isolada)

USA TODOS OS DADOS E TODAS AS VARIÁVEIS

Dados de treino

ID	X1	X2	X3	...	X _p	y
1	5.2	1.1	3.3	...	0.5	0
2	3.1	0.7	1.4	...	1.2	1
3	4.0	1.5	2.0	...	0.7	0
...	...	...	...	...	...	...
n	6.1	2.2	3.1	...	1.1	1

Variáveis disponíveis
(em cada split)
 $\{X_1, X_2, X_3, \dots, X_p\}$
(Todas)



- Não usa bootstrap (usa todas as linhas).
- Em cada split, considera todas as variáveis.
- Resultado: uma única árvore.

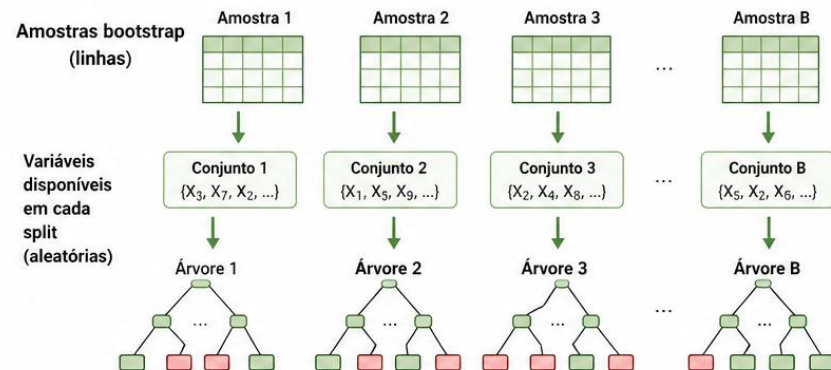
PREDIÇÃO



RANDOM FOREST (floresta aleatória)

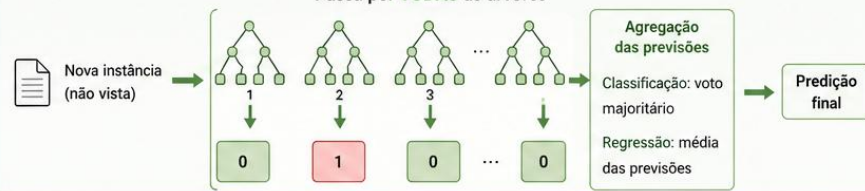
DUAS IDEIAS PRINCIPAIS

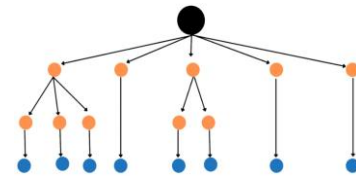
1. **Bootstrap das linhas:** cada árvore é treinada em uma amostra (com reposição) dos dados.
2. **Subconjunto aleatório de variáveis (max_features):** em cada split, considera apenas um subconjunto aleatório de variáveis.



- Cada árvore vê uma amostra diferente das linhas (bootstrap).
- Em cada split, só vê um subconjunto aleatório de variáveis.
- Resultado: várias árvores diferentes e pouco correlacionadas.

Passa por **TODAS** as árvores





ESPECIFICAÇÃO DE MODELOS FLORESTAS ALEATÓRIAS

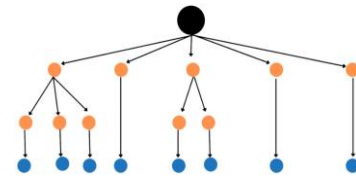
Limitações das Florestas Aleatórias

Embora reduza a variância e melhore a estabilidade em relação a árvores isoladas, as Random Forest ainda apresentam limitações. Por construir múltiplas árvores independentes e agregá-las por média ou voto, o modelo **não corrige erros sistemáticos (viés)**, podendo ter desempenho limitado em padrões mais complexos. Além disso, perde **interpretabilidade** em relação a uma única árvore e demanda **maior custo computacional**.

Características:

- RF reduz variação, mas repete viés, se existir
- Menor interpretabilidade
- Tem maior custo computacional
- As árvores são independentes (não aprendem com erros anteriores)

Se existe um viés estrutural nos dados, as árvores tendem a cometer os mesmos erros repetidamente.

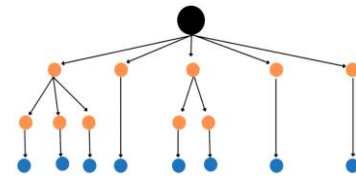


ESPECIFICAÇÃO DE MODELOS

GBT

Gradient Boosting

Gradient Boosting é um método que combina múltiplas árvores de decisão de forma sequencial, onde cada nova árvore é treinada para aprender os resíduos (erros) do modelo atual (foco no viés). Em vez de construir árvores independentes, o modelo aprende de maneira incremental, ajustando os resíduos a cada etapa. Esse processo permite reduzir principalmente o viés, tornando o modelo mais preciso em padrões complexos. A predição final é a soma das contribuições de todas as árvores.



ESPECIFICAÇÃO DE MODELOS

GBT

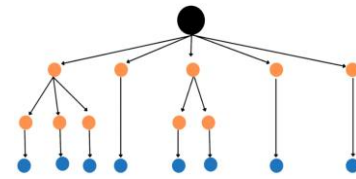
Gradient Boosting

Formalmente, Gradient Boosting se baseia em três ideias principais:

Aprendizado sequencial - Cada árvore depende das anteriores e é construída para melhorar o modelo atual.

Ajuste dos resíduos (erros) - As novas árvores são treinadas para prever os resíduos (erros) do modelo acumulado.

Atualização incremental do modelo - A predição final é construída como a soma das contribuições de todas as árvores.



ESPECIFICAÇÃO DE MODELOS

GBT

Gradient Boosting

Inicializa o modelo - Define uma previsão simples inicial (ex.: média dos valores, probabilidade das classes etc.).

Faz a primeira previsão - Usa esse valor inicial para prever todos os exemplos.

Calcula os resíduos (erros) - $\text{erro} = y_{\text{real}} - y_{\text{pred}}$.

Treina a primeira árvore - A árvore aprende a prever os resíduos (não o alvo original).

Gera a correção - A árvore produz ajustes para melhorar a previsão atual.

Aplica a taxa de aprendizado - Reduz o impacto da correção (learning_rate).

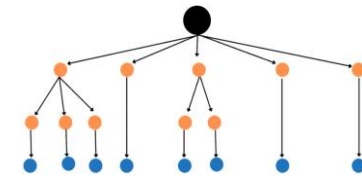
Atualiza a previsão - $\text{nova_pred} = \text{pred_anterior} + \text{correção}$.

Recalcula os resíduos - Baseado na nova previsão atualizada .

Repete o processo - Cada nova árvore aprende o erro restante .

Modelo final - Soma de todas as correções aprendidas.

Predição final - Resultado = soma das contribuições de todas as árvores .

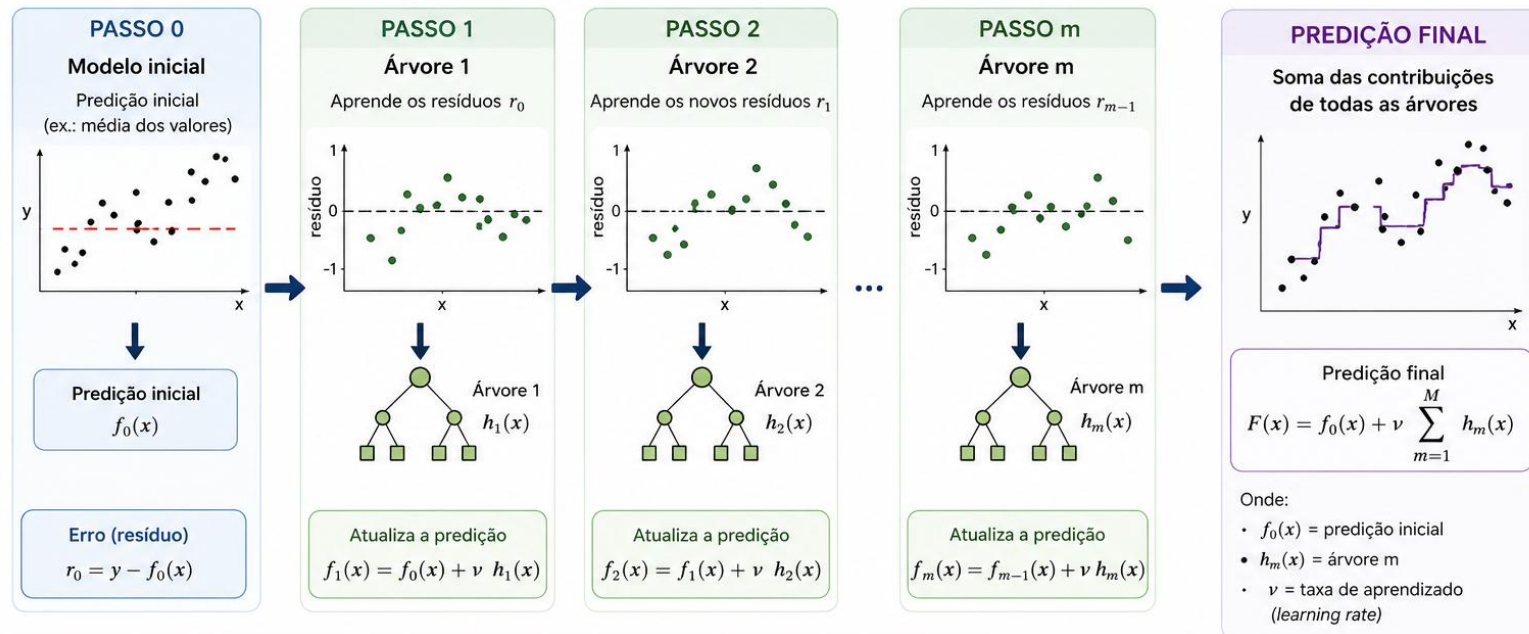


ESPECIFICAÇÃO DE MODELOS

GBT

Gradient Boosting (GBT)

Gradient Boosting é um método que combina múltiplas árvores de decisão de forma **sequencial**.
Cada nova árvore é treinada para aprender os **resíduos (erros)** do modelo atual (foco no **viés**).



AS TRÊS IDEIAS PRINCIPAIS

1. Aprendizado sequencial

Cada árvore depende das anteriores e é construída para melhorar o modelo atual.



2. Ajuste dos resíduos (erros)

As novas árvores são treinadas para prever os resíduos (erros) do modelo acumulado.



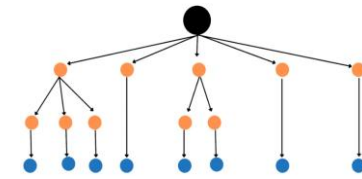
3. Atualização incremental do modelo

A predição final é construída como a soma das contribuições de todas as árvores.



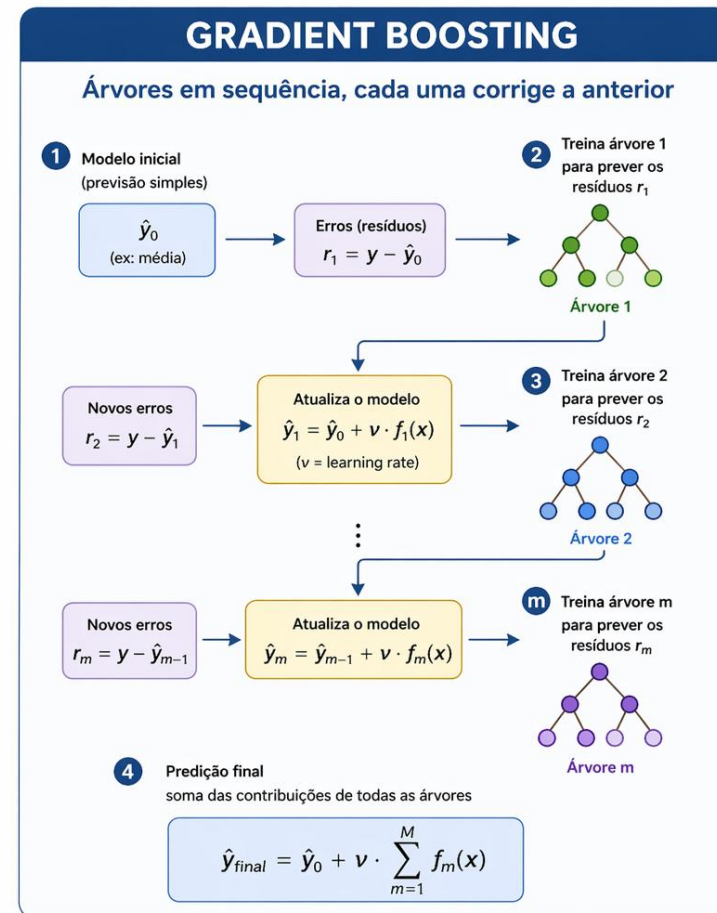
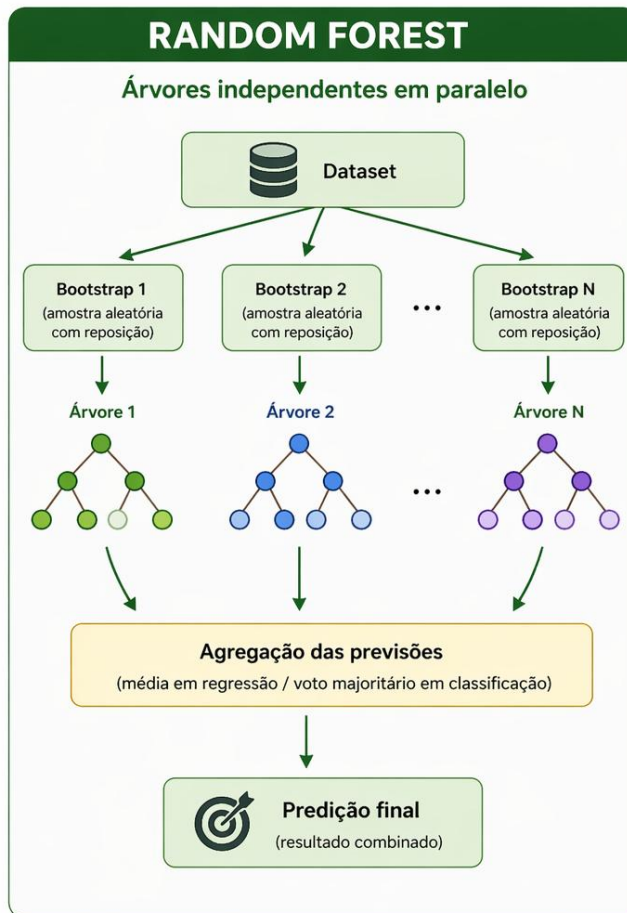
POR QUE FUNCIONA?

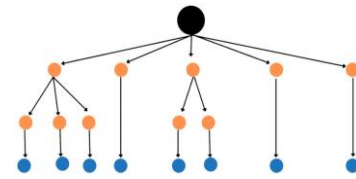
- ✓ Ao focar nos resíduos (gradientes da função de perda), o modelo corrige os erros de forma progressiva.
- ✓ Principalmente reduz o viés, tornando o modelo mais preciso em padrões complexos.



ESPECIFICAÇÃO DE MODELOS

GBT





ESPECIFICAÇÃO DE MODELOS

GBT

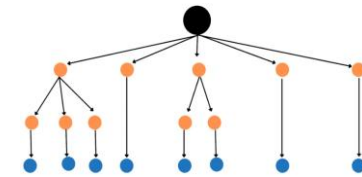
Gradient Boosting (clássico)

É o modelo sequencial que constrói **árvores pequenas** onde cada nova árvore tenta corrigir o erro da anterior usando gradiente do erro. É estável, mas não é otimizado para grandes volumes.

```
from sklearn.ensemble import GradientBoostingClassifier
```

A ordem histórica aproximada é:

1. AdaBoost (1995)
→ boosting clássico baseado em reponderação de erros.
2. Gradient Boosting (final dos anos 1990)
→ boosting baseado em gradientes/resíduos.
3. XGBoost (2014)
→ foco em performance, regularização e escalabilidade.
4. LightGBM (2017)
→ foco em velocidade e grandes volumes.
5. CatBoost (2017)
→ foco em variáveis categóricas.



BAGGING vs BOOSTING

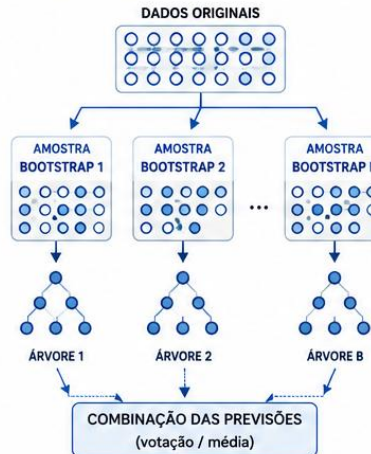
Duas formas de construir ensembles

BAGGING (Bootstrap Aggregating)

IDEIA PRINCIPAL: **reduzir variância**

- 1 Para cada modelo, é criada uma **amostra aleatória com reposição** (bootstrap) a partir dos dados.
- 2 Cada modelo é treinado de forma **independente**.
- 3 As previsões são combinadas (ex.: votação para classificação ou média para regressão).

MODELOS INDEPENDENTES
PODEM SER TREINADOS
EM **PARALELO**

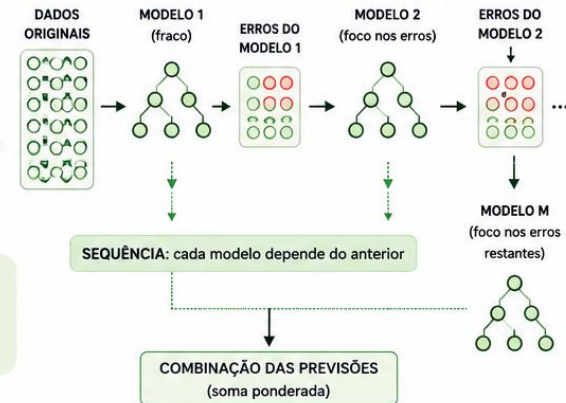


EXEMPLO CLÁSSICO: Random Forest
Bagging + aleatoriedade de variáveis

BOOSTING

IDEIA PRINCIPAL: **reduzir viés**

- 1 Os modelos são treinados **sequencialmente**.
- 2 Cada novo modelo foca em **corrigir os erros** do modelo anterior.
- 3 As previsões são combinadas (soma ponderada).



MODELOS DEPENDEM
UNS DOS OUTROS:
TREINAMENTO
SEQUENCIAL



EXEMPLOS CLÁSSICOS: Gradient Boosting,
XGBoost, LightGBM, AdaBoost

BAGGING

Modelos **independentes**

Treinamento **paralelo**

Reduz principalmente **variância**

Menos sensível a overfitting

Ex.: Random Forest



RELAÇÃO ENTRE MODELOS



FORMA DE TREINAMENTO



FOCO PRINCIPAL



RISCO DE OVERFITTING



EXEMPLO

BOOSTING

Modelos **dependentes**

Treinamento **sequencial**

Reduz principalmente **viés**

Mais sensível a overfitting (requer cuidado)

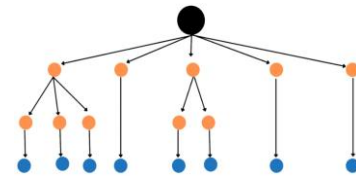
Ex.: Gradient Boosting (XGBoost, LightGBM)



EM RESUMO

Bagging = vários modelos aprendendo em paralelo com diferentes amostras dos dados. (ex.: Random Forest)

Boosting = modelos aprendendo em sequência, cada um corrigindo os erros do anterior. (ex.: GBT, XGBoost)



ESPECIFICAÇÃO DE MODELOS

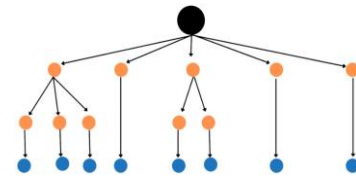
GBT

XGBoost (XG = eXtreme Gradient)

Versão otimizada (“extrema”) do Gradient Boosting que adiciona regularização, paralelismo e controle fino de overfitting.

É mais eficiente e geralmente entrega melhor desempenho em **problemas reais**.

```
from xgboost import XGBClassifier
```



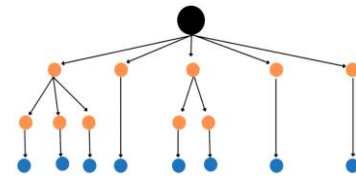
ESPECIFICAÇÃO DE MODELOS GBT

LightGBM

Implementação focada em velocidade e escalabilidade, usando histogramas e crescimento por folhas (leaf-wise). **Treina muito rápido em grandes datasets**, mas pode overfitar mais facilmente se não for bem ajustado.

○ **LightGBM** foi projetado especificamente para escala maior.

```
from lightgbm import LGBMClassifier
```



ESPECIFICAÇÃO DE MODELOS

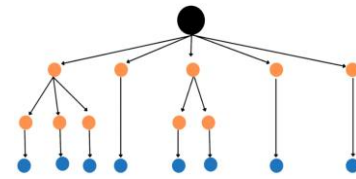
GBT

AdaBoost

Abordagem mais simples de boosting, onde cada modelo ajusta pesos das observações, dando mais importância aos erros anteriores.

É conceitualmente mais simples, mas geralmente menos potente que os métodos baseados em gradiente.

```
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier
```

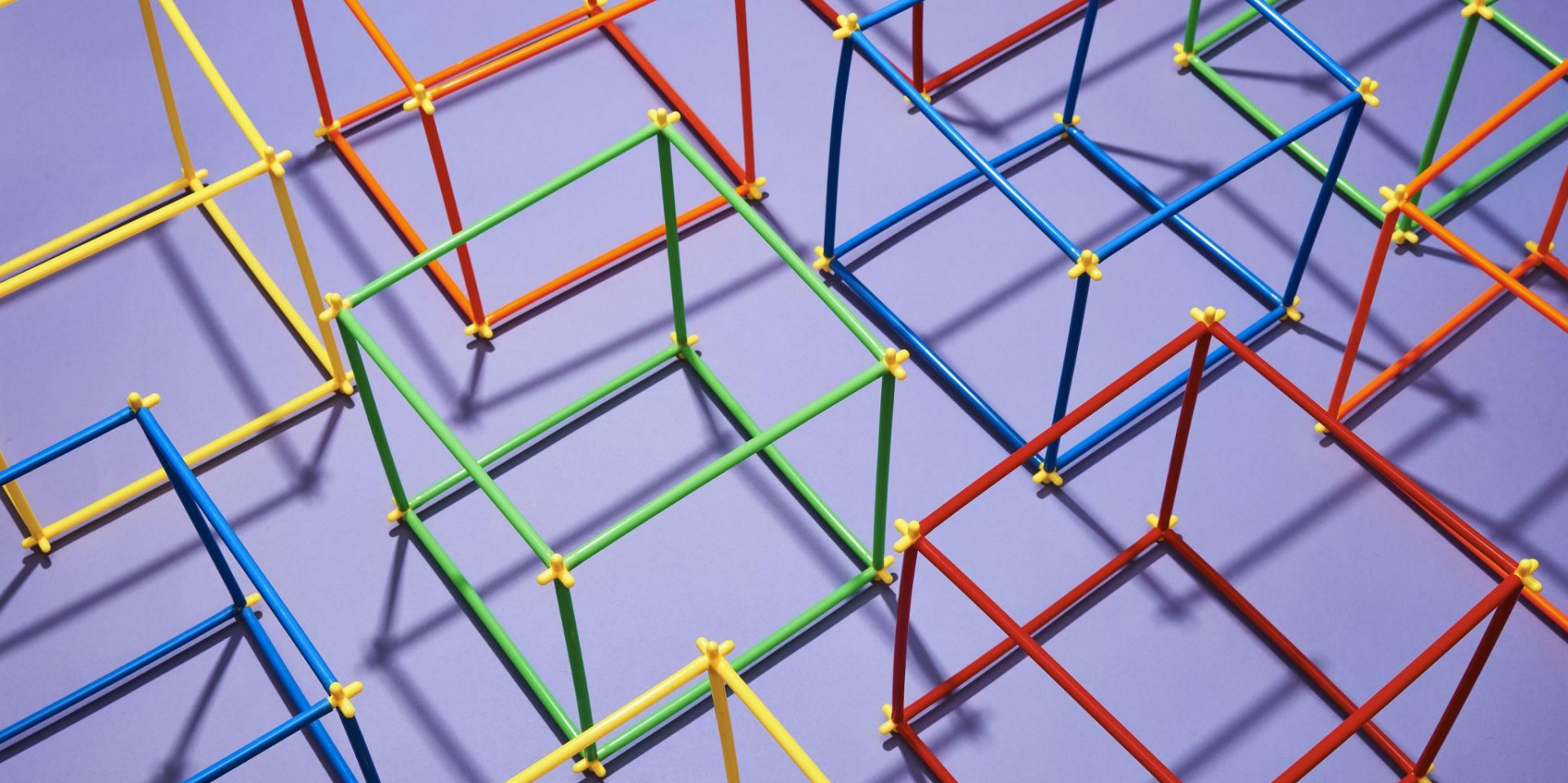


ESPECIFICAÇÃO DE MODELOS GBT

CatBoost

Desenvolvido para lidar naturalmente com **variáveis categóricas**, **evitando encoding manual**. Usa técnicas para reduzir overfitting e tende a performar bem em dados com muitas categorias.

```
from catboost import CatBoostClassifier
```

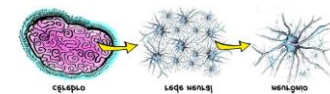


VI — REDES NEURAIIS ESPECIFICAÇÃO DE MODELOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica (PUC-Rio - 06)

Eng. de Computação (ITA-99)



ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

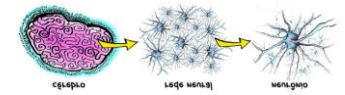
Neurônio Artificial

O neurônio artificial é a unidade básica de uma rede neural. Seu objetivo é receber valores de **entrada**, aplicar **pesos** para representar a **importância de cada entrada** e produzir uma **saída**.

Cada entrada é multiplicada por um peso associado. Os valores ponderados são somados juntamente com um termo adicional chamado bias. O resultado é então enviado para uma **função de ativação**, responsável por produzir a saída final do neurônio.

Em **análise preditiva**, cada instância do problema (linha da tabela) é representada por um conjunto de atributos de entrada ($x_1, x_2, \dots, x_n$) e a saída do neurônio (y) corresponde à previsão do modelo.

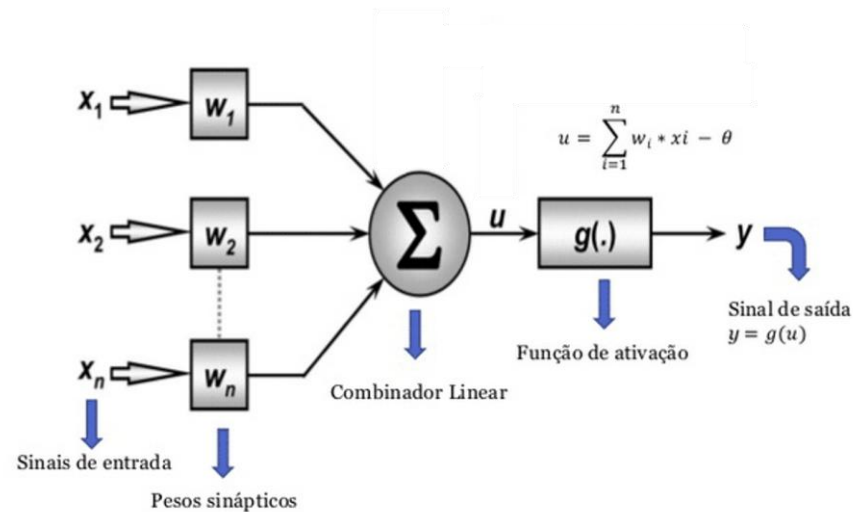
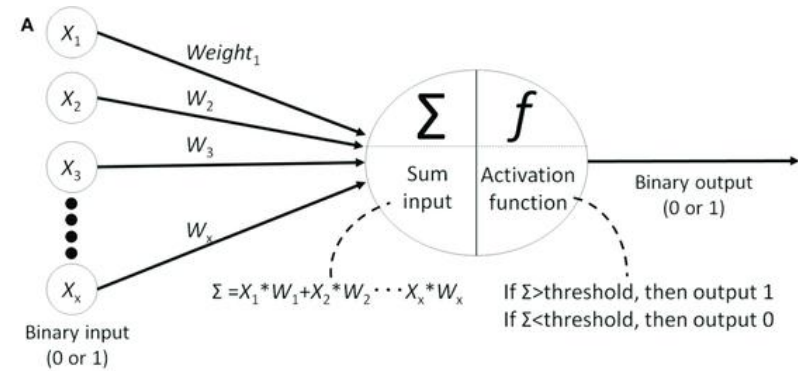
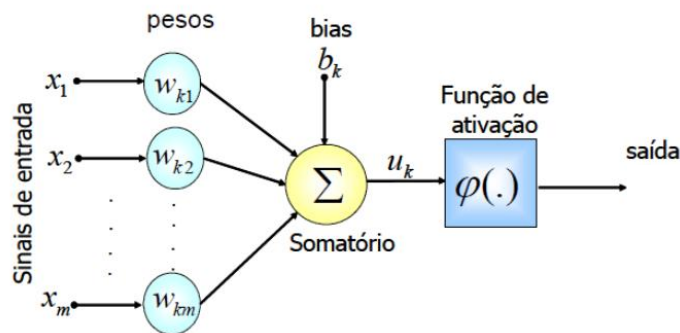
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
$$y = f(z)$$

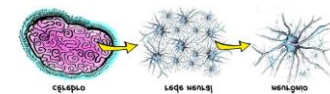


ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$y = f(z)$$





ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Neurônio Artificial - Unidade básica de uma rede neural responsável por receber entradas, processá-las e gerar uma saída numérica.

Entradas (x) – Em ANP, são os valores de entrada correspondentes às características (**atributos/features**) de cada instância da base de dados.

Pesos (w) - Parâmetros que definem a importância de cada entrada para a saída produzida pelo neurônio.

Bias (b) - Termo adicional utilizado para ajustar o ponto de ativação do neurônio, permitindo maior flexibilidade ao modelo.

Função de Ativação - Função matemática aplicada ao resultado do somatório para produzir a saída do neurônio.

Saída (y) - Resultado produzido pelo neurônio (**previsão**).

Neurônio Artificial aplicado a dados tabulares

1 Dados tabulares

	idade	renda	score	...	alvo (y)
1	25	3000	650	...	0
2	40	8000	700	...	1
3	35	5000	620	...	0
...	⋮	⋮	⋮	⋮	⋮
n	28	4500	680	...	1

Cada linha = uma instância (exemplo)

Cada coluna (exceto alvo) = uma característica

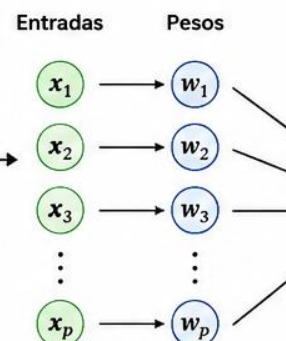
2 Após transformações / padronização

x_1	x_2	x_3	...	x_p	y
-0,45	-1,10	0,20	...	0,35	0
0,85	1,05	1,10	...	-0,15	1
0,30	-0,10	-0,30	...	0,60	0
⋮	⋮	⋮	⋮	⋮	⋮
-0,20	-0,60	0,40	...	0,10	1

Cada linha vira um vetor de entrada

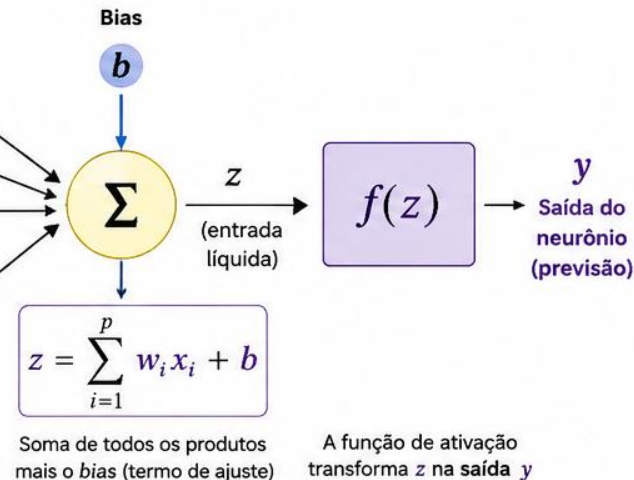
$$x = [x_1, x_2, \dots, x_p]$$

3 Entradas do neurônio (uma instância)



Cada característica x_i é multiplicada pelo seu peso w_i

4 Soma ponderada + bias



6 Saída é o que queremos prever

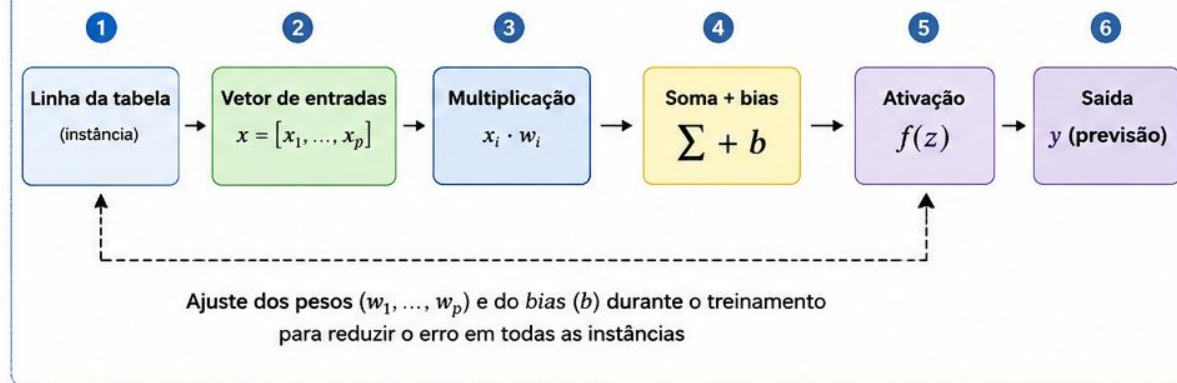
$\hat{y}$	y (real)
0,12	0
0,87	1
0,23	0
⋮	⋮
0,75	0

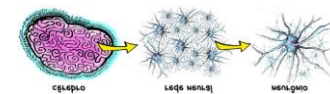
Comparação (erro/loss)

Onde cada elemento atua:

- Pesos (w_i): aprendem a importância de cada característica.
- Bias (b): desloca a função, permitindo ajustar o ponto de decisão.
- Função de ativação (f): introduz não linearidade e define o tipo de saída.

Resumo do fluxo de uma instância

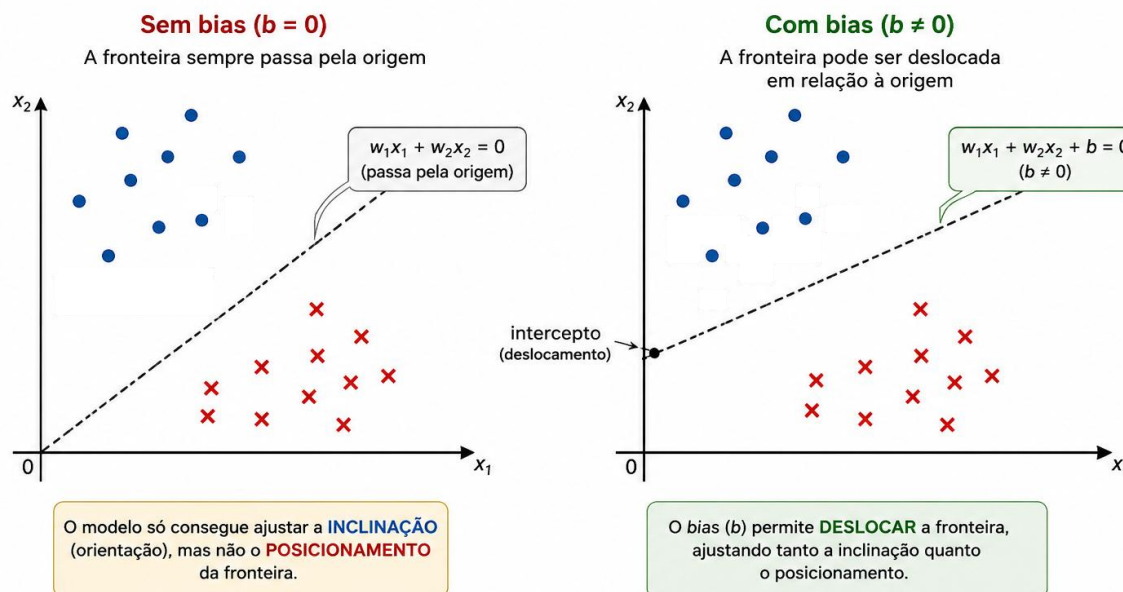


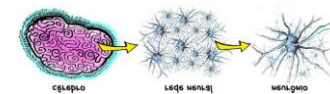


ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Bias

O **bias** desloca essa fronteira no espaço de entrada em relação à origem. Sem o bias, o modelo poderia aprender a orientação da fronteira (inclinação da reta, posição de um plano etc.), mas sempre passaria pela origem do espaço de entrada.





ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Função de Ativação

Função aplicada ao resultado do somador para produzir a saída do neurônio. Ela **recebe o valor linear** produzido pelo neurônio e aplicam uma **transformação não linear**, modificando o comportamento linear original. Em MLPs, ao combinar muitos neurônios com ativações não lineares, a rede neural consegue aprender padrões complexos e relações não lineares nos dados.

Sigmoid

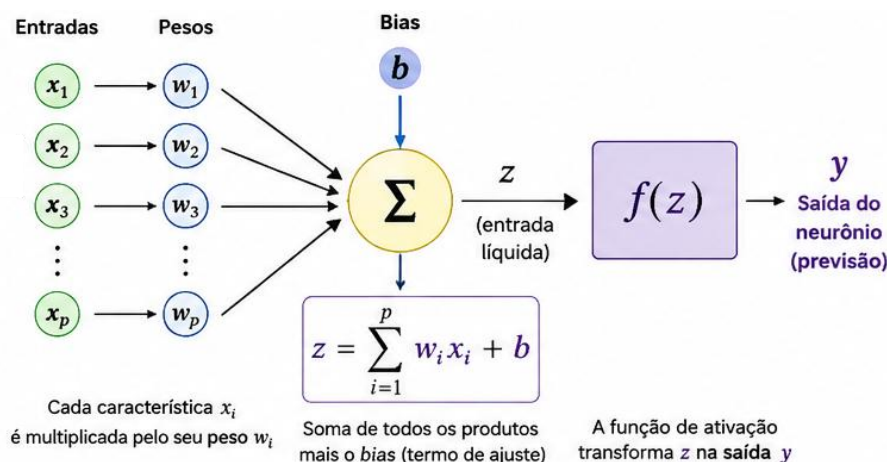
$$f(z) = \frac{1}{1+e^{-z}}$$

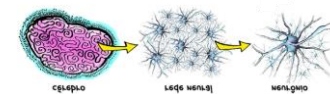
ReLU

$$f(z) = \max(0, z)$$

Tanh

$$f(z) = \tanh(z)$$





ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Função de Ativação

Sigmoid - Utilizada principalmente em classificação binária. Produz valores entre 0 e 1.

Sigmoid

$$f(z) = \frac{1}{1+e^{-z}}$$

Tanh (Tangente Hiperbólica) - Semelhante à sigmoid, porém produz valores entre -1 e 1.

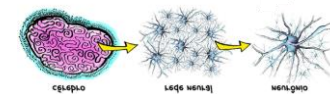
Tanh

$$f(z) = \tanh(z)$$

ReLU - Mantém valores positivos e transforma valores negativos em zero. É uma das funções mais utilizadas em redes profundas. Produz valores no intervalo $[0, \infty)$.

ReLU

$$f(z) = \max(0, z)$$



ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

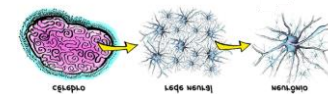
Perceptron Clássico de Rosenblatt

Modelo mais simples de **rede neural artificial**, baseado em um **único neurônio**, utilizado para realizar **classificação linear binária**.

O neurônio artificial é um conceito mais amplo, embora o perceptron apareça com frequência nos estudos por ter sido um dos primeiros modelos implementados. Tradicionalmente, o perceptron foi utilizado para tarefas de classificação linear binária, nas quais o objetivo é separar instâncias em duas classes, como **0 e 1**, **sim e não** ou **verdadeiro e falso**.

Usava a função degrau (*threshold/step*) como ativação.

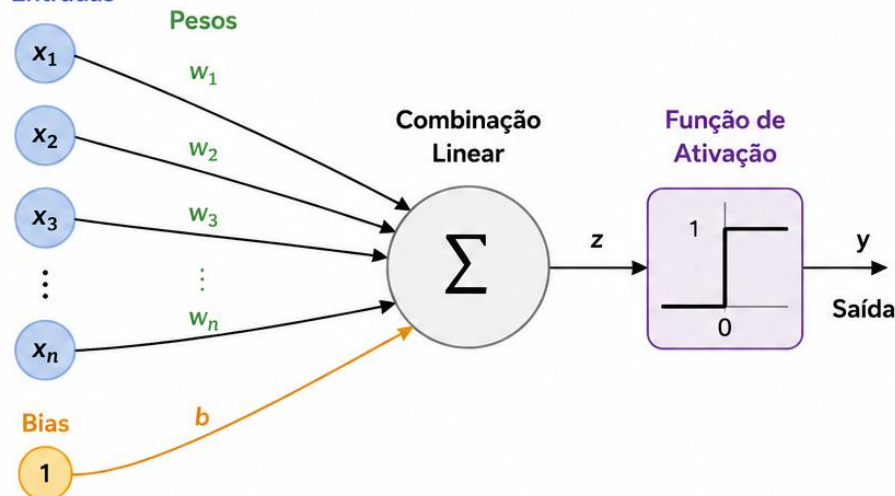




Perceptron Clássico

Modelo mais simples de rede neural artificial para classificação linear binária

Entradas



Combinação Linear:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

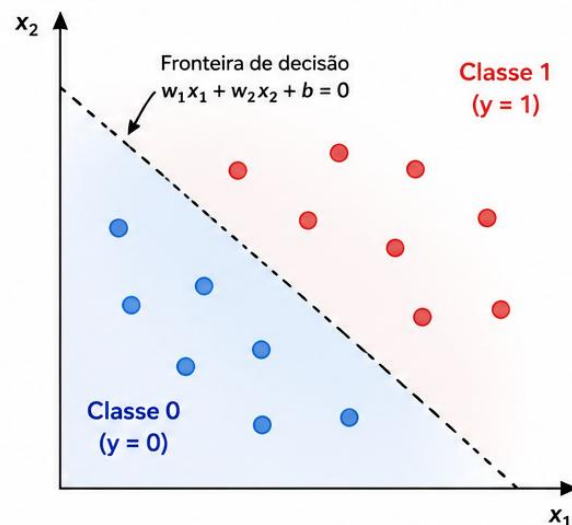
Função de Ativação (degrau):

$$y = f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

Onde:

x_i = entradas (atributos)
 w_i = pesos
 b = bias (termo de viés)
 f = função de ativação (degrau)
 y = saída (classe prevista)

Separação Linear no Espaço de Entrada (2D)

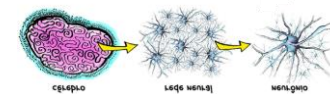


Interpretação da Saída

$y = 1 \rightarrow$ instância pertence à Classe 1

$y = 0 \rightarrow$ instância pertence à Classe 0

Resumo: O perceptron aprende os pesos ($w_1, w_2, \dots, w_n$) e o bias (b) a partir dos dados, de forma que a fronteira de decisão (linear) melhor separe as duas classes.



ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Perceptron - Limitado a Classificação Linear

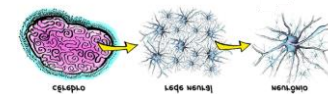
O perceptron realiza classificação linear, isto é, busca aprender uma fronteira de decisão capaz de separar duas classes por meio de uma reta (2D), plano (3D) ou hiperplano (múltiplas dimensões).

É linear devido a uma **combinação linear dos atributos de entrada**. Os pesos (**w**) controlam a influência de cada atributo na decisão do modelo e o bias (**b**) desloca a fronteira de decisão no espaço de entrada.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

A **fronteira aprendida** pelo perceptron corresponde ao ponto em que **$z = 0$** . Se o problema for linear, valores acima ou abaixo dessa fronteira levam o modelo a classificar a instância em classes diferentes.

Mas, se não for... O **XOR** virou historicamente um exemplo clássico da limitação do perceptron linear. **Não há fronteira que resolva o XOR!**



ESPECIFICAÇÃO DE MODELOS

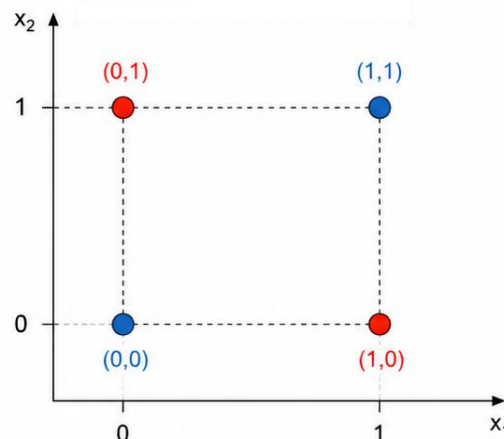
REDES NEURAIS

Problema XOR

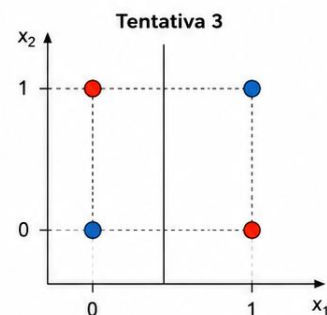
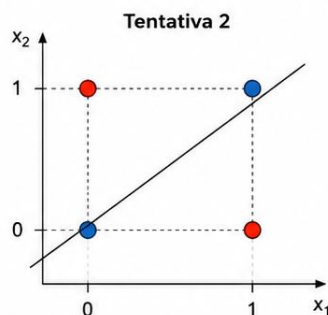
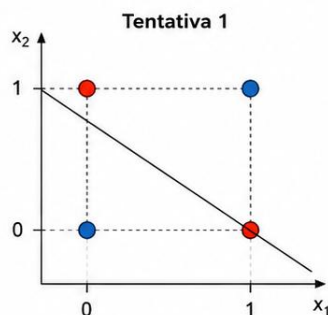
x_1	x_2	$x_1 \text{ XOR } x_2$ (saída)
0	0	0
0	1	1
1	0	1
1	1	0

● Classe 0 (saída = 0)

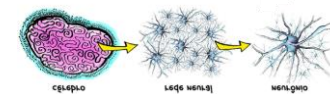
● Classe 1 (saída = 1)



Por que o perceptron (classificação linear) não resolve?



✗ Nenhuma reta (ou hiperplano) consegue separar as classes corretamente.



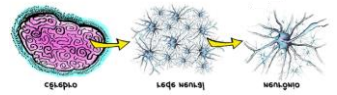
ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

MLP - Multilayer Perceptron

Embora o perceptron consiga aprender **fronteiras lineares** de decisão e solucionar muito problemas, há várias situações reais que se apresentam através de padrões mais complexos e **não linearmente separáveis**.

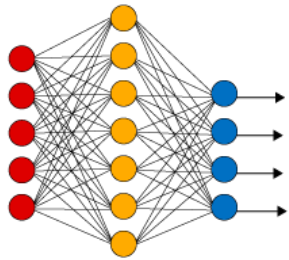
Essa limitação motivou o desenvolvimento das redes neurais multicamadas (*MLP - Multilayer Perceptron*), capazes de aprender tais representações mais complexas dos dados.

Diferentemente do perceptron simples, o **MLP utiliza camadas ocultas** e funções de ativação não lineares, permitindo aprender padrões mais complexos e não linearmente separáveis.

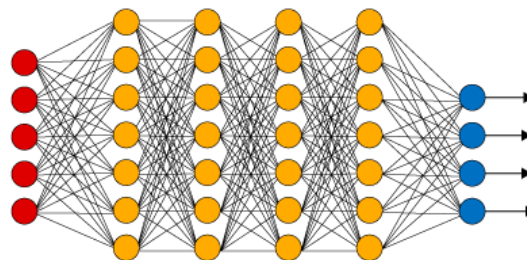


ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

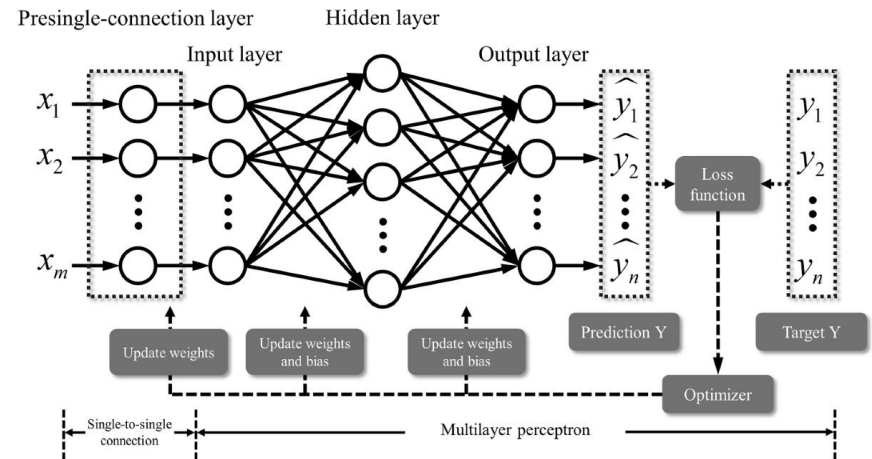
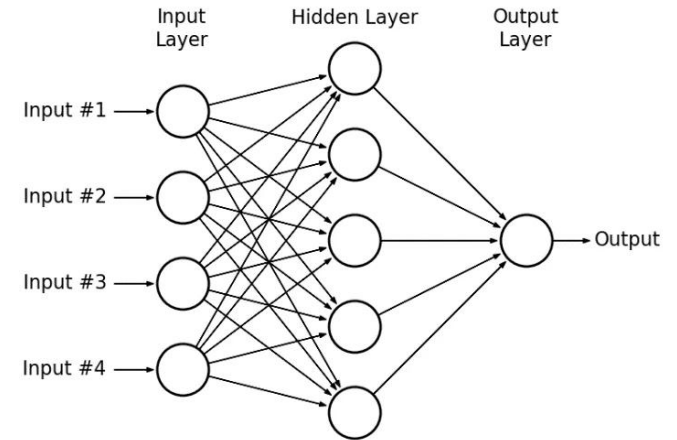
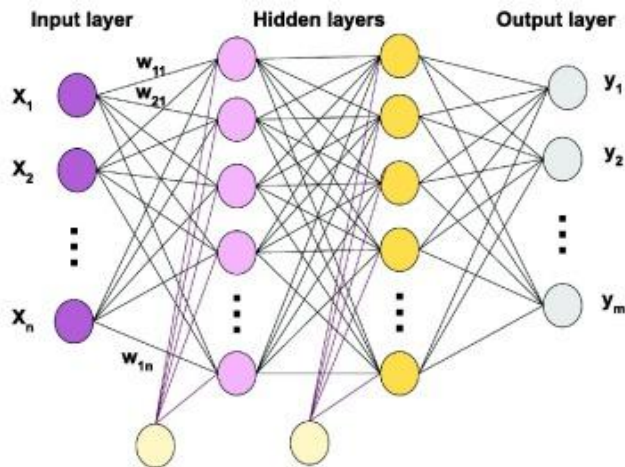
Simple Neural Network

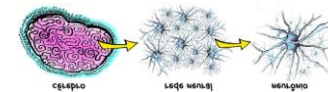


Deep Learning Neural Network



● Input Layer ● Hidden Layer ● Output Layer

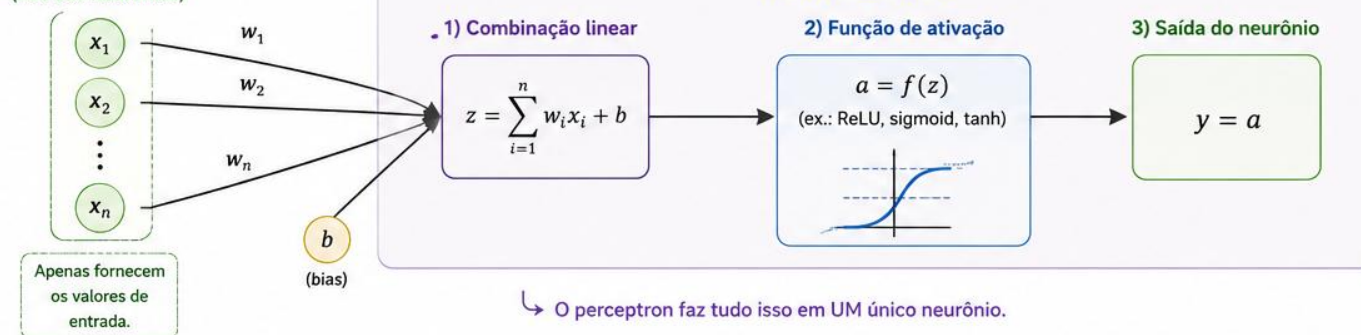




Do Perceptron ao MLP: entendendo os neurônios

PERCEPTRON (rede neural simples com 1 neurônio)

Camada de entrada
(não são neurônios)

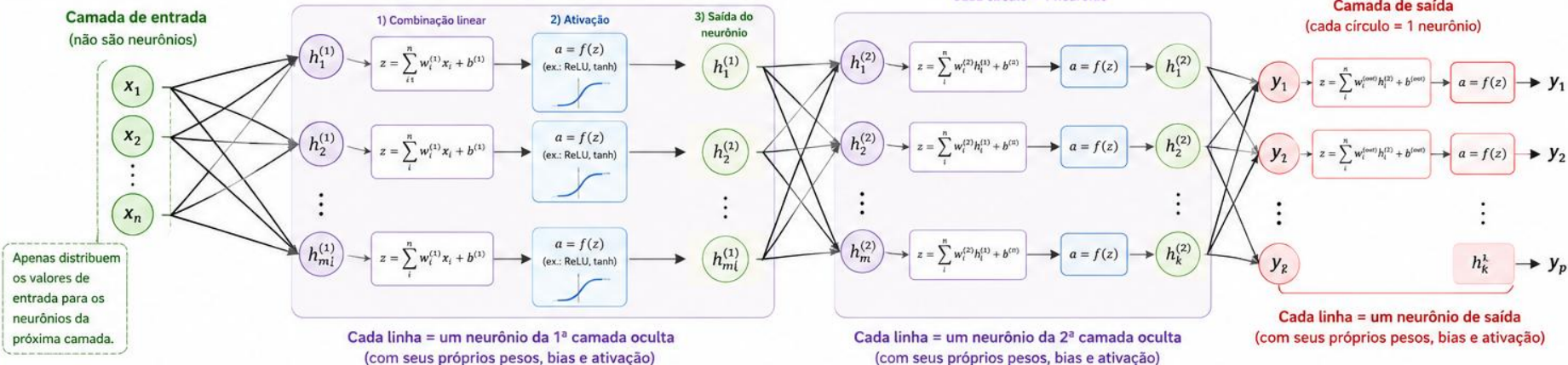


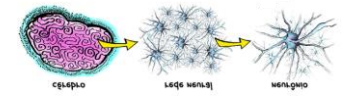
Resumo:

- 1 neurônio
- 1 conjunto de pesos ($w_1 \dots w_n$)
- 1 bias (b)
- 1 função de ativação
- 1 saída (y)

MLP (Multilayer Perceptron)

Camada de entrada
(não são neurônios)





ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

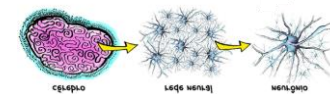
Redes Shallow

Redes Neurais Rasas (*shallow neural networks*) são redes compostas por uma camada de entrada, **uma única camada oculta** e uma camada de saída. Embora estruturalmente simples, essas redes já conseguem aprender padrões não lineares e resolver problemas que não podem ser tratados por modelos lineares simples, como o perceptron clássico.

Essa rede:

- possui vários neurônios na camada oculta;
- cada neurônio aprende uma transformação diferente;
- essas combinações permitem modelar padrões mais complexos.





ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Redes Shallow em Análise Preditiva

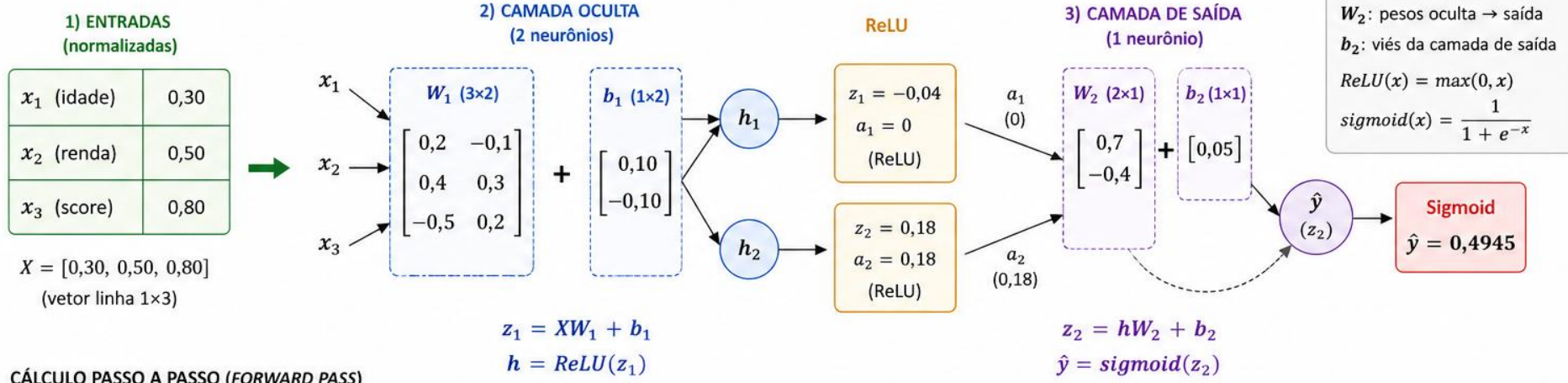
Em problemas de análise preditiva, cada linha da tabela representa uma instância contendo diversas características de entrada ($x_1, x_2, \dots, x_n$). O objetivo da rede neural é aprender uma função capaz de prever a saída y a partir dessas entradas.

Os valores de entrada são combinados pela matriz de pesos **W1**, que conecta as variáveis de entrada aos neurônios da camada oculta. Após a aplicação da função de ativação, a rede produz ativações intermediárias que são então combinadas pela matriz **W2** para gerar a saída final prevista.

Após a previsão, uma função de erro (*loss function*) **compara o valor previsto com o valor real esperado**. O erro obtido é utilizado para ajustar iterativamente os pesos das matrizes **W1** e **W2** durante o treinamento. O modelo treinado corresponde principalmente aos pesos aprendidos nessas matrizes, além dos biases.

EXEMPLO: REDE NEURAL RASA (COM BIASES)

3 ENTRADAS → 2 NEURÔNIOS OCULTOS → 1 SAÍDA



CÁLCULO PASSO A PASSO (FORWARD PASS)

1) Entrada (já normalizada)	2) Combinação linear na camada oculta: $z_1 = XW_1 + b_1$	3) Ativação (ReLU)	4) Combinação linear na camada de saída: $z_2 = hW_2 + b_2$	5) Ativação de saída (sigmoid)
$X = [0,30, 0,50, 0,80]$ Dimensão: (1 x 3)	2.1) Produto XW_1 $XW_1 = \begin{bmatrix} 0,30 & 0,50 & 0,80 \end{bmatrix} \begin{bmatrix} 0,2 & -0,1 \\ 0,4 & 0,3 \\ -0,5 & 0,2 \end{bmatrix}$ $= \begin{bmatrix} -0,14 & 0,28 \end{bmatrix}$ 2.2) Soma com bias b_1 $z_1 = \begin{bmatrix} -0,14 & 0,28 \end{bmatrix} + \begin{bmatrix} 0,10 & -0,10 \end{bmatrix}$ $z_1 = \begin{bmatrix} -0,04 & 0,18 \end{bmatrix}$	$h = \text{ReLU}(z_1) = \max(0, z_1)$ $z_1 = \begin{bmatrix} -0,04 & 0,18 \end{bmatrix}$ $h = \begin{bmatrix} \max(0, -0,04) & \max(0, 0,18) \end{bmatrix}$ $h = \begin{bmatrix} 0 & 0,18 \end{bmatrix}$	4.1) Produto hW_2 $hW_2 = \begin{bmatrix} 0 & 0,18 \end{bmatrix} \begin{bmatrix} 0,7 \\ -0,4 \end{bmatrix}$ $= 0(0,7) + 0,18(-0,4)$ $= -0,072$ 4.2) Soma com bias b_2 $z_2 = -0,072 + 0,05$ $z_2 = -0,022$	$\hat{y} = \text{sigmoid}(z_2)$ $= \frac{1}{1 + e^{-(-0,022)}}$ $= \frac{1}{1 + e^{0,022}}$ $\approx 0,4945$ $\hat{y} = 0,4945$

6) Função de perda (ex.: MSE)

Suponha que o valor real seja $y = 1,00$

$$\text{Loss}(\text{MSE}) = \frac{1}{2} (y - \hat{y})^2 = \frac{1}{2} (1,00 - 0,4945)^2 = 0,1275$$

Esse erro será usado no *backpropagation* para ajustar W_1 , b_1 , W_2 e b_2 .

RESUMO DO MODELO TREINADO

Os parâmetros aprendidos e armazenados são os pesos e vieses:

W_1 (3x2), b_1 (1x2), W_2 (2x1), b_2 (1x1)

Com esses parâmetros, o modelo pode prever novos valores de entrada.

$X = [0,30 \quad 0,50 \quad 0,80]$

$W1 =$
 $\begin{bmatrix} 0,2 & -0,1 \\ 0,4 & 0,3 \\ -0,5 & 0,2 \end{bmatrix}$

$b1 = [0,10 \quad -0,10]$

Produto:

$XW1 = [$
 $0,30(0,2) + 0,50(0,4) + 0,80(-0,5),$
 $0,30(-0,1) + 0,50(0,3) + 0,80(0,2)$
 $]$

$XW1 = [-0,14 \quad 0,28]$

Soma do bias:

$z1 = XW1 + b1$
 $z1 = [-0,14 \quad 0,28] + [0,10 \quad -0,10]$
 $z1 = [-0,04 \quad 0,18]$

Ativação ReLU:

$h = \text{ReLU}(z1)$
 $h = [0 \quad 0,18]$

Agora saída:

$W2 =$
 $\begin{bmatrix} 0,7 \\ -0,4 \end{bmatrix}$

$b2 = 0,05$

Produto:

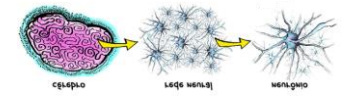
$hw2 = 0(0,7) + 0,18(-0,4)$
 $hw2 = -0,072$

Soma do bias:

$z2 = hw2 + b2$
 $z2 = -0,072 + 0,05$
 $z2 = -0,022$

Sigmoid:

$\hat{y} = \text{sigmoid}(-0,022)$
 $\hat{y} = 1 / (1 + e^{0,022})$
 $\hat{y} \approx 0,4945$



ESPECIFICAÇÃO DE MODELOS REDES NEURAIS

Laboratório

construir perceptron/MLP simples;

classificação tabular;

observar efeito das ativações;

observar fronteira de decisão;

comparar rapidamente com regressão logística.